Google study path: code only

288 questions in 43 groups, in the same order and with the same numbers as the study path book. Under each question there is only the code you write for the main question: the helper functions first, then the main answer. The brute force, the follow-up code and the tests are not here (they are in the study path book). Every group in the study path starts with its code template, so use that book to learn the pattern and this one to drill the solutions.

1. Frontier search (BFS or DFS), Tier 1: spread, count and measure distance (9)

1. Flood Fill
2. Number of Islands
3. Rotten Oranges / Multi-Source BFS (taxis)
4. Solve Grid and Counting Problems
5. Find shortest relationship path using BFS
6. Farthest City for a New Data Center
7. Compute distance to nearest taxi in grid
8. Infection Spread on a Grid (Cellular Automaton)
9. Shortest Distance From All Buildings

2. Sweep over events in time order (9)

10. Meeting Rooms / Interval Scheduling Variants
11. Train / Flight Schedule Reachability
12. Solve Rooms and Top-K Streams
13. Compute servers needed for daily recurring jobs
14. Gate Entry / Exit Time Resolution with Tie-Break Rules
15. Meeting Rooms III Variant
16. Count Overlapping Rectangle Updates on a Grid
17. Compute city skyline outline
18. Compute Turnstile Crossing Times

3. Tree recursion, Tier 1: one function that visits a node, asks its children, and reports upward (9)

19. Given two trees, determine if the pre-order traversals of two trees are same or not.
20. Calculate the height of a binary tree.
21. Find right-side view of binary tree
22. Expression Tree Single-Leaf Mutation
23. Longest Consecutive Path in Binary Tree
24. Given the root of a binary tree, return the length of the diameter of the tree.
25. Implement zigzag level-order traversal
26. Repeatedly Delete Leaves Matching a Target Value From a Binary Tree
27. Delete an Entire Binary Tree

4. A heap of what is available: pop the best, put back the changed one (9)

28. Streaming Median (LC 295) with Follow-ups
29. Compute minimum number of rooms needed
30. Implement a Reusable Locker Allocation System
31. Simulate room assignments for scheduled meetings
32. Trip Expense Settlement
33. Playlist Shuffle No Adjacent Artists
34. Minimum Makespan on Identical CPUs
35. Design server allocation for multi-type nodes
36. Huffman Tree Construction

5. Recurrence on a position or a range (memo or table) (9)

37. Target Expressions from Ordered Digits
38. Count Word Decompositions into Chemical Element Symbols
39. Find the longest palindromic subsequence using dynamic programming.
40. Jump Game with Profit
41. Fence Painting Minimum Operations (Gemini Team)
42. Maximize 1D deliveries within distance and minimize total distance
43. Edit distance
44. Count Perfect Wake Numbers up to N
45. Word Break: All Sentences

6. Best-first search with a heap: Dijkstra and its minimax twin (9)

46. Path Minimizing the Maximum Cell (Swim in Rising Water)
47. Nearest Favorite City by Travel Time
48. Given a train schedule with departure city, arrival city, start time, end time, and duration, and given a source and destination, find the sequence of trains that gets you there at the earliest possible time.
49. Find the shortest path in a graph.
50. Given a matrix with a rat, bread, cat, and obstacles, find a path from the rat to the bread that maximizes the minimum distance from cat.
51. Minimize maximum height along a grid path
52. Find minimum threshold enabling grid path
53. Minimize travel time with optional meeting point
54. Compute minimax grid path and network delay

7. Backtracking: choose, recurse, undo (9)

55. Mahjong Winning Hand Detection (14 Tiles)
56. Count Prefix-Matching Simple Paths in a Labeled Graph
57. Generate Parentheses
58. American Football Score Combinations
59. Permutations
60. Determine if a 14-tile hand is winning
61. Count Passcode Patterns on a Keypad
62. Match people to questions using tags and priorities
63. Matrix Flower Placement with House Adjacency

8. Runs and groups: extend while it fits, restart or close when it does not (9)

64. Group Timestamps Into Close Intervals
65. First Out-of-Order Character
66. Longest Non-Decreasing Subarray with One Replacement
67. Piano Hand Movement Grouping
68. Find longest increasing contiguous subarray
69. Sum of Good Subarrays (Adjacent Diff ±1)
70. Sustained-Error Time Window
71. Longest Non-Decreasing Subarray From Two Arrays
72. Count Super Streaks

9. Union-find: merge the pairs, then ask who is together (8)

73. Count Islands in a Grid and Track the Count as Land Cells Are Added
74. 2D Points Connected Components by Radius
75. Count Clusters of 2D Points Within a Radius
76. Find A Threshold-Limited Path With Minimum Required Safety
77. Movie Recommendations via Transitive Similarity
78. Determine Reporting Relationships
79. Determine whether two nodes are related
80. Friend Graph with Timeline (Friend / Unfriend Events)

10. A dict of per-key state that you update, expire, or rank (8)

81. Robot Status Message Deduplication Window
82. Active Communication Top Users
83. Filter Repeated Robot Status Messages
84. Next-Word Predictor
85. Batch and Streaming Log Deduplication
86. Design a logger that returns top-K messages
87. Hit Counter
88. Website Activity Analytics: Unique Users + Session Time

11. Binary search on a yes/no condition (8)

89. First Index Reaching a Threshold (Binary Search)
90. Cutting Ribbons (LC 1891)
91. First Bad Version with Parallel Search Follow-up
92. Solve Banana Speed and Interval Merge
93. Given a list with response time for each server, calculate how long time it will take to process n requests.
94. Median of Two Sorted Arrays (LC 4) with a Follow-up Variant
95. Find the median of two sorted arrays.
96. Find longest duplicated substring

12. Greedy: scan once, carry the best choice so far, act only when forced (8)

97. Maximum Profit

98. Alarm for the Next Upcoming Birthday
99. Coins on the Table
100. Visible People in a Line (Last Person, Then Everyone)
101. Bounded Knapsack Variant
102. Find the Largest Monotone Increasing Number
103. Min boards to cover roof holes
104. Jump Game II (Minimum Jumps)

13. Sets and counters: remember the first collection, check the second against it (8)

105. Exclude First-K-Unique From Another List
106. Find the Duplicates
107. Positional Matches Between Two Lists
108. Contains Duplicate
109. Filter Common Elements Per Window (Two Lists)
110. Reconstruct original array from doubled shuffle
111. Min deletions to avoid overlap in first k
112. Smallest Missing Positive Integer in Linear Time and Constant Extra Space

14. bisect on a sorted list: find a position, count a range, keep it sorted (8)

113. Count People in a Postal Code Range
114. Sliding-Window Mean After Removing the Largest K Values
115. Russian Doll Envelopes
116. Count Sorted Strings With a Given Prefix
117. Streaming Insert + Find K-th Largest
118. Given positions of N kids and M ice cream sellers on a 1D line, find the smallest distance E such that every kid lies within the range [P - E, P + E] of at least one seller.
119. Find Maximum Envelope Nesting
120. Count of Smaller Numbers After Self

15. Prefix sums, Tier 1: precompute once, answer ranges and balance points in O(1) (8)

121. Parity Alternating Subarray Query
122. Cake Balance Line
123. Find the Pivot Index (Balanced Split) in O(n)
124. Range Duplicate Check with Constant-Time Queries
125. Find a Horizontal Cut That Bisects Rectangle Area
126. Check pivot after removing one element
127. Gold Chain Split into Two Equal Halves
128. Count the number of ways to split a string.

16. Sorted intervals: merge while they touch, then read the gaps or search (8)

129. Check an Interval Against Existing Intervals for Conflicts
130. Find Containing Range
131. Union of Sorted Interval Lists
132. Range Module (Insert and Point Query)
133. Meetings Minus a Do-Not-Schedule Block
134. Merge Intervals
135. Find safe travel intervals between planet influences
136. Implement a function that processes a stream of interval data.

17. Trie: nested dicts, one character per level (8)

137. URL Request Handler Router
138. Trie Prefix Search / Autocomplete
139. Build Prefix Lookup with a Trie
140. Greedy Longest-Match Dictionary Tokenizer
141. Check if all substrings are dictionary words
142. Return dictionary words matching a prefix
143. Implement Trie Insert and Exact-Word Search
144. Implement Longest-Match Text Replacement

18. A dict for lookup plus an ordered container for who is next (lazy deletion) (8)

145. Implement Batched Undo/Redo Layer
146. Design a restaurant waitlist system
147. Implement Python's itertools.tee
148. Store With Cheapest-Item Removal
149. Round-Robin Job Scheduler
150. Aggregate FizzBuzz Offer Matches

151. Restaurant Waitlist: join / delete / find_first_match
152. Implement LRU Cache.

19. Rolling state: the next state comes only from the previous one (7)

153. Climbing Stairs
154. Fibonacci Numbers
155. Maximize sum without choosing adjacent elements
156. Given n houses in a line with money in each, find the maximum amount a robber can steal, without stealing from two adjacent houses.
157. Paint House II
158. Write SQL code to publish the Fibonacci series.
159. Constrained Dice Roll Sequences

20. Two pointers: move i and j toward each other (or along two lists) (7)

160. Reverse a linked list.
161. Sort a binary array in linear time.
162. Is this a valid palindrome?
163. Reverse a Sentence
164. Find median of merged RLE arrays
165. Run-Length Vector Storage + Dot Product
166. Count Unique Triplets That Sum to Zero

21. Walk two strings at once with two indices (7)

167. Find the First Occurrence of a Substring
168. Determine if two words are sorted in lexicographic order.
169. Return all unique substrings of a given string.
170. Can board states be transformed?
171. Decide string transform with directional tokens and walls
172. Determine if two sentences are similar.
173. Find Dictionary Matches for a Jammed String

22. Simulate a machine: a class with state and one branch per kind of event (7)

174. Mario-Style Score Tracker (OOP)
175. Tic-Tac-Toe System with K Players and 3-in-a-Row Win
176. Design Tic-Tac-Toe With K Players
177. Infinite Deck Card Duel
178. Implement iterator merging lists with filtering
179. Design an elevator control system
180. Simulate In-Place Cellular State Updates

23. Top-K by frequency: Counter plus nsmallest (6)

181. Top-K Frequent in Large Logs
182. Rearrange Tasks With Cooldown
183. Implement frequency + distance top■K queries
184. Solve chat, grid paths, and car rentals
185. Top-K Chatters by Word Count
186. Find top/bottom-k words in list or stream

24. Digits and characters: peel one at a time and carry (6)

187. Caesar Cipher: Recover Key, Encrypt, and Decrypt
188. Add two big integers from digit lists
189. Solve several coding interview problems
190. Check Whether Digits Are Monotone Increasing
191. Check Digits and Combine Ranges
192. Permutation Sequence

25. Look at the input once and keep only a small summary (and count your calls) (6)

193. Two-Day Log Intersection under Memory Limit
194. Find shared objects across two log files
195. Streaming Points: Emit K Within Distance
196. Minimize calls to find all bad test pairs
197. Find All Bad Pairs Using runTest Oracle
198. Find maximum value in huge text file

26. Grid DP: each cell from its neighbours above and to the left (5)

199. Grid Path Count with Diagonal Moves and Checkpoints
200. Longest Same-Color Grid Sequence
201. Count Same-Color Squares in Matrix

202. Find the Longest Straight Run of One Color
203. Count same-color squares in a character grid

27. Nested structures: a stack of contexts, or recursion with a cache (5)

204. Decode String
205. Template Variable Expansion with Follow-Ups
206. Expand Recursive Key-Value Placeholders in a Template
207. Nested List Weight Sum (LC 339) with Follow-Ups
208. Nested Function Expression Evaluator

28. Frontier search (BFS or DFS), Tier 2: free steps, early exit, neighbours worked out on the fly (5)

209. Shortest Path Nodes and Escaping a Pursuer
210. Radio Broadcast Reachability
211. Minimum Direction Violations in a Directed Graph
212. Compute shortest paths with blocked nodes
213. Shortest Directed Cycle Through a Node

29. Topological sort (Kahn) and cycle detection (5)

214. Course Scheduling and Dependency Variants
215. Validate Race Results (Cycle Detection)
216. Solve tree, graph, sliding-window problems
217. Validate course catalog dependencies
218. Course Schedule

30. Counting pairs by looking up each partner in a Counter or a bucket (5)

219. Progressive Array Partitioning
220. K-diff Pairs in an Array
221. Count Axis-Aligned Squares Among Grid Points
222. Sum of Two Squares in Two Ways
223. Maximum-Area Rectangle from Planar Points

31. Tree DP: build a parent order once, then add up from the leaves (5)

224. Compute File System Total Size
225. Minimum-Cost Tree Disconnection
226. Given a large n-ary tree where the root node can pass information to one child per iteration, and in subsequent iterations, nodes can pass information to one child while the root continues with others, find the minimum number of iterations to reach all nodes.
227. Compute distance-sum from every tree node
228. Tree Distance Sum (Re-root DP)

32. Counting or reaching an amount: the knapsack loop (5)

229. Maximize Shelf Space (Unbounded Knapsack)
230. Coin Change
231. Partition an array into two sub-arrays with equal sum.
232. Explain how to find a target sum in an array.
233. Reverse Coin Change

33. Prefix sums, Tier 2: running total plus a dict of earlier totals (5)

234. Maximum-Sum Subarray With Equal Endpoints
235. Maximum Subarray Sum With a Minimum Length Constraint
236. Solve array-sum and city-community problems
237. Find the maximum subarray sum.
238. Given an integer array nums and an integer k, return true if nums has a subarray of at least two elements whose sum is a multiple of k.

34. Group by a canonical key (5)

239. Given an array of strings, return the words that can be typed using letters of the alphabet on only one row of American keyboard.
240. Check if all substrings are anagrams of words
241. Group anagrams
242. Given an array of strings, return groups of "buddy strings," where the strings have the same length and the distance between corresponding characters is the same.
243. Given two strings and two allowed operations, swapping positions of letters and converting all occurrences of one character to another, can the first string be transformed into the second?

35. K-way merge: a heap of the next item from each source (5)

244. Find k pairs with smallest sums
245. K Largest Elements Across Sorted Lists
246. Sort a nearly sorted (or K sorted) array.

247. Merge k sorted linked lists.
248. Return top-k subset sums in descending order

36. Tree recursion, Tier 2: pass a rule down, return a node or a yes/no up (5)

249. Next Birthday (BST In-Order Successor with Wraparound)
250. Lowest Common Ancestor
251. Determine if a given binary tree is a binary search tree (BST).
252. Find mode in a trinary search tree
253. Next Upcoming Birthday in a Family Tree

37. Random pick without repeats: swap the chosen item with the last one and pop (5)

254. Random Generator Without Repeats
255. Random Song Shuffler With a No-Repeat Window
256. Design set with O(1) random access
257. Unique Random Number Generator
258. Shuffle array using random integer API

38. Build and serialize: write a structure out, read it back with the same walk (5)

259. Construct a binary tree given its preorder and inorder traversals.
260. Write functions to serialize and deserialize a list of strings.
261. Print a Directory Tree
262. Implement a decode and encode function of a general N-ary tree.
263. Serialize and Deserialize a Graph

39. Games: win or lose with a cache, and guessing with feedback (5)

264. Choose optimal guesses for green-only Wordle
265. Wordle-Style 5-Letter Minimum-Guess Strategy
266. Determine winner in stack-merging game
267. Find secret word with match-count feedback
268. Babylon Tile-Stack Merge Game

40. Sliding window: grow on the right, shrink on the left while it is too bad (5)

269. Temperature Monitoring Data Structure
270. Shortest Subarray with K Distinct Integers
271. Find the minimum window substring.
272. Footpath Beauty After Repainting
273. Necklace Cut into Two Halves with Equal D/R Counts

41. Parent pointers: store who is above each node and climb (5)

274. Tree Node.
275. Employee Hierarchy Add / Delete
276. Validate parent array forms a tree
277. Lowest Common Ancestor of a Binary Tree III
278. Root a Tree So It Becomes a Binary Tree

42. Build or transform a grid with index arithmetic (5)

279. Filled ASCII Diamond
280. Connected Crop Arrangement on a Grid
281. Rotate Any m×n 2D Matrix
282. Solve matrix groups and recipe inventory
283. Range Sum Query 2D (Immutable Matrix)

43. Split into pieces, fix each piece, join (and pack words into lines) (5)

284. Implement string formatting and request tracking tasks
285. Snake Case to Camel Case with Repository-Wide Refactor
286. Google Doc Line Wrap (Word Wrap with Whitespace and Newlines)
287. Implement a function to justify a list of words within a given width.
288. Convert Valid Snake Case to Lower Camel Case

GROUP 1 OF 43 | 9 QUESTIONS, ONE CODE TEMPLATE
Frontier search (BFS or DFS), Tier 1: spread, count and measure distance

1. Flood Fill [E]

```
# BASE QUESTION (optimal): Iterative DFS with a guard
# Idea: Spread over same-coloured neighbours and repaint;
# skip everything when the colour does not change.
def flood_fill(image, sr, sc, new_color):
    old = image[sr][sc]
    if old == new_color:
        return image
    stack = [(sr, sc)]
    image[sr][sc] = new_color
    while stack:
        r, c = stack.pop()
        for nr, nc in (
            (r + 1, c),
            (r - 1, c),
            (r, c + 1),
            (r, c - 1),
        ):
            if (
                0 <= nr < len(image)
                and 0 <= nc < len(image[0])
                and image[nr][nc] == old
            ):
                image[nr][nc] = new_color
                stack.append((nr, nc))
    return image
```

2. Number of Islands [M]

```
# HELPERS for the base question
DIRS = ((1, 0), (-1, 0), (0, 1), (0, -1))


def neighbors(r, c, rows, cols):
    # the cells beside (r, c) that are inside the grid
    for dr, dc in DIRS:
        nr, nc = r + dr, c + dc
        if 0 <= nr < rows and 0 <= nc < cols:
            yield nr, nc


def flood(grid, start, seen):
    # visit all land joined to start; return the count
    rows, cols = len(grid), len(grid[0])
    seen.add(start)
    stack, size = [start], 0
    # a stack instead of recursion: no recursion limit
    while stack:
        r, c = stack.pop()
        size += 1
        for nb in neighbors(r, c, rows, cols):
            if grid[nb[0]][nb[1]] == 1 and nb not in seen:
                seen.add(nb)
                stack.append(nb)
    return size


def island_sizes(grid):
    if not grid:
        return []
    seen, sizes = set(), []
    for r in range(len(grid)):
        for c in range(len(grid[0])):
            # an island starts at land not visited yet
            if grid[r][c] == 1 and (r, c) not in seen:
                sizes.append(flood(grid, (r, c), seen))
    return sizes
```

```
# BASE QUESTION (optimal): Iterative DFS flood fill
# Idea: Scan every cell; each unvisited land cell starts a
# new island, and an explicit stack marks all connected
# land as visited. Iterative DFS avoids Python's recursion
# limit on large grids.
def num_islands(grid):
    # one entry per island
    return len(island_sizes(grid))
```

3. Rotten Oranges / Multi-Source BFS (taxis) [M]

```
from collections import deque


# HELPERS for the base question
def multi_source_bfs(rows, cols, sources, open_cell):
    # distance from the nearest source to every cell
    dist = {s: 0 for s in sources}
    queue = deque(sources)
    while queue:
        r, c = queue.popleft()
        for nr, nc in ((r + 1, c), (r - 1, c),
                       (r, c + 1), (r, c - 1)):
            inside = 0 <= nr < rows and 0 <= nc < cols
            if inside and (nr, nc) not in dist:
                if open_cell(nr, nc):
                    dist[(nr, nc)] = dist[(r, c)] + 1
                    queue.append((nr, nc))
    return dist


# BASE QUESTION (optimal): Multi-source BFS by minute
# Idea: Push every rotten orange into the queue at minute
# 0 and expand level by level; each level is one minute.
# Count the fresh oranges and decrement as they rot, so
# the answer is -1 if any remain at the end.
def rotting_minutes(grid):
    rows, cols = len(grid), len(grid[0])
    cells = [(r, c) for r in range(rows)
             for c in range(cols)]
    rotten = [x for x in cells if grid[x[0]][x[1]] == 2]
    fresh = [x for x in cells if grid[x[0]][x[1]] == 1]
    # rot spreads through any non-empty cell
    dist = multi_source_bfs(
        rows, cols, rotten, lambda r, c: grid[r][c] != 0)
    # a fresh orange that is never reached means -1
    if any(x not in dist for x in fresh):
        return -1
    return max((dist[x] for x in fresh), default=0)
```

4. Solve Grid and Counting Problems [M]

```
from collections import deque


# HELPERS for the base question
STEPS = ((1, 0), (-1, 0), (0, 1), (0, -1))


# BASE QUESTION (optimal): Islands: iterative BFS
# Idea: Same scan, but spread with a queue instead of
# recursion, so the depth of an island cannot overflow the
# stack. Marking visited cells in place (or with a set)
# keeps each cell processed once.
def num_islands(grid):
    # '1' is land: a BFS marks each island once
    if not grid:
        return 0
    m, n = len(grid), len(grid[0])
    seen, count = set(), 0
    for r in range(m):
        for c in range(n):
            if grid[r][c] != "1" or (r, c) in seen:
                continue
```

```
            count += 1
            seen.add((r, c))
            queue = deque([(r, c)])
            while queue:
                x, y = queue.popleft()
                for dx, dy in STEPS:
                    a, b = x + dx, y + dy
                    if not (0 <= a < m and 0 <= b < n):
                        continue
                    land = grid[a][b] == "1"
                    if land and (a, b) not in seen:
                        seen.add((a, b))
                        queue.append((a, b))
    return count
```

5. Find shortest relationship path using BFS [M]

```
from collections import deque


# BASE QUESTION (optimal): Breadth-first search with a
# parent map
# Idea: Queue the source and mark it visited. Pop a
# person, queue each unvisited neighbour with parent =
# this person, and stop when the target is queued. Follow
# parents back to the source and reverse to get the path.
def shortest_path(adj, s, t):
    # BFS: the first time we reach a node is by a shortest
    # path
    if s == t:
        return [s]
    parent, queue = {s: None}, deque([s])
    while queue:
        u = queue.popleft()
        for v in adj.get(u, ()):
            if v not in parent:
                parent[v] = u
                if v == t:
                    # walk the parent links back to s
                    path = [v]
                    while parent[path[-1]] is not None:
                        path.append(parent[path[-1]])
                    return path[::-1]
                queue.append(v)
    return None
```

6. Farthest City for a New Data Center [M]

```
from collections import deque


# BASE QUESTION (optimal): One multi-source BFS
# Idea: Seed the queue with every data center at distance
# 0; the first visit is the nearest one.
def farthest_city(num_cities, edges, dcs):
    adj = [[] for _ in range(num_cities)]
    for u, v in edges:
        adj[u].append(v)
        adj[v].append(u)
    dist = [-1] * num_cities
    queue = deque(dcs)
    for d in dcs:
        dist[d] = 0
    while queue:
        u = queue.popleft()
        for v in adj[u]:
            if dist[v] < 0:
                dist[v] = dist[u] + 1
                queue.append(v)
    return dist.index(max(dist))
```

7. Compute distance to nearest taxi in grid [M]

```
from collections import deque


# HELPERS for the base question
STEPS = ((1, 0), (-1, 0), (0, 1), (0, -1))


def spread(grid, dist, queue, better=False):
    # grow outward; with better=True a cell may be updated
    # when a
    # new, closer source reaches it (distances only go
    # down)
    R, C = len(grid), len(grid[0])
    while queue:
        r, c = queue.popleft()
        for dr, dc in STEPS:
            a, b = r + dr, c + dc
            if not (0 <= a < R and 0 <= b < C):
                continue
            if grid[a][b] == "#":
                continue
            new = dist[r][c] + 1
            unseen = dist[a][b] == -1
            if unseen or (better and new < dist[a][b]):
                dist[a][b] = new
                queue.append((a, b))


# BASE QUESTION (optimal): One multi-source BFS
# Idea: Initialise the queue with every taxi at distance 0
# and expand in rounds. A cell receives its distance the
# first time it is reached; blocked cells are never
# entered; cells that no taxi can reach stay at -1.
def taxi_distances(grid):
    # 'T' taxi, '.' road, '#' wall. One BFS that starts
    # from
    # every taxi at once gives each cell its nearest taxi.
    R, C = len(grid), len(grid[0])
    dist = [[-1] * C for _ in range(R)]
    queue = deque()
    for r in range(R):
        for c in range(C):
            if grid[r][c] == "T":
                dist[r][c] = 0
                queue.append((r, c))
    spread(grid, dist, queue)
    return dist
```

8. Infection Spread on a Grid (Cellular Automaton) [H]

```
# HELPERS for the base question
STEPS = ((1, 0), (-1, 0), (0, 1), (0, -1))


# TIME each cell gets infected
def infection_times(grid):
    # step at which each cell gets infected (a BFS level);
    # 'I' = infected at time 0, '.' = healthy, '#' =
    # blocked
    rows, cols = len(grid), len(grid[0])
    when = {}
    frontier = [(r, c) for r in range(rows)
                for c in range(cols) if grid[r][c] == "I"]
    for cell in frontier:
        when[cell] = 0
    step = 0
    while frontier:
        step += 1
        nxt = []
        for r, c in frontier:
            for dr, dc in STEPS:
                a, b = r + dr, c + dc
```

```
                inside = 0 <= a < rows and 0 <= b < cols
                # every healthy neighbour is infected next
                # step
                if inside and grid[a][b] == "." \
                        and (a, b) not in when:
                    when[(a, b)] = step
                    nxt.append((a, b))
        frontier = nxt
    return when


# BASE QUESTION (optimal): Frontier BFS (only recently
# infected cells spread)
# Idea: Put every initially infected cell in the frontier.
# Each step, look only at the neighbors of frontier cells,
# infect the healthy ones, and make them the next
# frontier.
def spread_steps(grid):
    # steps until nothing changes any more
    when = infection_times(grid)
    return max(when.values(), default=0)
```

9. Shortest Distance From All Buildings [H]

```
from collections import deque


from collections import deque


# BASE QUESTION (optimal): One BFS per house
# Idea: Every house adds its distance to the empty cells
# it reaches; valid cells were reached by all houses.
def school_totals(grid):
    rows, cols = len(grid), len(grid[0])
    total = [[0] * cols for _ in range(rows)]
    reached = [[0] * cols for _ in range(rows)]
    houses = 0
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] != 1:
                continue
            houses += 1
            seen, queue = {(r, c)}, deque([(r, c, 0)])
            while queue:
                x, y, d = queue.popleft()
                for nx, ny in (
                    (x + 1, y),
                    (x - 1, y),
                    (x, y + 1),
                    (x, y - 1),
                ):
                    if (
                        0 <= nx < rows
                        and 0 <= ny < cols
                        and (nx, ny) not in seen
                        and grid[nx][ny] == 0
                    ):
                        seen.add((nx, ny))
                        total[nx][ny] += d + 1
                        reached[nx][ny] += 1
                        queue.append((nx, ny, d + 1))
    return total, reached, houses


# BASE QUESTION (optimal): One BFS per house
# Idea: Every house adds its distance to the empty cells
# it reaches; valid cells were reached by all houses.
def shortest_distance_from_all_buildings(grid):
    total, reached, houses = school_totals(grid)
    best, spot = float("inf"), (-1, -1)
    for r in range(len(grid)):
        for c in range(len(grid[0])):
```

```
        if (
            grid[r][c] == 0
            and reached[r][c] == houses
            and total[r][c] < best
        ):
            best, spot = total[r][c], (r, c)
    return spot
```

GROUP 2 OF 43 | 9 QUESTIONS, ONE CODE TEMPLATE
Sweep over events in time order

10. Meeting Rooms / Interval Scheduling Variants [M]

```
# BASE QUESTION (optimal): Sweep line over start and end
# events
# Idea: Create (time, +1) for each start and (time, -1)
# for each end, sort so that an end at time t comes before
# a start at t (rooms free up the moment a meeting ends)
# and track the running maximum.
def min_rooms(intervals):
    # +1 at every start, -1 at every end
    events = []
    for s, e in intervals:
        events.append((s, 1))
        events.append((e, -1))
    # at equal times ends (-1) sort before starts (+1)
    events.sort()
    cur = best = 0
    for _, delta in events:
        cur += delta
        best = max(best, cur)
    return best
```

11. Train / Flight Schedule Reachability [M]

```
# HELPERS used by the code below
INF = float("inf")


# BASE QUESTION (optimal): Sort by departure and keep
# earliest arrivals
# Idea: Sort the legs by departure time. Keep
# earliest[station] = earliest known arrival, with the
# source at minus infinity.
def earliest_arrivals(legs, source, strict=True):
    """Time is part of the state: keep the earliest time
    we can be at each station. Legs go in departure
    order, so every leg that could feed this one is
    already done."""
    earliest = {source: -INF}
    by_departure = sorted(legs, key=lambda x: x[2])
    for frm, to, dep, arr in by_departure:
        have = earliest.get(frm, INF)
        # we must already be there before it leaves
        if have < dep or (not strict and have == dep):
            if arr < earliest.get(to, INF):
                earliest[to] = arr
    return earliest


# BASE QUESTION (optimal): Sort by departure and keep
# earliest arrivals
# Idea: Sort the legs by departure time. Keep
# earliest[station] = earliest known arrival, with the
# source at minus infinity.
def reachable(legs, source, dest, strict=True):
    return dest in earliest_arrivals(legs, source, strict)
```

12. Solve Rooms and Top-K Streams [M]

```
# BASE QUESTION (optimal): Rooms: sorted starts and ends
# with two pointers
# Idea: Sort starts and ends separately. For each start:
# if it is at or after the earliest unfinished end, that
# room is reused (advance the end pointer); otherwise a
# new room is needed.
def min_rooms(intervals):
    # two sorted lists: every start with no room free
    # needs
    # a new room, otherwise it reuses one that just ended
```

```
    starts = sorted(s for s, _ in intervals)
    ends = sorted(e for _, e in intervals)
    rooms = j = 0
    for s in starts:
        if s >= ends[j]:
            # the earliest-ending meeting is over: reuse
            # it
            j += 1
        else:
            rooms += 1
    return rooms
```

13. Compute servers needed for daily recurring jobs [M]

```
from collections import defaultdict


# HELPERS for the base question
def merge_by_job(intervals):
    # intervals of the SAME job may overlap on one server,
    # so
    # merge them first: (job, start, end) -> disjoint
    # blocks
    by_job = defaultdict(list)
    for job, start, end in intervals:
        by_job[job].append((start, end))
    blocks = []
    for job, spans in by_job.items():
        cur = None
        for s, e in sorted(spans):
            if cur and s <= cur[1]:
                cur[1] = max(cur[1], e)
            else:
                if cur:
                    blocks.append((cur[0], cur[1], job))
                cur = [s, e]
        blocks.append((cur[0], cur[1], job))
    return blocks


# BASE QUESTION (optimal): Merge each job's intervals,
# then sweep start and end events
# Idea: Group the intervals by job_id, sort and merge each
# group's overlapping or touching intervals, then collect
# all merged intervals.
# Answer: Remove those jobs from the daily intervals and
# add one server per such job: servers = (number of
# dedicated jobs) + (maximum overlap of the remaining
# jobs).
def servers_needed(intervals, dedicated=()):
    # FOLLOW-UP 1: rare multi-day jobs get their own
    # server
    # dedicated jobs (they span several days) get their
    # own
    # server; the rest share servers: the answer is the
    # largest number of blocks running at the same moment
    dedicated = set(dedicated)
    shared = [i for i in intervals
              if i[0] not in dedicated]
    events = []
    for start, end, _ in merge_by_job(shared):
        events.append((start, 1))
        # [start, end): at equal times an end comes first
        events.append((end, -1))
    run = best = 0
    for _, delta in sorted(events):
        run += delta
        best = max(best, run)
    used = {i[0] for i in intervals if i[0] in dedicated}
    return best + len(used)
```

14. Gate Entry / Exit Time Resolution with Tie-Break Rules [M]

```
from collections import deque


# BASE QUESTION (optimal): Event-driven simulation with
# two queues
# Idea: Sort people by (arrival time, id) and keep one
# FIFO queue for waiting enters and one for exits. At each
# second pick from the queues using the previous-second
# state.
# Answer: Keep the simulation and make the idle winner a
# parameter: only the line that picks the queue changes.
# With 'exit first' it equals the standard rule; with
# 'enter first' an enter and an exit arriving together
# swap places.
def gate_pass_times(events, idle_first="exit"):
    # events[i] = (arrival time, "enter" or "exit")
    n = len(events)
    # people sorted by arrival, then by id
    order = sorted(range(n),
                   key=lambda i: (events[i][0], i))
    queues = {"enter": deque(), "exit": deque()}
    result = [0] * n
    t, i, prev, last_used = 0, 0, None, -10
    while i < n or queues["enter"] or queues["exit"]:
        if not queues["enter"] and not queues["exit"]:
            # nobody waits: jump the clock to the next
            # arrival
            t = max(t, events[order[i]][0])
            # a gap of idle time forgets the previous
            # action
            if t > last_used + 1:
                prev = None
        # everyone who arrived by now joins their line
        while i < n and events[order[i]][0] <= t:
            person = order[i]
            queues[events[person][1]].append(person)
            i += 1
        both = queues["enter"] and queues["exit"]
        if both:
            # FOLLOW-UP 4: idle tie-break set by
            # idle_first
            # tie: continue the direction used last second
            # (when idle, use idle_first)
            want = prev or idle_first
        else:
            want = "enter" if queues["enter"] else "exit"
        person = queues[want].popleft()
        result[person] = t
        prev, last_used = want, t
        t += 1
    return result
```

15. Meeting Rooms III Variant [M]

```
# HELPERS used by the code below
DAY = 24


# HELPERS for the base question
def pieces(meetings, day=DAY):
    # a meeting whose end is before its start wraps past
    # midnight: split it into the part before midnight and
    # the part after it
    for s, e in meetings:
        if e < s:
            yield s, day
            yield 0, e
        else:
            yield s, e


# BASE QUESTION (optimal): Split wrapped meetings and
```

```
# sweep (maximum overlap)
# Idea: Turn every meeting into one or two ordinary
# intervals, create +1 events at starts and -1 events at
# ends, sort them (ends before starts at the same minute,
# because intervals are half-open), and take the maximum
# running count.
def min_rooms_sweep(meetings, day=DAY):
    # sweep the time line: +1 at a start, -1 at an end. At
    # equal times an end comes first (a meeting ending at
    # 5
    # and one starting at 5 can share a room).
    events = []
    for s, e in pieces(meetings, day):
        events.append((s, 1))
        events.append((e, -1))
    events.sort()
    cur = best = 0
    for _, step in events:
        cur += step
        best = max(best, cur)
    return best
```

16. Count Overlapping Rectangle Updates on a Grid [H]

```
# BASE QUESTION (optimal): 2D difference array and prefix
# sums
# Idea: For each rectangle add +1 at (r1, c1), -1 at (r1,
# c2 + 1), -1 at (r2 + 1, c1) and +1 at (r2 + 1, c2 + 1)
# in an (n + 1) x (n + 1) array.
def range_add_counts(n, rects):
    # rects are [r1, c1, r2, c2], inclusive. Mark only the
    # 4
    # corners of each rectangle in a difference array ...
    d = [[0] * (n + 1) for _ in range(n + 1)]
    for r1, c1, r2, c2 in rects:
        d[r1][c1] += 1
        d[r1][c2 + 1] -= 1
        d[r2 + 1][c1] -= 1
        d[r2 + 1][c2 + 1] += 1
    # ... then two prefix sums spread them over the cells
    for r in range(n + 1):
        for c in range(1, n + 1):
            d[r][c] += d[r][c - 1]
    for c in range(n + 1):
        for r in range(1, n + 1):
            d[r][c] += d[r - 1][c]
    return [row[:n] for row in d[:n]]
```

17. Compute city skyline outline [H]

```
import heapq


# BASE QUESTION (optimal): Sweep line with a max-heap and
# lazy deletion
# Idea: Make events (x, -height) for left edges and (x, 0,
# right) for right edges, sorted so that at the same x
# taller starts come first and ends come after starts.
# Keep a heap of (-height, right edge).
def skyline(buildings):
    # buildings are [left, right, height]
    # start events carry -height so taller starts sort
    # first;
    # end events carry 0
    events = sorted([(l, -h, r) for l, r, h in buildings]
                    + [(b[1], 0, 0) for b in buildings])
    out = []
    # the active buildings: (-height, right edge), ground
    # first
    heap = [(0, float("inf"))]
    for x, neg_h, right in events:
        # buildings that ended at or before x are gone
        while heap[0][1] <= x:
            heapq.heappop(heap)
```

```
        if neg_h:
            heapq.heappush(heap, (neg_h, right))
        top = -heap[0][0]
        # a key point only where the height changes
        if not out or out[-1][1] != top:
            out.append([x, top])
    return out
```

18. Compute Turnstile Crossing Times [H]

```
from collections import deque


# BASE QUESTION (optimal): Event-driven simulation with
# two queues
# Idea: Sort arrivals once. When both queues are empty,
# jump straight to the next arrival time; if the jump
# leaves at least one idle second, reset the previous
# state to 'none'.
# Answer: Keep the same loop and make the idle-state
# winner a parameter. Only the line that picks the queue
# changes, which is why the state machine should be
# written first.
def turnstile_times(events, idle_first="exit"):
    # events[i] = (arrival time, "enter" or "exit");
    # the result is the second each person goes through
    n = len(events)
    order = sorted(range(n),
                   key=lambda i: (events[i][0], i))
    lines = {"enter": deque(), "exit": deque()}
    result = [0] * n
    t, i, prev, last_used = 0, 0, None, -10
    while i < n or lines["enter"] or lines["exit"]:
        if not lines["enter"] and not lines["exit"]:
            # nobody is waiting: jump to the next arrival
            t = max(t, events[order[i]][0])
            # an idle second forgets the last direction
            if t > last_used + 1:
                prev = None
        # everyone who has arrived by now joins a line
        while i < n and events[order[i]][0] <= t:
            person = order[i]
            lines[events[person][1]].append(person)
            i += 1
        if lines["enter"] and lines["exit"]:
            # FOLLOW-UP 1: idle tie-break set by
            # idle_first
            # both lines: keep the direction used last
            # second,
            # and use idle_first after an idle gap
            want = prev or idle_first
        else:
            want = "enter" if lines["enter"] else "exit"
        person = lines[want].popleft()
        result[person] = t
        prev, last_used = want, t
        t += 1
    return result
```

GROUP 3 OF 43 | 9 QUESTIONS, ONE CODE TEMPLATE
Tree recursion, Tier 1: one function that visits a node, asks its children, and reports upward

19. Given two trees, determine if the pre-order traversals of two trees are same or not. [E]

```
from itertools import zip_longest


# HELPERS for the base question
MISSING = object()


def preorder(root):
    # root, then left, then right; iterative with a stack
    stack = [root] if root else []
    while stack:
        node = stack.pop()
        yield node.val
        # right goes first so the left child comes out
        # first
        if node.right:
            stack.append(node.right)
        if node.left:
            stack.append(node.left)


# BASE QUESTION (optimal): Lazy pre-order generators
# compared in lockstep
# Idea: Generate each pre-order lazily with an explicit
# stack (push right before left), and compare the two
# streams element by element with zip_longest, so the
# first mismatch stops the work and no list is built.
def same_preorder(a, b):
    # lazily compare the two sequences value by value; a
    # shorter sequence is padded with MISSING and so
    # differs
    pairs = zip_longest(preorder(a), preorder(b),
                        fillvalue=MISSING)
    return all(x == y for x, y in pairs)
```

20. Calculate the height of a binary tree. [E]

```
from collections import deque


# BASE QUESTION (optimal): Level-order traversal counting
# levels
# Idea: Process the tree one level at a time with a queue
# and count the levels. No recursion, so depth cannot
# overflow the stack; memory is the widest level.
def height(root):
    # height in NODES: level order, count the levels. An
    # empty tree is 0 and a single node is 1. (In edges,
    # subtract one: empty is -1, single node is 0.)
    levels, queue = 0, deque([root] if root else [])
    while queue:
        levels += 1
        for _ in range(len(queue)):
            node = queue.popleft()
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)
    return levels
```

21. Find right-side view of binary tree [E]

```
# BASE QUESTION (optimal): DFS visiting right before left,
# first node at each depth
# Idea: Visit the right child first. The first node you
# meet at a depth that is not yet recorded is the
# rightmost one at that depth. Only O(height) memory is
# needed.
def right_side_view(root):
    # DFS, right child first: the first node seen at each
    # depth is the rightmost one
    out = []

    def dfs(node, depth):
        if node is None:
            return
        if depth == len(out):
            out.append(node.val)
        dfs(node.right, depth + 1)
        dfs(node.left, depth + 1)

    dfs(root, 0)
    return out
```

22. Expression Tree Single-Leaf Mutation [M]

```
# HELPERS for the base question
def combine(op, a, b=None):
    # value of one gate from its input values
    if op == "AND":
        return a and b
    if op == "OR":
        return a or b
    if op == "XOR":
        return a != b
    # NOT has one input
    return not a


def right_value(value, node):
    # NOT has no right child
    return value[id(node.right)] if node.right else None


def build_cache(root):
    # one traversal caches parents, values and leaves
    parent, value, leaves = {id(root): None}, {}, []
    order, stack = [], [root]
    while stack:
        node = stack.pop()
        order.append(node)
        for child in (node.left, node.right):
            if child is not None:
                parent[id(child)] = node
                stack.append(child)
    # reversed order visits children before parents
    for node in reversed(order):
        if node.op is None:
            value[id(node)] = node.val
            leaves.append(node)
        else:
            a = value[id(node.left)]
            b = right_value(value, node)
            value[id(node)] = combine(node.op, a, b)
    return parent, value, leaves


# BASE QUESTION (optimal): Cache values and parents,
# recompute only the path with early stop
# Idea: One traversal stores every node's value and its
# parent.
def flip_answers(root):
    parent, value, leaves = build_cache(root)
    answers = []
```

```
    for leaf in leaves:
        undo, node = [], leaf
        new = not value[id(leaf)]
        # walk up only while a value really changes
        while node is not None and value[id(node)] != new:
            undo.append((node, value[id(node)]))
            value[id(node)] = new
            node = parent[id(node)]
            if node is None:
                break
            a = value[id(node.left)]
            b = right_value(value, node)
            new = combine(node.op, a, b)
        answers.append(value[id(root)])
        # put every changed value back for the next leaf
        for node, old in reversed(undo):
            value[id(node)] = old
    return leaves, answers
```

23. Longest Consecutive Path in Binary Tree [M]

```
# BASE QUESTION (optimal): Top-down DFS carrying the
# current run
# Idea: Pass the length of the run ending at the parent.
# If the child is exactly parent + 1 the run grows,
# otherwise it restarts at 1. Track the global best.
# Length is counted in nodes; state that choice out loud.
def longest_consecutive(root, want_path=False):
    # parent -> child path whose values go up by exactly 1
    best, best_path = 0, []
    stack = [(root, None, [])] if root else []
    while stack:
        node, parent, run = stack.pop()
        # extend the run or start a new one at this node
        if parent is not None and node.val == parent + 1:
            run = run + [node.val]
        else:
            run = [node.val]
        if len(run) > best:
            best, best_path = len(run), run
        for child in (node.left, node.right):
            if child is not None:
                stack.append((child, node.val, run))
    # FOLLOW-UP 3: want_path=True returns the path itself
    return best_path if want_path else best
```

24. Given the root of a binary tree, return the length of the diameter of the tree. [M]

```
# BASE QUESTION (optimal): One post-order pass returning
# heights and updating the best
# Idea: depth(node) returns the height of the node's
# subtree and, on the way, updates best = max(best, left +
# right). Every node is visited once, and the answer is
# best when the traversal ends (edges, not nodes).
def diameter_nodes(root):
    # longest path between any two nodes, counted in
    # NODES.
    # Iterative post-order: height of every node, then the
    # best path through each node = left + right + 1.
    if root is None:
        return 0
    height, best, stack = {None: 0}, 0, [(root, False)]
    while stack:
        node, done = stack.pop()
        if not done:
            stack.append((node, True))
            for child in (node.left, node.right):
                if child:
                    stack.append((child, False))
            continue
        lh, rh = height[node.left], height[node.right]
        height[node] = 1 + max(lh, rh)
        best = max(best, lh + rh + 1)
    return best
```

25. Implement zigzag level-order traversal [M]

```
from collections import deque


# BASE QUESTION (optimal): BFS filling each level from the
# left or the right
# Idea: Keep a left-to-right flag. For each level build a
# deque; append on the right when going left to right and
# on the left when going right to left. Flip the flag
# after each level.
def zigzag_level_order(root):
    # BFS by levels; flip the direction on every level
    out, ltr = [], True
    queue = deque([root] if root else [])
    while queue:
        level = deque()
        for _ in range(len(queue)):
            node = queue.popleft()
            if ltr:
                level.append(node.val)
            else:
                level.appendleft(node.val)
            for child in (node.left, node.right):
                if child:
                    queue.append(child)
        out.append(list(level))
        ltr = not ltr
    return out
```

26. Repeatedly Delete Leaves Matching a Target Value From a Binary Tree [H]

```
# BASE QUESTION (optimal): One post-order pass
# Idea: Process both children first, then look at the
# node: if it is now a leaf and its value is the target,
# return None to its parent.
def remove_leaves(root, target):
    # recursive post-order: children first, then the node
    deleted = 0

    def dfs(node):
        nonlocal deleted
        if node is None:
            return None
        node.left = dfs(node.left)
        node.right = dfs(node.right)
        # a leaf with the target value disappears
        if not node.left and not node.right \
                and node.val == target:
            deleted += 1
            return None
        return node

    return dfs(root), deleted
```

27. Delete an Entire Binary Tree [H]

```
# BASE QUESTION (optimal): Recursive post-order
# Idea: Free the left subtree, the right subtree and then
# the node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right


# BASE QUESTION (optimal): Recursive post-order
# Idea: Free the left subtree, the right subtree and then
# the node.
def delete_tree(root):
    freed = []

    def free(node):
```

```
        if node is None:
            return
        free(node.left)
        free(node.right)
        freed.append(node.val)
        node.left = node.right = None

    free(root)
    return freed
```

GROUP 4 OF 43 | 9 QUESTIONS, ONE CODE TEMPLATE
A heap of what is available: pop the best, put back the changed one

28. Streaming Median (LC 295) with Follow-ups [M]

```
import heapq


# BASE QUESTION (optimal): Two heaps
# Idea: Lower half in a max-heap, upper half in a min-
# heap, sizes differing by at most one and every lower
# value <= every upper value. The median is the larger
# heap's root, or the average of both roots.
class MedianFinder:
    def __init__(self):
        # low: max-heap (stored negated); high: min-heap
        self.low, self.high = [], []

    def add(self, x):
        # push through low so the halves stay ordered
        heapq.heappush(self.low, -x)
        top = -heapq.heappop(self.low)
        heapq.heappush(self.high, top)
        # keep low the same size as high, or one bigger
        if len(self.high) > len(self.low):
            top = heapq.heappop(self.high)
            heapq.heappush(self.low, -top)

    def median(self):
        if len(self.low) > len(self.high):
            return float(-self.low[0])
        return (-self.low[0] + self.high[0]) / 2
```

29. Compute minimum number of rooms needed [M]

```
import heapq


# BASE QUESTION (optimal): Sort by start with a min-heap
# of end times
# Idea: Process meetings by start time. If the earliest-
# ending room has finished by this meeting's start, reuse
# it (pop and push the new end); otherwise open a new room
# (push).
def min_meeting_rooms(intervals):
    # half-open [start, end): a meeting ending at t does
    # not
    # clash with one starting at t. A heap holds the end
    # times
    # of the rooms in use.
    ends = []
    meetings = sorted(m for m in intervals if m[0] < m[1])
    for start, end in meetings:
        if ends and ends[0] <= start:
            # the earliest room is free: reuse it
            heapq.heapreplace(ends, end)
        else:
            heapq.heappush(ends, end)
    return len(ends)
```

30. Implement a Reusable Locker Allocation System [M]

```
import heapq


# BASE QUESTION (optimal): Min-heap of released numbers
# plus a next-unused counter
# Idea: store: if the customer already has a locker return
# -1; take the smallest released number from the heap if
# any, else the next never-used number if it is at most n,
# else -1.
class Lockers:
```

```
    def __init__(self, n):
        self.n = n
        # the next never-used locker number
        self.next_new = 1
        # released lockers, smallest on top
        self.free = []
        # customer -> locker number
        self.of = {}
        # locker of the latest successful store
        self.last_store = -1

    def store(self, customer):
        if customer in self.of:
            return -1
        if self.free:
            # a released locker is always smaller than any
            # new one
            locker = heapq.heappop(self.free)
        elif self.next_new <= self.n:
            locker = self.next_new
            self.next_new += 1
        else:
            return -1
        self.of[customer] = locker
        self.last_store = locker
        return locker

    def pickup(self, customer):
        locker = self.of.pop(customer, None)
        if locker is None:
            return -1
        # it becomes available again at once
        heapq.heappush(self.free, locker)
        return locker

    def last(self):
        # failed operations never change this
        return self.last_store
```

31. Simulate room assignments for scheduled meetings [M]

```
import heapq


# BASE QUESTION (optimal): Two heaps: free rooms and busy
# rooms
# Idea: Process meetings in start order. Move rooms that
# finished by the meeting's start into the free heap
# (lowest index first). If a room is free, use it.
# Answer: Add each meeting's duration (its original
# length, not the delay) to the room's tally; the choice
# of room is unchanged.
def most_booked_room(n, meetings, by="count"):
    # a meeting takes the lowest-numbered free room; if
    # none is
    # free it waits for the earliest room and keeps its
    # length
    free = list(range(n))
    heapq.heapify(free)
    # busy rooms: (time it becomes free, room)
    busy = []
    used = [0] * n
    for start, end in sorted(meetings):
        # rooms that finished by now are free again
        while busy and busy[0][0] <= start:
            _, room = heapq.heappop(busy)
            heapq.heappush(free, room)
        if free:
            room = heapq.heappop(free)
            heapq.heappush(busy, (end, room))
        else:
            # delayed: starts when the earliest room frees
            # up
```

```
            finish, room = heapq.heappop(busy)
            delayed_end = finish + (end - start)
            heapq.heappush(busy, (delayed_end, room))
        # FOLLOW-UP 3: by="time" ranks by total busy time
        # count meetings, or total length (not the delay)
        used[room] += 1 if by == "count" else end - start
    # ties go to the lowest room number
    return used.index(max(used))
```

32. Trip Expense Settlement [M]

```
import heapq


# HELPERS used by the code below
def balances(records):
    # net balance per person: what they paid minus what
    # they
    # owe. Positive: the group owes them. All sum to zero.
    net = {}
    for rec in records:
        payer = rec["payer"]
        net[payer] = net.get(payer, 0) + rec["amount"]
        for person, share in shares_of(rec).items():
            net[person] = net.get(person, 0) - share
    return {p: v for p, v in net.items() if v}


# BASE QUESTION (optimal): Heaps: match the largest debtor
# with the largest creditor
# Idea: Push creditors (positive net) in a max-heap and
# debtors (negative net) in another. Repeatedly pay
# min(debt, credit) from the biggest debtor to the biggest
# creditor, and push back whoever still has a remainder.
def settle_greedy(records):
    # the biggest debtor pays the biggest creditor, as
    # much
    # as one side can: at most n - 1 payments
    net = balances(records)
    owed = [(-v, p) for p, v in net.items() if v > 0]
    debt = [(v, p) for p, v in net.items() if v < 0]
    heapq.heapify(owed)
    heapq.heapify(debt)
    out = []
    while owed and debt:
        o_amt, creditor = heapq.heappop(owed)
        d_amt, debtor = heapq.heappop(debt)
        pay = min(-o_amt, -d_amt)
        out.append({"payer": debtor, "amount": pay,
                    "payee": creditor})
        if -o_amt > pay:
            heapq.heappush(owed, (o_amt + pay, creditor))
        if -d_amt > pay:
            heapq.heappush(debt, (d_amt + pay, debtor))
    return out


# Answer: Work in integer cents and decide where the
# remainder goes (to the payer, or round-robin among
# payees). Never use floats for money, and make the rule
# deterministic.
def split_equally(amount, people):
    # follow-up: cents that do not divide evenly. Work in
    # whole cents; the leftover cents go to the first
    # people
    # in sorted order, so the rule is always the same.
    people = sorted(people)
    base, extra = divmod(amount, len(people))
    return {p: base + (1 if i < extra else 0)
            for i, p in enumerate(people)}


# Answer: Replace the equal share with an explicit amount
```

```
# per payee in each record; the net balance per person is
# computed the same way.
def shares_of(record):
    # follow-up: explicit shares per person instead of
    # equal
    payees = record["payees"]
    if isinstance(payees, dict):
        if sum(payees.values()) != record["amount"]:
            raise ValueError("shares do not add up")
        return payees
    return split_equally(record["amount"], payees)
```

33. Playlist Shuffle No Adjacent Artists [M]

```
import heapq


import random


from collections import deque


# HELPERS for the base question
def release(waiting, heap):
    # the oldest waiting artist may play again (if it has
    # any song left)
    entry = waiting.popleft()
    if entry[0]:
        heapq.heappush(heap, entry)


def take_oldest(waiting, heap):
    # break the gap on purpose: the first waiting artist
    # that still has songs plays early
    while waiting:
        entry = waiting.popleft()
        if entry[0]:
            heapq.heappush(heap, entry)
            return


# BASE QUESTION (optimal): Greedy with a max-heap, holding
# back the last artist
# Idea: Keep a heap of (remaining songs, artist). Each
# step pop the artist with the most songs left; play one
# of their songs, and push them back only after the next
# artist has played (so they cannot repeat).
# Answer: Break ties between equally frequent artists
# randomly and pick the song within the artist at random,
# as in the code. The result is valid but not uniform
# among all valid shuffles; a uniform sampler needs
# counting arrangements.
# Answer: Same idea with a waiting queue of length K:
# after playing an artist, hold them for K - 1 more songs
# before they are eligible again (the LeetCode 358
# construction).
def shuffle_playlist(songs, gap=2, strict=False,
                     rng=random):
    # songs are (artist, title). The artist with the most
    # songs left plays next and then waits `gap - 1` other
    # songs before it may play again (gap 2 = never twice
    # in
    # a row). Ties between artists and the song picked
    # inside
    # an artist are random, so every run looks different.
    by_artist = {}
    for artist, title in songs:
        by_artist.setdefault(artist, []).append(title)
    for titles in by_artist.values():
        # FOLLOW-UP 1: random song order inside an artist
        rng.shuffle(titles)
    # FOLLOW-UP 1: random tie-break between artists
```

```
    heap = [(-len(t), rng.random(), a)
            for a, t in by_artist.items()]
    heapq.heapify(heap)
    waiting, out = deque(), []
    while len(out) < len(songs):
        if not heap:
            # everyone left is still waiting
            if strict:
                return None
            take_oldest(waiting, heap)
        left, _, artist = heapq.heappop(heap)
        out.append((artist, by_artist[artist].pop()))
        # an artist stays in the queue even with no songs
        # left: it still marks one position of spacing
        waiting.append((left + 1, rng.random(), artist))
        # FOLLOW-UP 2: gap=K keeps an artist out for K - 1
        # songs
        if len(waiting) >= gap:
            release(waiting, heap)
    return out
```

34. Minimum Makespan on Identical CPUs [M]

```
import heapq


# BASE QUESTION (optimal): Sorted jobs and a heap of free
# times
# Idea: Each job takes the CPU that frees first, but never
# starts before its release.
def min_makespan(release_times, num_cpus, duration):
    free = [0] * num_cpus
    last = 0
    for r in sorted(release_times):
        start = max(r, heapq.heappop(free))
        heapq.heappush(free, start + duration)
        last = max(last, start + duration)
    return last
```

35. Design server allocation for multi-type nodes [M]

```
import heapq


from collections import defaultdict


# BASE QUESTION (optimal): Per-type heaps ordered by
# flexibility, with lazy deletion
# Idea: For each type keep a heap of (number of supported
# types, id) for the servers that support it.
# Answer: Give every allocation a lease (expiry time). A
# periodic sweep returns servers whose lease has expired
# to the free pool (the lazy-deletion indexes make a
# release cheap), and clients renew the lease while they
# are alive.
class ServerPool:
    """Servers with several instance types. Each type has
    a heap of (number of types, id): the LEAST flexible
    server goes first, which keeps versatile ones free."""

    def __init__(self, servers):
        # servers: id -> set of instance types
        self.types = {s: frozenset(t)
                      for s, t in servers.items()}
        self.allocated = set()
        self.heaps = defaultdict(list)
        # FOLLOW-UP 4: a dead client's lease expires
        # lease expiry time of every allocated server
        self.lease = {}
        for s, ts in self.types.items():
            for t in ts:
                heap = self.heaps[t]
                heapq.heappush(heap, (len(ts), s))
```

```
def best_for(self, t, skip):
    # the best free server of type t that is not in
    # `skip`;
    # allocated entries are dropped lazily
    heap, held = self.heaps[t], []
    while heap:
        flex, s = heap[0]
        if s in self.allocated:
            heapq.heappop(heap)
        elif s in skip:
            held.append(heapq.heappop(heap))
        else:
            break
    best = heap[0] if heap else None
    # put back the entries we only looked past
    for item in held:
        heapq.heappush(heap, item)
    return best

def get_server(self, count, types):
    # pick servers first, commit only if the whole
    # request works
    chosen = []
    for t in types:
        # a server already chosen may cover this type
        if any(t in self.types[c] for c in chosen):
            continue
        top = self.best_for(t, set(chosen))
        if top is None:
            return None
        chosen.append(top[1])
    # fill up to `count` with the least flexible free
    # servers
    while len(chosen) < count:
        skip = set(chosen)
        found = [self.best_for(t, skip)
                 for t in types]
        options = [o for o in found if o]
        if not options:
            return None
        chosen.append(min(options)[1])
    # more servers than allowed were needed to cover
    # the types
    if len(chosen) > count:
        return None
    self.allocated |= set(chosen)
    return chosen

def release(self, ids):
    for s in ids:
        if s in self.allocated:
            self.allocated.discard(s)
            self.lease.pop(s, None)
            flex = len(self.types[s])
            for t in self.types[s]:
                heap = self.heaps[t]
                heapq.heappush(heap, (flex, s))

def rent(self, count, types, now, ttl):
    # a client that dies must not hold servers forever
    got = self.get_server(count, types)
    for s in got or []:
        self.lease[s] = now + ttl
    return got

def expire(self, now):
    # return servers whose lease ran out to the free
    # pool
    late = [s for s in self.lease
            if self.lease[s] <= now]
    self.release(late)
    return late
```

36. Huffman Tree Construction [H]

```
import heapq


# HELPERS for the base question
def huffman_lengths(freq):
    # code length of every symbol (depth in the tree)
    if not freq:
        return {}
    if len(freq) == 1:
        # a lone symbol still needs one bit
        return {next(iter(freq)): 1}
    # heap entries: (weight, tie-break, symbols below)
    heap = [(w, i, [s]) for i, (s, w) in
            enumerate(sorted(freq.items()))]
    heapq.heapify(heap)
    depth = {s: 0 for s in freq}
    tie = len(heap)
    while len(heap) > 1:
        # merge the two lightest; everything below goes
        # deeper
        w1, _, a = heapq.heappop(heap)
        w2, _, b = heapq.heappop(heap)
        for s in a + b:
            depth[s] += 1
        heapq.heappush(heap, (w1 + w2, tie, a + b))
        tie += 1
    return depth


# Answer: Keep only each symbol's code length from the
# tree, sort symbols by (length, symbol), and assign
# consecutive binary codes. A decoder then needs just the
# lengths rather than the whole tree, which is how DEFLATE
# stores its tables.
# Answer: Sort symbols by (code length, symbol) and assign
# consecutive binary numbers, shifting left when the
# length grows. A decoder rebuilds the codes from the
# lengths alone (this is how DEFLATE stores its tables).
def canonical_codes(lengths):
    # assign consecutive binary numbers by (length,
    # symbol)
    code, prev, out = 0, 0, {}
    for sym, length in sorted(lengths.items(),
                              key=lambda x: (x[1], x[0])):
        # moving to a longer length shifts the code left
        code <<= length - prev
        out[sym] = format(code, f"0{length}b")
        code += 1
        prev = length
    return out


# BASE QUESTION (optimal): Min-heap merge of the two
# lightest nodes
# Idea: Put every symbol in a min-heap keyed by frequency,
# repeatedly pop the two lightest nodes, merge them under
# a new internal node with the summed weight and push it
# back. The last node is the root.
def huffman_codes(freq):
    return canonical_codes(huffman_lengths(freq))
```

GROUP 5 OF 43 | 9 QUESTIONS, ONE CODE TEMPLATE
Recurrence on a position or a range (memo or table)

37. Target Expressions from Ordered Digits [M]

```
from collections import defaultdict


# HELPERS for the base question
def build(digits):
    # table[(l, r)] = {value: {(text, precedence)}}
    table = {}

    def solve(l, r):
        if (l, r) in table:
            return table[(l, r)]
        res = defaultdict(set)
        if r - l == 1:
            # a single digit is an atom (precedence 3)
            res[digits[l]].add((str(digits[l]), 3))
        # try every split point
        for m in range(l + 1, r):
            for lv, ls in solve(l, m).items():
                for rv, rs in solve(m, r).items():
                    combine(res, lv, ls, rv, rs)
        table[(l, r)] = res
        return res

    return solve(0, len(digits))


def combine(res, lv, ls, rv, rs):
    ops = (("+", 1, lv + rv), ("-", 1, lv - rv),
           ("*", 2, lv * rv))
    for lt, lp in ls:
        for rt, rp in rs:
            for op, prec, val in ops:
                # brackets only where precedence needs
                left = f"({lt})" if lp < prec else lt
                need = rp < prec
                if rp == prec and op == "-":
                    need = True
                right = f"({rt})" if need else rt
                res[val].add((left + op + right, prec))


# BASE QUESTION (optimal): Interval DP: all expressions
# for digits[l:r]
# Idea: Every expression over digits[l:r] is two sub-
# expressions over a split point joined by +, - or *.
# Memoize, for each interval, a map from numeric value to
# the set of expression strings that produce it.
def expressions_for_target(digits, target):
    found = build(digits).get(target, ())
    return sorted({text for text, _ in found})
```

38. Count Word Decompositions into Chemical Element Symbols [M]

```
# HELPERS used by the code below
ELEMENTS = set("""h he li be b c n o f ne na mg al si
p s cl ar k ca sc ti v cr mn fe co ni cu zn ga ge as
se br kr rb sr y zr nb mo tc ru rh pd ag cd in sn sb
te i xe cs ba la ce pr nd pm sm eu gd tb dy ho er tm
yb lu hf ta w re os ir pt au hg tl pb bi po at rn fr
ra ac th pa u np pu am cm bk cf es fm md no lr rf db
sg bh hs mt ds rg cn nh fl mc lv ts og""".split())


# BASE QUESTION (optimal): Suffix-count DP
# Idea: ways[i] is the number of decompositions of
# word[i:]. A symbol has one or two letters, so ways[i] =
# ways[i + 1] (if word[i] is a symbol) + ways[i + 2] (if
```

```
# word[i:i + 2] is a symbol).
def count_decompositions(word, symbols=ELEMENTS,
                         mod=None):
    w, n = word.lower(), len(word)
    # ways[i]: ways to split the suffix starting at i
    ways = [0] * (n + 2)
    ways[n] = 1
    for i in range(n - 1, -1, -1):
        # a one-letter symbol, then a two-letter symbol
        if w[i] in symbols:
            ways[i] += ways[i + 1]
        if i + 2 <= n and w[i:i + 2] in symbols:
            ways[i] += ways[i + 2]
        if mod:
            ways[i] %= mod
    return ways[0]
```

39. Find the longest palindromic subsequence using dynamic programming. [M]

```
# HELPERS for the base question
def lps_table(s):
    # dp[i][j] = longest palindromic SUBSEQUENCE of
    # s[i..j].
    # Equal ends are both used; otherwise drop one end.
    n = len(s)
    dp = [[0] * n for _ in range(n)]
    for i in range(n - 1, -1, -1):
        dp[i][i] = 1
        for j in range(i + 1, n):
            if s[i] == s[j]:
                dp[i][j] = dp[i + 1][j - 1] + 2
            else:
                dp[i][j] = max(dp[i + 1][j], dp[i][j - 1])
    return dp


# BASE QUESTION (optimal): Interval DP with a rolling row
# Idea: dp[i][j] is the length of the longest palindromic
# subsequence of s[i..j]. Fill i from the end to the start
# and j from i + 1 up: equal ends give dp[i + 1][j - 1] +
# 2, otherwise max(dp[i + 1][j], dp[i][j - 1]).
def lps_length(s):
    return lps_table(s)[0][-1] if s else 0
```

40. Jump Game with Profit [M]

```
import heapq


# HELPERS used by the code below
NEG = float("-inf")


# BASE QUESTION (optimal): Reading A: arr[i] is a maximum
# jump, profit[i] is gained on landing, maximize total
# profit to the last index
# Idea: Let best[i] be the largest profit achievable when
# standing on i. Landing on i from any j < i with j +
# arr[j] >= i gives best[i] = profit[i] + max(best[j]).
# Unreachable cells stay at minus infinity.
def max_profit_fast(arr, profit):
    # a max-heap of (-best, reach) with lazy deletion
    n = len(arr)
    best = [NEG] * n
    best[0] = profit[0]
    heap = [(-best[0], arr[0])]
    for i in range(1, n):
        # forget cells that can no longer reach i
        while heap and heap[0][1] < i:
            heapq.heappop(heap)
        if heap:
            best[i] = -heap[0][0] + profit[i]
            heapq.heappush(heap, (-best[i], i + arr[i]))
    return -1 if best[-1] == NEG else best[-1]
```

41. Fence Painting Minimum Operations (Gemini Team) [M]

```
# HELPERS for the base question
def sparse_table(a):
    # table[j][i] = index of the smallest plank in a[i : i
    # + 2^j]
    table = [list(range(len(a)))]
    j = 1
    while (1 << j) <= len(a):
        prev, row = table[-1], []
        for i in range(len(a) - (1 << j) + 1):
            x, y = prev[i], prev[i + (1 << (j - 1))]
            row.append(x if a[x] <= a[y] else y)
        table.append(row)
        j += 1
    return table


# Answer: Record which option won in each range and
# rebuild the list by walking the recursion again:
# horizontal bands as (row, l, r), vertical strokes as
# (plank).
def paint_strokes(a, width=None):
    # returns (number of strokes, the list of strokes)
    n = len(a)
    if n == 0:
        return 0, []
    table = sparse_table(a)

    def argmin(l, r):
        # smallest plank in a[l..r] in O(1)
        k = (r - l + 1).bit_length() - 1
        x, y = table[k][l], table[k][r - (1 << k) + 1]
        return x if a[x] <= a[y] else y

    def solve(l, r, base):
        # paint planks l..r whose bottom `base` rows are
        # done
        if l > r:
            return 0, []
        m = argmin(l, r)
        size = r - l + 1
        # a band of rows over the whole range needs one
        # stroke per row (or per `width` planks)
        per_row = 1
        if width is not None:
            per_row = -(-size // width)
        band = (a[m] - base) * per_row
        lc, ls = solve(l, m - 1, a[m])
        rc, rs = solve(m + 1, r, a[m])
        horizontal = band + lc + rc
        # or paint every plank by itself, top to bottom
        if size <= horizontal:
            planks = range(l, r + 1)
            return size, [("vertical", i) for i in planks]
        rows = [("horizontal", h, l, r)
                for h in range(base + 1, a[m] + 1)]
        return horizontal, rows + ls + rs

    return solve(0, n - 1, 0)


# BASE QUESTION (optimal): Same recursion with a sparse
# table for range minimum
# Idea: The recursion is correct; only the minimum lookup
# is slow. A sparse table answers range-minimum-index
# queries in O(1), and each plank becomes the split point
# at most once, so the recursion visits n ranges.
def paint(a, width=None):
    # FOLLOW-UP 1: width= limits the horizontal brush
    return paint_strokes(a, width)[0]
```

42. Maximize 1D deliveries within distance and minimize total distance [M]

```
# BASE QUESTION (optimal): DP over sorted orders and
# drivers, maximising (matches, -distance)
# Idea: Sort both lists. dp[i][j] is the best (matches,
# -total distance) using the first i orders and the first
# j drivers: skip order i, skip driver j, or match them if
# their distance is at most D.
def deliveries(orders, drivers, D):
    # first maximise the number of deliveries, then
    # minimise
    # the total distance: a DP over the sorted positions
    o, d = sorted(orders), sorted(drivers)
    # dp[i][j] = (deliveries, -distance) using the first i
    # orders and j drivers; tuples compare
    # lexicographically
    dp = [[(0, 0)] * (len(d) + 1)
          for _ in range(len(o) + 1)]
    for i in range(1, len(o) + 1):
        for j in range(1, len(d) + 1):
            # skip this order or skip this driver ...
            best = max(dp[i - 1][j], dp[i][j - 1])
            gap = abs(o[i - 1] - d[j - 1])
            if gap <= D:
                # ... or pair them
                count, neg = dp[i - 1][j - 1]
                best = max(best, (count + 1, neg - gap))
            dp[i][j] = best
    count, neg = dp[-1][-1]
    return count, -neg
```

43. Edit distance [H]

```
# BASE QUESTION (optimal): DP table with a rolling row
# Idea: dp[j] holds the distance from the current prefix
# of a to the first j characters of b.
def edit_distance(a, b):
    # dp[i][j]: edits to turn a[:i] into b[:j]
    m, n = len(a), len(b)
    dp = [[0] * (n + 1) for _ in range(m + 1)]
    for i in range(m + 1):
        dp[i][0] = i
    for j in range(n + 1):
        dp[0][j] = j
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if a[i - 1] == b[j - 1]:
                dp[i][j] = dp[i - 1][j - 1]
            else:
                swap = dp[i - 1][j - 1]
                drop = dp[i - 1][j]
                add = dp[i][j - 1]
                dp[i][j] = 1 + min(swap, drop, add)
    return dp[m][n], dp
```

44. Count Perfect Wake Numbers up to N [H]

```
from math import comb


# HELPERS for the base question
# Answer: Pick the k distinct digits from 1..9 (C(9, k)
# ways); the largest must be the peak, and each of the
# other k - 1 digits goes either left or right of it in
# sorted order, so 2^(k-1) arrangements: C(9, k) *
# 2^(k-1).
def with_k_digits(k):
    # choose the k digits; the largest is the peak and
    # each
    # other digit goes left or right of it
    return comb(9, k) * 2 ** (k - 1) if k >= 1 else 0


# BASE QUESTION (optimal): test a single number
```

```
# Idea: Check the three rules directly on the digit list:
# no '0', all digits distinct, and for each interior digit
# not (left > digit < right). The first and last digits
# are never valleys.
def is_perfect(n):
    # distinct digits, no zero, and no valley: no digit is
    # smaller than both of its neighbours
    d = list(map(int, str(n)))
    if 0 in d or len(set(d)) != len(d):
        return False
    return all(not (d[i - 1] > d[i] < d[i + 1])
               for i in range(1, len(d) - 1))
```

45. Word Break: All Sentences [H]

```
# BASE QUESTION (optimal): Memoized DFS on the start index
# Idea: The sentences of a suffix do not depend on how you
# reached it, so compute them once.
def word_break_all(s, word_dict):
    words = set(word_dict)
    memo = {}

    def sentences(start):
        if start == len(s):
            return [""]
        if start in memo:
            return memo[start]
        out = []
        for end in range(start + 1, len(s) + 1):
            if s[start:end] in words:
                for rest in sentences(end):
                    out.append(
                        s[start:end]
                        + (" " + rest if rest else "")
                    )
        memo[start] = out
        return out

    return sentences(0)
```

GROUP 6 OF 43 | 9 QUESTIONS, ONE CODE TEMPLATE
Best-first search with a heap: Dijkstra and its minimax twin

46. Path Minimizing the Maximum Cell (Swim in Rising Water) [M]

```
import heapq


# HELPERS for the base question
INF = float("inf")


def around(r, c, n, m):
    # four-neighbour cells inside an n x m grid
    for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
        a, b = r + dr, c + dc
        if 0 <= a < n and 0 <= b < m:
            yield a, b


# BASE QUESTION (optimal): Dijkstra on the running maximum
# Idea: Heap entries are (cost so far, cell). Popping a
# cell fixes its cost; each neighbour gets max(cost,
# neighbour value). Stop when the end cell is popped. Skip
# stale heap entries.
def min_max_path(g, start, end):
    n, m = len(g), len(g[0])
    # best[cell] = smallest possible "highest cell so far"
    best = {start: g[start[0]][start[1]]}
    heap = [(best[start], start)]
    while heap:
        cost, (r, c) = heapq.heappop(heap)
        # skip outdated heap entries
        if cost > best[(r, c)]:
            continue
        if (r, c) == end:
            return cost
        for a, b in around(r, c, n, m):
            # path cost = highest cell so far or this cell
            nxt = max(cost, g[a][b])
            if nxt < best.get((a, b), INF):
                best[(a, b)] = nxt
                heapq.heappush(heap, (nxt, (a, b)))
    return None
```

47. Nearest Favorite City by Travel Time [M]

```
import heapq


# BASE QUESTION (optimal): Dijkstra from the start
# Idea: Cities are finalised in order of distance, so the
# first favourite reached is the nearest.
def nearest_favorite(
    num_cities, roads, start, favorites
):
    adj = [[] for _ in range(num_cities)]
    for u, v, w in roads:
        adj[u].append((v, w))
        adj[v].append((u, w))
    dist = [float("inf")] * num_cities
    dist[start] = 0
    heap, wanted = [(0, start)], set(favorites)
    best_d, best_c = None, -1
    while heap:
        d, u = heapq.heappop(heap)
        if d > dist[u]:
            continue
        if best_d is not None and d > best_d:
            break
        if u in wanted and (
            best_d is None or u < best_c
```

```
        ):
            best_d, best_c = d, u
        for v, w in adj[u]:
            if d + w < dist[v]:
                dist[v] = d + w
                heapq.heappush(heap, (dist[v], v))
    return best_c
```

48. Given a train schedule with departure city, arrival city, start time, end time, and duration, and given a source and destination, find the sequence of trains that gets you there at the earliest possible time. [M]

```
import heapq


# BASE QUESTION (optimal): Dijkstra on earliest arrival
# times, with the train sequence
# Idea: best[city] is the earliest time you can be there.
# Answer: Replace the boarding test dep >= t with dep >= t
# + layover for every city except the origin. The
# earliest-arrival property still holds (a later arrival
# never helps because the layover is the same), so
# Dijkstra remains valid.
def earliest_route(trains, source, target, layover=0):
    # Dijkstra on arrival time. You may board a train if
    # it
    # leaves at or after your arrival plus the layover (no
    # layover is needed at the origin).
    by_city = {}
    for tr in trains:
        by_city.setdefault(tr.src, []).append(tr)
    best = {source: 0}
    took = {}
    heap = [(0, source)]
    while heap:
        time, city = heapq.heappop(heap)
        if time > best.get(city, float("inf")):
            continue
        if city == target:
            break
        # FOLLOW-UP 1: a minimum layover when changing
        # trains
        wait = 0 if city == source else layover
        for tr in by_city.get(city, []):
            old = best.get(tr.dst, float("inf"))
            if tr.dep >= time + wait and tr.arr < old:
                best[tr.dst] = tr.arr
                took[tr.dst] = tr
                heapq.heappush(heap, (tr.arr, tr.dst))
    if target not in best:
        return None
    # walk back from the target to list the trains used
    path, city = [], target
    while city != source:
        path.append(took[city])
        city = took[city].src
    return best[target], path[::-1]
```

49. Find the shortest path in a graph. [M]

```
import heapq


from collections import defaultdict


# HELPERS used by the code below
INF = float("inf")


# BASE QUESTION (optimal): Dijkstra with a heap (non-
# negative weights), returning the path
# Idea: Keep the best known distance to each node and a
# min-heap of (distance, node). Pop the closest unsettled
```

```
# node, relax its edges, and remember the parent of every
# improvement.
def dijkstra_path(edges, s, t):
    # weighted, non-negative edges (u, v, w)
    graph = defaultdict(list)
    for u, v, w in edges:
        graph[u].append((v, w))
    dist, parent, heap = {s: 0}, {s: None}, [(0, s)]
    while heap:
        d, u = heapq.heappop(heap)
        if d > dist[u]:
            continue
        if u == t:
            # the first pop of t is final: rebuild the
            # path
            path = []
            while u is not None:
                path.append(u)
                u = parent[u]
            return d, path[::-1]
        for v, w in graph[u]:
            if d + w < dist.get(v, INF):
                dist[v], parent[v] = d + w, u
                heapq.heappush(heap, (d + w, v))
    return None
```

50. Given a matrix with a rat, bread, cat, and obstacles, find a path from the rat to the bread that maximizes the minimum distance from cat. [M]

```
import heapq


from collections import deque


# HELPERS used by the code below
INF = 10 ** 9


def around(r, c, grid):
    # the open neighbours of a cell (obstacles are "#")
    for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
        nr, nc = r + dr, c + dc
        if 0 <= nr < len(grid) and 0 <= nc < len(grid[0]):
            if grid[nr][nc] != "#":
                yield nr, nc


def find(grid, ch):
    for r, row in enumerate(grid):
        if ch in row:
            return r, row.index(ch)
    return None


def cat_distances(grid):
    # one BFS from the cat gives every cell's distance to
    # it
    dist = [[INF] * len(grid[0]) for _ in grid]
    cat = find(grid, "C")
    if cat is None:
        return dist
    dist[cat[0]][cat[1]] = 0
    queue = deque([cat])
    while queue:
        r, c = queue.popleft()
        for nr, nc in around(r, c, grid):
            if dist[nr][nc] == INF:
                dist[nr][nc] = dist[r][c] + 1
                queue.append((nr, nc))
    return dist
# BASE QUESTION (optimal): Multi-source BFS from the cat,
# then a max-heap bottleneck search
```

```
# Idea: BFS from every cat gives the distance of each
# cell.
def safest_value(grid):
    # a path is as safe as its closest cell to the cat.
    # Always extend the safest frontier cell (like
    # Dijkstra
    # with max and min instead of min and sum).
    dist = cat_distances(grid)
    start, goal = find(grid, "R"), find(grid, "B")
    best = {start: dist[start[0]][start[1]]}
    heap = [(-best[start], start)]
    while heap:
        neg, cell = heapq.heappop(heap)
        if cell == goal:
            return -neg
        if -neg < best[cell]:
            continue
        for nr, nc in around(cell[0], cell[1], grid):
            safety = min(-neg, dist[nr][nc])
            if safety > best.get((nr, nc), -1):
                best[(nr, nc)] = safety
                heapq.heappush(heap, (-safety, (nr, nc)))
    return None
```

51. Minimize maximum height along a grid path [H]

```
import heapq


# HELPERS for the base question
INF = float("inf")


def around(r, c, m, n):
    for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
        a, b = r + dr, c + dc
        if 0 <= a < m and 0 <= b < n:
            yield a, b


# BASE QUESTION (optimal): Dijkstra with cost = maximum
# cell on the path
# Idea: Heap of (largest cell so far, cell). Pop the cell
# with the smallest value, extend to each neighbour with
# cost max(current, neighbour), and keep only
# improvements.
def min_max_path(H, with_path=False):
    # Dijkstra where a path costs the MAX of its cells
    m, n = len(H), len(H[0])
    best, parent = {(0, 0): H[0][0]}, {}
    heap = [(H[0][0], 0, 0)]
    while heap:
        cost, r, c = heapq.heappop(heap)
        if cost > best[(r, c)]:
            continue
        if (r, c) == (m - 1, n - 1):
            if not with_path:
                return cost
            path = [(r, c)]
            while (r, c) in parent:
                r, c = parent[(r, c)]
                path.append((r, c))
            return cost, path[::-1]
        for a, b in around(r, c, m, n):
            nxt = max(cost, H[a][b])
            if nxt < best.get((a, b), INF):
                best[(a, b)] = nxt
                parent[(a, b)] = (r, c)
                heapq.heappush(heap, (nxt, a, b))
```

52. Find minimum threshold enabling grid path [H]

```
import heapq


# HELPERS for the base question
INF = float("inf")


def around(r, c, m, n):
    for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
        a, b = r + dr, c + dc
        if 0 <= a < m and 0 <= b < n:
            yield a, b


# ALSO: the Dijkstra version
def best_path(H, step_cost, start_cost):
    # Dijkstra where a path costs the MAX of its steps;
    # the
    # first time a cell leaves the heap its cost is final
    m, n = len(H), len(H[0])
    best = {(0, 0): start_cost}
    parent = {}
    heap = [(start_cost, 0, 0)]
    while heap:
        cost, r, c = heapq.heappop(heap)
        if cost > best[(r, c)]:
            continue
        if (r, c) == (m - 1, n - 1):
            path = [(r, c)]
            while (r, c) in parent:
                r, c = parent[(r, c)]
                path.append((r, c))
            return cost, path[::-1]
        for a, b in around(r, c, m, n):
            nxt = max(cost, step_cost(H, r, c, a, b))
            if nxt < best.get((a, b), INF):
                best[(a, b)] = nxt
                parent[(a, b)] = (r, c)
                heapq.heappush(heap, (nxt, a, b))
    return None, []


# BASE QUESTION (optimal): Dijkstra with cost = maximum
# cell on the path
# Idea: Heap of (largest cell so far, cell). Pop the cell
# with the smallest value, extend to each neighbour with
# cost max(current, neighbour), and keep only
# improvements.
def min_max_path(H):
    # cost of a path = its largest cell value
    # (water rising over time is the same problem)
    def step(H, r, c, a, b):
        return H[a][b]
    return best_path(H, step, H[0][0])
```

53. Minimize travel time with optional meeting point [H]

```
import heapq


# HELPERS used by the code below
INF = float("inf")


# HELPERS for the base question
def dijkstra(graph, source):
    dist = [INF] * len(graph)
    dist[source] = 0
    heap = [(0, source)]
    while heap:
        d, u = heapq.heappop(heap)
        if d > dist[u]:
```

```
            continue
        for v, w in graph[u]:
            if d + w < dist[v]:
                dist[v] = d + w
                heapq.heappush(heap, (d + w, v))
    return dist


def build(n, edges, directed):
    # forward graph, and the reversed graph for "to dest"
    fwd = [[] for _ in range(n)]
    rev = [[] for _ in range(n)]
    for u, v, w in edges:
        fwd[u].append((v, w))
        rev[v].append((u, w))
        if not directed:
            fwd[v].append((u, w))
            rev[u].append((v, w))
    return fwd, rev


# BASE QUESTION (optimal): Three Dijkstra runs (from A,
# from B, and into the destination)
# Idea: distA and distB come from forward searches. The
# distance from every node to the destination is a search
# from the destination on the reversed graph (the same
# graph if undirected).
def meeting_point(n, edges, a, b, dest, directed=False):
    # both start at the same time, meet at node m, wait
    # for
    # each other, then travel together to dest. Meeting at
    # dest itself covers the "no meeting" plan.
    fwd, rev = build(n, edges, directed)
    da, db = dijkstra(fwd, a), dijkstra(fwd, b)
    # distance from every node TO dest = search the
    # reverse graph
    to_dest = dijkstra(rev, dest)
    best = (INF, -1)
    for m in range(n):
        total = max(da[m], db[m]) + to_dest[m]
        if total < best[0]:
            best = (total, m)
    return [-1, -1] if best[0] == INF else list(best)
```

54. Compute minimax grid path and network delay [H]

```
import heapq


# HELPERS for the base question
INF = float("inf")


def neighbors(r, c, m, n):
    for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
        a, b = r + dr, c + dc
        if 0 <= a < m and 0 <= b < n:
            yield a, b


# BASE QUESTION (optimal): Dijkstra where the cost of a
# path is its maximum cell
# Idea: Heap of (best maximum so far, cell). Pop the cell
# with the smallest bottleneck, and for each neighbour the
# new bottleneck is max(current, neighbour). Improve if
# smaller.
def minimax_path(H):
    # Part 1: the path whose highest cell is as low as
    # possible
    m, n = len(H), len(H[0])
    best = {(0, 0): H[0][0]}
    heap = [(H[0][0], 0, 0)]
    while heap:
```

```
        cost, r, c = heapq.heappop(heap)
        if cost > best[(r, c)]:
            continue
        if (r, c) == (m - 1, n - 1):
            return cost
        for a, b in neighbors(r, c, m, n):
            # the cost of a path is its maximum, never a
            # sum
            nxt = max(cost, H[a][b])
            if nxt < best.get((a, b), INF):
                best[(a, b)] = nxt
                heapq.heappush(heap, (nxt, a, b))
    return -1
```

GROUP 7 OF 43 | 9 QUESTIONS, ONE CODE TEMPLATE
Backtracking: choose, recurse, undo

55. Mahjong Winning Hand Detection (14 Tiles) [M]

```
from collections import Counter


# HELPERS for the base question
def is_run_start(tile, count):
    # numbered suits are 0..2 with ranks 1..9; the tile
    # id is suit * 10 + rank. Honours (30+) never run.
    ok = tile < 30 and tile % 10 <= 7
    nxt, nxt2 = count.get(tile + 1), count.get(tile + 2)
    return ok and nxt and nxt2


def melds_left(count, need):
    # can the tiles in `count` form exactly `need` melds?
    live = [t for t in sorted(count) if count[t]]
    tile = live[0] if live else None
    if tile is None:
        return need == 0
    if need == 0:
        return False
    # the smallest tile is either in a triplet ...
    if count[tile] >= 3:
        count[tile] -= 3
        ok = melds_left(count, need - 1)
        count[tile] += 3
        if ok:
            return True
    # ... or starts a run of three
    if is_run_start(tile, count):
        for t in (tile, tile + 1, tile + 2):
            count[t] -= 1
        ok = melds_left(count, need - 1)
        for t in (tile, tile + 1, tile + 2):
            count[t] += 1
        return ok
    return False


# BASE QUESTION (optimal): Try each pair, then greedy-
# smallest with two options
# Idea: Tiles are encoded as suit * 10 + rank for the
# numbered suits (ranks 1 to 9, suits 0 to 2) and 30 and
# above for honours, which never form runs.
# Answer: Ask what happens to the other six tiles before
# coding: if they are free, check whether any 8 tiles form
# a pair and two melds with the same search over subsets
# of the 14.
def is_winning_hand(hand, melds=4):
    # FOLLOW-UP 1: melds=2 handles the 8-tile variant
    # one pair plus `melds` melds uses every tile
    if len(hand) != 2 + 3 * melds:
        return False
    count = Counter(hand)
    # try every tile value as the pair
    for tile in list(count):
        if count[tile] >= 2:
            count[tile] -= 2
            ok = melds_left(count, melds)
            count[tile] += 2
            if ok:
                return True
    return False
```

56. Count Prefix-Matching Simple Paths in a Labeled Graph [M]

```
from collections import defaultdict


# BASE QUESTION (optimal): DFS with label pruning and a
# visited bitmask
# Idea: Start from every vertex whose label equals the
# first query letter. Extend to an unvisited neighbour
# only if its label equals the next query letter, adding
# one to the count for the current length at each step.
def prefix_paths(labels, edges, query, undirected=False):
    # counts[d]: simple paths whose first d + 1 labels
    # spell
    # the first d + 1 letters of the query
    adj = defaultdict(list)
    for u, v in edges:
        adj[u].append(v)
        # FOLLOW-UP 1: undirected=True adds both
        # directions
        if undirected:
            adj[v].append(u)
    counts = [0] * len(query)

    def go(u, depth, mask):
        counts[depth] += 1
        if depth + 1 == len(query):
            return
        for w in adj[u]:
            # a simple path never revisits a vertex
            if mask >> w & 1:
                continue
            if labels[w] == query[depth + 1]:
                go(w, depth + 1, mask | 1 << w)

    # a path may start at any vertex with the first letter
    for s in range(len(labels)):
        if query and labels[s] == query[0]:
            go(s, 0, 1 << s)
    return counts
```

57. Generate Parentheses [M]

```
# BASE QUESTION (optimal): Backtracking with open and
# close counters
# Idea: Track how many '(' and ')' have been placed. Add
# '(' if open < n; add ')' if close < open. When the
# string reaches length 2n it is valid.
def generate(n):
    # build the string one character at a time. Adding "("
    # is allowed while opens < n; adding ")" only while
    # closes < opens: that rule keeps every string valid.
    out = []

    def build(text, opens, closes):
        if len(text) == 2 * n:
            out.append(text)
            return
        if opens < n:
            build(text + "(", opens + 1, closes)
        if closes < opens:
            build(text + ")", opens, closes + 1)

    build("", 0, 0)
    return out
```

58. American Football Score Combinations [M]

```
# BASE QUESTION (optimal): Backtracking in increasing
# order
# Idea: Choosing plays in non-decreasing order lists every
# multiset exactly once.
def football_combinations(total):
    plays = (2, 3, 6, 7, 8)
    out = []

    def dfs(start, left, path):
        if left == 0:
            out.append(path[:])
            return
        for i in range(start, len(plays)):
            if plays[i] <= left:
                path.append(plays[i])
                # the same play may be used again, so i
                # stays
                dfs(i, left - plays[i], path)
                path.pop()

    dfs(0, total, [])
    return out
```

59. Permutations [M]

```
# BASE QUESTION (optimal): Backtracking with a used array
# Idea: Fill positions left to right. At each position try
# every unused value, mark it, recurse, and unmark. Only
# valid prefixes are ever extended, so every step leads to
# a permutation.
def permute(nums):
    # backtracking: choose an unused element, recurse,
    # undo
    out, cur, used = [], [], [False] * len(nums)

    def go():
        if len(cur) == len(nums):
            out.append(cur[:])
            return
        for i, x in enumerate(nums):
            if not used[i]:
                used[i] = True
                cur.append(x)
                go()
                cur.pop()
                used[i] = False

    go()
    return out
```

60. Determine if a 14-tile hand is winning [M]

```
from collections import Counter


# HELPERS for the base question
def melds_ok(cnt, v=1):
    # can the tiles in cnt (value -> count) form only
    # melds?
    while v <= 9 and cnt[v] == 0:
        v += 1
    if v > 9:
        return True
    # the smallest tile is either in a triplet ...
    if cnt[v] >= 3:
        cnt[v] -= 3
        ok = melds_ok(cnt, v)
        cnt[v] += 3
        if ok:
            return True
    # ... or the start of a run v, v + 1, v + 2
    if v <= 7 and cnt[v + 1] and cnt[v + 2]:
```

```
        for t in (v, v + 1, v + 2):
            cnt[t] -= 1
        ok = melds_ok(cnt, v)
        for t in (v, v + 1, v + 2):
            cnt[t] += 1
        return ok
    return False


# BASE QUESTION (optimal): Counts, try each pair, then
# always group the smallest remaining tile
# Idea: Keep counts of 1..9. For each value with at least
# two tiles, take it as the pair.
def is_winning_hand(tiles):
    # one pair plus four melds, single suit, values 1..9
    cnt = Counter({v: 0 for v in range(1, 12)})
    for t in tiles:
        cnt[t] += 1
    for pair in range(1, 10):
        if cnt[pair] >= 2:
            cnt[pair] -= 2
            ok = melds_ok(cnt)
            cnt[pair] += 2
            if ok:
                return True
    return False
```

61. Count Passcode Patterns on a Keypad [M]

```
# BASE QUESTION (optimal): DFS with backtracking from
# every key
# Idea: Extend the path one adjacent unused key at a time
# and undo the choice afterwards.
def count_patterns(rows, cols, k):
    def dfs(r, c, used, seen):
        if used == k:
            return 1
        total = 0
        for dr, dc in (
            (1, 0),
            (-1, 0),
            (0, 1),
            (0, -1),
        ):
            nr, nc = r + dr, c + dc
            if (
                0 <= nr < rows
                and 0 <= nc < cols
                and (nr, nc) not in seen
            ):
                seen.add((nr, nc))
                total += dfs(nr, nc, used + 1, seen)
                seen.remove((nr, nc))
        return total

    return sum(
        dfs(r, c, 1, {(r, c)})
        for r in range(rows)
        for c in range(cols)
    )
```

62. Match people to questions using tags and priorities [M]

```
# BASE QUESTION (optimal): Augmenting paths (Kuhn's
# algorithm)
# Idea: For each person run a DFS that tries each
# acceptable question: if it is free take it, otherwise
# try to move its current owner to another question
# (recursively), using a visited set so no question is
# tried twice in one attempt.
def max_matching(adj):
    # the plain version: adj[p] = list of questions for
    # person p
    owner = {}
```

```
    def try_assign(p, seen):
        for q in adj[p]:
            if q in seen:
                continue
            seen.add(q)
            free = q not in owner
            if free or try_assign(owner[q], seen):
                owner[q] = p
                return True
        return False

    people = range(len(adj))
    return sum(try_assign(p, set()) for p in people)
```

63. Matrix Flower Placement with House Adjacency [H]

```
# HELPERS for the base question
STEPS = ((1, 0), (-1, 0), (0, 1), (0, -1))


# BASE QUESTION (optimal): Row-by-row backtracking with
# used-column set and delayed house checks
# Idea: Place at most one flower per row, tracking used
# columns, and only on empty cells ('0').
def place_flowers(grid, exactly_one=False,
                  find_all=False):
    # 'H' house, '0' empty. At most one F per row and
    # column; every house needs an F on a side.
    R, C = len(grid), len(grid[0])
    cells = [list(row) for row in grid]
    houses = [(r, c) for r in range(R) for c in range(C)
              if cells[r][c] == "H"]
    found = []

    def house_ok(r, c):
        # is some neighbour of this house a flower?
        for dr, dc in STEPS:
            a, b = r + dr, c + dc
            if 0 <= a < R and 0 <= b < C \
                    and cells[a][b] == "F":
                return True
        return False

    def row_houses_ok(r):
        return all(house_ok(hr, hc)
                   for hr, hc in houses if hr == r)

    def solve(r, used):
        # FOLLOW-UP 4: find_all=True lists every solution
        if found and not find_all:
            return
        # rows up to r-1 are decided, so row r-2 is final
        if r >= 2 and not row_houses_ok(r - 2):
            return
        if r == R:
            # FOLLOW-UP 3: exactly_one=True: a flower in
            # every column
            full = len(used) == C
            if row_houses_ok(R - 1) and (full or
                                         not exactly_one):
                found.append(["".join(x) for x in cells])
            return
        # try a flower in every free column of this row
        for c in range(C):
            if c not in used and cells[r][c] == "0":
                cells[r][c] = "F"
                solve(r + 1, used | {c})
                cells[r][c] = "0"
        # or leave the row empty (not allowed for exactly
        # one)
        if not exactly_one:
            solve(r + 1, used)
    solve(0, frozenset())
```

```
if find_all:
    return found
return found[0] if found else None
```

GROUP 8 OF 43 | 9 QUESTIONS, ONE CODE TEMPLATE
Runs and groups: extend while it fits, restart or close when it does not

64. Group Timestamps Into Close Intervals [E]

```
# BASE QUESTION (optimal): One pass with a run start
# Idea: Close the run whenever the gap to the previous
# value reaches k.
def group_into_intervals(timestamps, k):
    if not timestamps:
        return []
    runs, start = [], timestamps[0]
    for prev, cur in zip(timestamps, timestamps[1:]):
        if cur - prev >= k:
            runs.append([start, prev])
            start = cur
    runs.append([start, timestamps[-1]])
    return runs
```

65. First Out-of-Order Character [E]

```
# BASE QUESTION (optimal): One pass, compare with the
# previous letter
# Idea: Order is broken exactly where a letter is smaller
# than the one before it.
def first_out_of_order(s):
    for i in range(1, len(s)):
        if s[i] < s[i - 1]:
            return i
    return -1
```

66. Longest Non-Decreasing Subarray with One Replacement [M]

```
# HELPERS for the base question
def run_lengths(a, ok):
    n = len(a)
    # left[i]: run ending at i, right[i]: starting at i
    left, right = [1] * n, [1] * n
    for i in range(1, n):
        if ok(a[i - 1], a[i]):
            left[i] = left[i - 1] + 1
    for i in range(n - 2, -1, -1):
        if ok(a[i], a[i + 1]):
            right[i] = right[i + 1] + 1
    return left, right


# BASE QUESTION (optimal): Left and right run lengths
# Idea: left[i] and right[i] are the non-decreasing run
# lengths ending and starting at i. Changing a[i] gives
# left[i-1] + 1 + right[i+1] if a[i-1] <= a[i+1],
# otherwise max(left[i-1], right[i+1]) + 1.
# Answer: Use < in the run lengths, and when the replaced
# value must fit between the neighbours check a[i+1] -
# a[i-1] >= 2 (an integer strictly between them exists)
# instead of a[i-1] <= a[i+1]. Clarify strict versus non-
# strict before coding.
def longest_with_one_change(a, strict=False):
    n = len(a)
    # nothing to fix in arrays of length 0, 1 or 2
    if n <= 2:
        return n
    # FOLLOW-UP 1: strict=True means strictly increasing
    # non-decreasing uses <=, strictly increasing uses <
    def ok(x, y):
        return x < y if strict else x <= y

    # can a value fit between the two neighbours?
    gap = 2 if strict else 0
    left, right = run_lengths(a, ok)
    best = 1
    # try changing each element a[i]
```

```
    for i in range(n):
        L = left[i - 1] if i else 0
        R = right[i + 1] if i + 1 < n else 0
        # at an end, or when a value fits between the
        # neighbours, the change joins both runs
        if not L or not R or a[i + 1] - a[i - 1] >= gap:
            best = max(best, L + 1 + R)
        # otherwise it joins only the longer run
        else:
            best = max(best, max(L, R) + 1)
    return best
```

67. Piano Hand Movement Grouping [M]

```
# HELPERS for the base question
def hand_positions(notes, k):
    # split the notes into groups that one hand position
    # (a span of at most k) can play, as early as possible
    groups = []
    for i, x in enumerate(notes):
        if groups:
            start, lo, hi = groups[-1]
            # does the note fit the current span?
            lo2, hi2 = min(lo, x), max(hi, x)
            if hi2 - lo2 <= k:
                groups[-1] = (start, lo2, hi2)
                continue
        # otherwise the hand must move: a new group starts
        groups.append((i, x, x))
    return groups


# BASE QUESTION (optimal): Greedy scan, tracking the
# group's smallest and largest note
# Idea: Keep lo and hi of the current group. Extend while
# hi - lo stays within k, otherwise close the group and
# start a new one at this note.
def groups(notes, k):
    # how many hand placements are needed
    return len(hand_positions(notes, k))
```

68. Find longest increasing contiguous subarray [M]

```
# BASE QUESTION (optimal): Left and right run lengths, try
# every position as the changed element
# Idea: left[i] is the length of the strictly increasing
# run ending at i and right[i] the length of the one
# starting at i.
def longest_one_change(a, integers=True):
    # strictly increasing after changing AT MOST one value
    n = len(a)
    if n == 0:
        return 0
    # left[i]: run ending at i; right[i]: run starting at
    # i
    left, right = [1] * n, [1] * n
    for i in range(1, n):
        if a[i] > a[i - 1]:
            left[i] = left[i - 1] + 1
    for i in range(n - 2, -1, -1):
        if a[i] < a[i + 1]:
            right[i] = right[i + 1] + 1
    # the gap needed between two neighbours: 2 for whole
    # numbers (one number must fit between), > 0 for reals
    best = max(left)
    for i in range(n):
        # change a[i] to join the run on its left ...
        if i > 0:
            best = max(best, left[i - 1] + 1)
        # ... or the run on its right ...
        if i < n - 1:
            best = max(best, right[i + 1] + 1)
        # ... or both, when a value fits between the
        # neighbours
```

```
        if 0 < i < n - 1:
            room = a[i + 1] - a[i - 1]
            fits = room >= 2 if integers else room > 0
            if fits:
                joined = left[i - 1] + 1 + right[i + 1]
                best = max(best, joined)
    return min(best, n)
```

69. Sum of Good Subarrays (Adjacent Diff ±1) [M]

```
# BASE QUESTION (optimal): Two running run totals
# Idea: For an increasing run of length L ending at i let
# S be the sum of all suffix sums of the run. When the
# next element x extends it, S becomes S + (L + 1) * x and
# L becomes L + 1; a break restarts at S = x, L = 1.
def good_subarray_sum(a):
    # a good subarray has all steps +1 or all steps -1.
    # For each index keep, per direction, the sum of the
    # sums of all good subarrays that END here.
    total = 0
    up_sum = up_len = down_sum = down_len = 0
    for i, x in enumerate(a):
        step = x - a[i - 1] if i else None
        if step == 1:
            # extend the +1 run: every old subarray grows
            # by
            # x, and [x] alone is new: (len + 1) * x in
            # all
            up_sum += (up_len + 1) * x
            up_len += 1
        else:
            up_sum, up_len = x, 1
        if step == -1:
            down_sum += (down_len + 1) * x
            down_len += 1
        else:
            down_sum, down_len = x, 1
        # [x] is in both totals: count it once
        total += up_sum + down_sum - x
    return total
```

70. Sustained-Error Time Window [M]

```
# BASE QUESTION (optimal): Expand around the reading
# Idea: The longest valid run around the index is the best
# one, so only its span matters.
def has_sustained_error_window(
    times, errors, second, interval, baseline
):
    at = times.index(second)
    if errors[at] < baseline:
        return False
    left = right = at
    while left > 0 and errors[left - 1] >= baseline:
        left -= 1
    while (
        right + 1 < len(times)
        and errors[right + 1] >= baseline
    ):
        right += 1
    return times[right] - times[left] >= interval
```

71. Longest Non-Decreasing Subarray From Two Arrays [M]

```
# BASE QUESTION (optimal): Two-state DP
# Idea: dp1 and dp2 are the best run ending at i using
# nums1[i] or nums2[i].
def longest_non_decreasing_subarray(nums1, nums2):
    dp1 = dp2 = best = 1
    for i in range(1, len(nums1)):
        n1 = n2 = 1
        for prev_val, prev_len in (
            (nums1[i - 1], dp1),
            (nums2[i - 1], dp2),
```

```
        ):
            if prev_val <= nums1[i]:
                n1 = max(n1, prev_len + 1)
            if prev_val <= nums2[i]:
                n2 = max(n2, prev_len + 1)
        dp1, dp2 = n1, n2
        best = max(best, dp1, dp2)
    return best
```

72. Count Super Streaks [M]

```
# BASE QUESTION (optimal): One pass over the sorted events
# Idea: Keep only the open streak's start, last time, size
# and type.
def count_super_streaks(events, N, T, X):
    count = 0
    start = prev = kind = None
    size = 0
    for _, etype, etime in sorted(
        events, key=lambda e: e[2]
    ):
        if kind == etype and etime - prev <= T:
            size += 1
        else:
            if size >= N and prev - start >= X:
                count += 1
            start, kind, size = etime, etype, 1
        prev = etime
    if size >= N and prev - start >= X:
        count += 1
    return count
```

GROUP 9 OF 43 | 8 QUESTIONS, ONE CODE TEMPLATE
Union-find: merge the pairs, then ask who is together

73. Count Islands in a Grid and Track the Count as Land Cells Are Added [M]

```
# HELPERS for the base question
STEPS = ((1, 0), (-1, 0), (0, 1), (0, -1))


# BASE QUESTION (optimal): Union-Find updated cell by cell
# Idea: Build the initial islands with unions (count
# them), then for each addition: ignore it if the cell is
# already land; otherwise make it a new set (count + 1)
# and union it with each land neighbour, subtracting one
# from the count every time two different sets merge.
class Islands:
    def __init__(self):
        # land cell -> parent cell, and sizes of the roots
        self.parent, self.size = {}, {}
        self.count, self.largest = 0, 0

    def find(self, x):
        # path halving keeps the trees flat
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]
            x = self.parent[x]
        return x

    def add(self, r, c):
        if (r, c) in self.parent:
            return
        self.parent[(r, c)] = (r, c)
        self.size[(r, c)] = 1
        self.count += 1
        self.largest = max(self.largest, 1)
        # join with every land neighbour
        for dr, dc in STEPS:
            nb = (r + dr, c + dc)
            if nb in self.parent:
                self.union((r, c), nb)

    def union(self, a, b):
        a, b = self.find(a), self.find(b)
        if a == b:
            return
        # union by size: the small island joins the big
        # one
        if self.size[a] < self.size[b]:
            a, b = b, a
        self.parent[b] = a
        self.size[a] += self.size[b]
        self.count -= 1
        self.largest = max(self.largest, self.size[a])


# BASE QUESTION (optimal): Union-Find updated cell by cell
# Idea: Build the initial islands with unions (count
# them), then for each addition: ignore it if the cell is
# already land; otherwise make it a new set (count + 1)
# and union it with each land neighbour, subtracting one
# from the count every time two different sets merge.
def island_counts(grid, additions):
    # counts before any addition, then after each one
    isl = Islands()
    for r, row in enumerate(grid):
        for c, v in enumerate(row):
            if v == 1:
                isl.add(r, c)
    out = [isl.count]
    for r, c in additions:
        isl.add(r, c)
```

```
        out.append(isl.count)
    return out
```

74. 2D Points Connected Components by Radius [M]

```
from collections import defaultdict


# HELPERS for the base question
class UnionFind:
    def __init__(self, n):
        self.parent = list(range(n))

    def find(self, x):
        # path halving keeps the trees flat
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]
            x = self.parent[x]
        return x

    def union(self, a, b):
        self.parent[self.find(a)] = self.find(b)


# Answer: sqrt is slow and inexact; dist <= r is the same
# as dist^2 <= r^2 for non-negative numbers, and with
# integer coordinates the comparison is exact. It also
# avoids floating-point surprises right at the boundary.
def near(p, q, r):
    # squared distance: no square root, exact for ints
    dx, dy = p[0] - q[0], p[1] - q[1]
    return dx * dx + dy * dy <= r * r


# BASE QUESTION (optimal): Union-find with a spatial grid
# Idea: Put each point in the cell (x // r, y // r). Two
# points within distance r are in the same or adjacent
# cells, so only those 9 cells are compared.
def count_clusters(points, r):
    # grid of cells r wide: only 9 cells need a look
    size = r if r > 0 else 1
    cells = defaultdict(list)
    for i, (x, y) in enumerate(points):
        cells[(x // size, y // size)].append(i)
    uf = UnionFind(len(points))
    for i, (x, y) in enumerate(points):
        cx, cy = x // size, y // size
        for dx in (-1, 0, 1):
            for dy in (-1, 0, 1):
                cell = cells.get((cx + dx, cy + dy), ())
                for j in cell:
                    q = points[j]
                    if j > i and near((x, y), q, r):
                        uf.union(i, j)
    return len({uf.find(i) for i in range(len(points))})
```

75. Count Clusters of 2D Points Within a Radius [M]

```
from collections import Counter, defaultdict


# HELPERS for the base question
# Answer: Insert the point into its grid cell, union it
# with every point within r in the 9 surrounding cells,
# and keep the component count: components += 1 - (number
# of distinct roots merged).
class Clusters:
    """Points arrive one by one; count the clusters."""

    def __init__(self, r):
        self.r = r
        # grid cell (side r) -> indices of points inside
        self.cells = defaultdict(list)
        self.points, self.parent = [], []
```

```
        self.components = 0

    def find(self, x):
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]
            x = self.parent[x]
        return x

    def near(self, p, q):
        dx, dy = p[0] - q[0], p[1] - q[1]
        return dx * dx + dy * dy <= self.r * self.r

    def add(self, x, y):
        i = len(self.points)
        self.points.append((x, y))
        self.parent.append(i)
        self.components += 1
        size = self.r or 1
        cx, cy = x // size, y // size
        self.cells[(cx, cy)].append(i)
        # look only at the 9 cells around the new point
        for dx in (-1, 0, 1):
            for dy in (-1, 0, 1):
                key = (cx + dx, cy + dy)
                cell = self.cells.get(key, ())
                for j in cell:
                    q = self.points[j]
                    if j != i and self.near((x, y), q):
                        a, b = self.find(i), self.find(j)
                        if a != b:
                            self.parent[a] = b
                            self.components -= 1
        return self.components

    def largest(self):
        sizes = Counter(self.find(i)
                        for i in range(len(self.points)))
        return max(sizes.values(), default=0)


# BASE QUESTION (optimal): Grid hashing with cells of side
# r, then Union-Find
# Idea: Put each point in the cell (x // r, y // r). A
# point within distance r of another can only be in the
# same cell or one of the 8 neighbouring cells, so compare
# only with those.
def count_clusters(points, r):
    c = Clusters(r)
    for x, y in points:
        c.add(x, y)
    return c.components
```

76. Find A Threshold-Limited Path With Minimum Required Safety [M]

```
# HELPERS for the base question
class UnionFind:
    def __init__(self, n):
        self.parent = list(range(n))

    def find(self, x):
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]
            x = self.parent[x]
        return x

    def union(self, a, b):
        self.parent[self.find(a)] = self.find(b)


# BASE QUESTION (optimal): Union-Find adding edges from
# strongest to weakest
# Idea: Sort the edges by safety, strongest first, and
# union their endpoints one by one.
```

```
def best_path_safety(n, edges, s, t, threshold=None):
    # edges are (u, v, safety). The safety of a path is
    # its
    # weakest edge; return the best safety, or -1 if s and
    # t
    # are not connected through edges >= threshold
    if s == t:
        return float("inf")
    uf = UnionFind(n)
    # add the safest edges first: the edge that joins s
    # and t
    # is the weakest edge of the best path
    for u, v, w in sorted(edges, key=lambda e: -e[2]):
        if threshold is not None and w < threshold:
            break
        uf.union(u, v)
        if uf.find(s) == uf.find(t):
            return w
    return -1
```

77. Movie Recommendations via Transitive Similarity [M]

```
# BASE QUESTION (optimal): Union-find and a sort
# Idea: The target's component is its group; rank it by
# rating.
def recommend(target, ratings, n, similarities):
    parent = {}

    def find(x):
        parent.setdefault(x, x)
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x

    for x, y in similarities:
        parent[find(x)] = find(y)
    root = find(target)
    group = [
        m
        for m in list(parent)
        if m != target and find(m) == root
    ]
    group.sort(key=lambda m: (-ratings[m], m))
    return group[:n]
```

78. Determine Reporting Relationships [M]

```
# BASE QUESTION (optimal): Union-Find for peer groups with
# a manager stored on each group
# Idea: Each employee starts in its own group. add_peer
# unions the two groups (and keeps the manager, if either
# has one). add_manager sets the manager of b's group to
# a.
# Answer: Collect the chain of managers above one employee
# into a set, then climb from the other until you meet it.
# O(depth).
class Org:
    def __init__(self):
        # union-find over peers (people with the same
        # manager)
        self.parent = {}
        # peer-group root -> the manager of that group
        self.boss = {}

    def find(self, x):
        self.parent.setdefault(x, x)
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]
            x = self.parent[x]
        return x

    def add_peer(self, a, b):
        ra, rb = self.find(a), self.find(b)
```

```
        if ra == rb:
            return
        # two groups with different managers contradict
        if ra in self.boss and rb in self.boss \
                and self.boss[ra] != self.boss[rb]:
            # FOLLOW-UP 2: contradictory facts are
            # rejected
            raise ValueError("conflicting managers")
        self.parent[rb] = ra
        # the merged group keeps whichever manager is
        # known
        if rb in self.boss and ra not in self.boss:
            self.boss[ra] = self.boss[rb]

    def add_manager(self, a, b):
        # a is the direct manager of b (and b's peers)
        self.find(a)
        root = self.find(b)
        if root in self.boss and self.boss[root] != a:
            raise ValueError("conflicting managers")
        self.boss[root] = a

    def manager_of(self, x):
        return self.boss.get(self.find(x))

    def is_manager(self, a, b):
        # climb b's chain of managers; a visited set stops
        # cycles
        seen, cur = set(), b
        while cur is not None and cur not in seen:
            seen.add(cur)
            cur = self.manager_of(cur)
            if cur == a:
                return True
        return False

    # FOLLOW-UP 1: the closest common manager
    def common_manager(self, a, b):
        # the closest manager above both people
        chain, cur = set(), self.manager_of(a)
        while cur is not None and cur not in chain:
            chain.add(cur)
            cur = self.manager_of(cur)
        cur, seen = self.manager_of(b), set()
        while cur is not None and cur not in seen:
            if cur in chain:
                return cur
            seen.add(cur)
            cur = self.manager_of(cur)
        return None
```

79. Determine whether two nodes are related [M]

```
# BASE QUESTION (optimal): Union-Find built once,
# constant-time queries
# Idea: Union the endpoints of every relationship; two
# nodes are related exactly when they have the same root.
class Relations:
    """Related = joined by a chain of relations."""

    def __init__(self, n, edges=()):
        self.parent = list(range(n))
        self.size = [1] * n
        for u, v in edges:
            self.union(u, v)

    def find(self, x):
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]
            x = self.parent[x]
        return x

    def union(self, a, b):
```

```
        # relations can arrive over time: near-constant
        # time
        ra, rb = self.find(a), self.find(b)
        if ra == rb:
            return
        # the smaller group joins the bigger one
        if self.size[ra] < self.size[rb]:
            ra, rb = rb, ra
        self.parent[rb] = ra
        self.size[ra] += self.size[rb]

    def related(self, a, b):
        return self.find(a) == self.find(b)
```

80. Friend Graph with Timeline (Friend / Unfriend Events) [H]

```
# HELPERS for the base question
def by_time(events):
    # events are (time, a, b, "friend" or "unfriend")
    return sorted(events, key=lambda e: e[0])


# Answer: Union by size without path compression,
# recording the time of each union on the edge it creates;
# two people are connected at t if the largest time on the
# path between them is at most t.
class TimedUnionFind:
    """Union by size, no path compression, so every
    link keeps the time it was made."""

    def __init__(self, n):
        # every person starts alone in a group of one
        self.parent = list(range(n))
        # sizes decide which group hangs under which
        self.size = [1] * n
        # time a node was linked under its parent
        self.link_time = [0] * n
        # how many separate groups exist right now
        self.groups = n

    def find(self, x):
        # walk up to the leader of x's group
        while self.parent[x] != x:
            x = self.parent[x]
        return x

    def union(self, a, b, t):
        # find the leaders of both groups
        ra, rb = self.find(a), self.find(b)
        # already in one group: nothing changes
        if ra == rb:
            return False
        # make ra the bigger group
        if self.size[ra] < self.size[rb]:
            ra, rb = rb, ra
        # hang the smaller group under the bigger one
        self.parent[rb] = ra
        self.size[ra] += self.size[rb]
        # remember when this link appeared
        self.link_time[rb] = t
        # two groups became one
        self.groups -= 1
        return True

    def connected_at(self, a, b, t):
        # a person is always connected to themselves
        if a == b:
            return True
        # climb from a, noting the newest link
        seen, newest, x = {a: 0}, 0, a
        while self.parent[x] != x:
            newest = max(newest, self.link_time[x])
            x = self.parent[x]
```

```
            seen[x] = newest
        # climb from b until we reach an ancestor of a
        newest, y = 0, b
        while y not in seen:
            # b's leader reached without meeting a
            if self.parent[y] == y:
                return False
            newest = max(newest, self.link_time[y])
            y = self.parent[y]
        # connected at t if all path links are <= t
        return max(newest, seen[y]) <= t


# BASE QUESTION (optimal): earliest time everyone is
# connected (Union-Find on sorted events)
# Idea: Sort by time and union the two people for each
# friend event. The first time the component count reaches
# 1 is the answer; -1 if it never does.
def earliest_all_connected(n, events):
    # zero or one person is connected from the start
    if n <= 1:
        return 0
    uf = TimedUnionFind(n)
    # handle friendships in time order
    for t, a, b, kind in by_time(events):
        # the base question has friend events only
        if kind == "friend":
            uf.union(a, b, t)
            # one group left: everyone is connected now
            if uf.groups == 1:
                return t
    # the groups never merged into one
    return -1
```

GROUP 10 OF 43 | 8 QUESTIONS, ONE CODE TEMPLATE
A dict of per-key state that you update, expire, or rank

81. Robot Status Message Deduplication Window [E]

```
# BASE QUESTION (optimal): Hash map of last displayed time
# Idea: If the message was displayed less than `window`
# seconds ago hide it without changing the stored time;
# otherwise display it and store the new time.
class Dedup:
    """Chronological stream: remember the last display."""

    def __init__(self, window=10, boundary_allowed=True):
        self.window, self.last = window, {}
        # True: exactly `window` seconds later may show
        self.boundary_allowed = boundary_allowed

    def too_soon(self, gap):
        # is this gap still inside the hiding window?
        if self.boundary_allowed:
            return gap < self.window
        return gap <= self.window

    def show(self, t, msg):
        # only a displayed event starts a new window
        last = self.last.get(msg)
        if last is not None and self.too_soon(t - last):
            return False
        self.last[msg] = t
        return True
```

82. Active Communication Top Users [M]

```
import heapq


from collections import Counter, defaultdict


# BASE QUESTION (optimal): Edge set, counts per user,
# count buckets for the maximum
# Idea: A new edge increments the count of both users.
# Buckets map each count to the set of users with it, so a
# user moves from bucket c to bucket c + 1.
# Answer: Pass a flag to the class: a self pair is one
# edge that adds one to that user's count (and only once).
# The dedup set already handles repeated self pairs.
class Active:
    def __init__(self, count_self=False):
        # distinct edges seen so far (for the dedupe rule)
        self.edges = set()
        # user -> how many different partners
        self.count = Counter()
        # count -> users having exactly that count
        self.bucket = defaultdict(set)
        # the highest count right now (it never drops)
        self.top = 0
        self.count_self = count_self

    def bump(self, user):
        c = self.count[user]
        # move the user from bucket c to bucket c + 1
        if c:
            self.bucket[c].discard(user)
        self.count[user] = c + 1
        self.bucket[c + 1].add(user)
        self.top = max(self.top, c + 1)

    def add(self, a, b):
        # A-B and B-A are the same edge
        pair = (min(a, b), max(a, b))
        # follow-ups: skip self pairs (unless counted)
```

```
        # and skip a repeated pair
        skip_self = a == b and not self.count_self
        if skip_self or pair in self.edges:
            return
        self.edges.add(pair)
        self.bump(a)
        # a self pair counts once, not twice
        if a != b:
            self.bump(b)

    def most_active(self):
        # O(1): read the top bucket
        if not self.top:
            return set()
        return set(self.bucket[self.top])

    def top_k(self, k):
        # k users by count, ties broken by name
        def rank(u):
            return (-self.count[u], u)
        return heapq.nsmallest(k, self.count, key=rank)
```

83. Filter Repeated Robot Status Messages [M]

```
# BASE QUESTION (optimal): Dictionary from message to the
# time it was last shown
# Idea: For each message in order: if it was never shown
# or now - last > 10, emit it and set last = now;
# otherwise drop it and leave last unchanged. One
# dictionary lookup per message.
def filter_status(stream, gap=10):
    # show a message only if MORE than `gap` seconds
    # passed
    # since it was last shown; hidden repeats change
    # nothing
    last, shown = {}, []
    for ts, msg in stream:
        if msg not in last or ts - last[msg] > gap:
            last[msg] = ts
            shown.append((ts, msg))
    return shown
```

84. Next-Word Predictor [M]

```
import heapq


from collections import Counter, defaultdict


# BASE QUESTION (optimal): Counts plus a maintained best
# successor
# Idea: counts[word][next] is incremented for every
# adjacent pair. After the increment compare that one
# count with the current best for the word (higher count
# wins, then the smaller word).
# Answer: Keep the Counter per word and use
# heapq.nsmallest on (-count, word): O(d log K) per query,
# or keep a sorted structure per word if queries dominate.
# Answer: The key becomes the pair (previous two words)
# and everything else is identical, at the price of more
# memory. Back off to the bigram table when the pair was
# never seen.
class Predictor:
    # FOLLOW-UP 2: orders=(2, 1) means two words of
    # context
    def __init__(self, orders=(2, 1)):
        # orders: how many previous words are the context
        self.orders = sorted(orders, reverse=True)
        # order -> context tuple -> Counter of next words
        self.counts = {k: defaultdict(Counter)
                       for k in orders}
        # order -> context tuple -> best next word so far
        self.best = {k: {} for k in orders}
```

```
    def train(self, sentence):
        words = sentence.split()
        for k in self.orders:
            for i in range(len(words) - k):
                ctx = tuple(words[i:i + k])
                nxt = words[i + k]
                self.update(k, ctx, nxt)

    def update(self, k, ctx, nxt):
        table = self.counts[k][ctx]
        table[nxt] += 1
        cur = self.best[k].get(ctx)
        # more counts win; a tie goes to the smaller word
        better = cur is None or table[nxt] > table[cur]
        tie = cur is not None and table[nxt] == table[cur]
        if better or (tie and nxt < cur):
            self.best[k][ctx] = nxt

    def predict(self, *context):
        # O(1): try the longest context, then back off
        for k in self.orders:
            if len(context) >= k:
                ctx = tuple(context[len(context) - k:])
                if ctx in self.best[k]:
                    return self.best[k][ctx]
        return None

    # FOLLOW-UP 1: the top K predictions
    def top_k(self, word, k):
        # k most likely next words after one word
        by_ctx = self.counts.get(1, {})
        table = by_ctx.get((word,), Counter())
        return heapq.nsmallest(
            k, table, key=lambda w: (-table[w], w))
```

85. Batch and Streaming Log Deduplication [M]

```
# BASE QUESTION (optimal): Dictionary on the message, then
# sort
# Idea: One pass with a dict from message to the retained
# timestamp (first seen: keep the first; latest: replace
# when the new timestamp is larger), then sort the
# retained entries for the output.
def dedupe(logs, policy="first"):
    # logs: (timestamp, message) pairs; keep one per
    # message
    kept = {}
    for ts, msg in logs:
        # "first" keeps the first seen, "latest" the
        # newest
        if msg not in kept or (
                policy == "latest" and ts > kept[msg]):
            kept[msg] = ts
    # retained entries sorted by time, then message
    return sorted((ts, msg) for msg, ts in kept.items())
```

86. Design a logger that returns top-K messages [M]

```
import heapq


from bisect import bisect_left, bisect_right


from collections import defaultdict


# BASE QUESTION (optimal): Per-message sorted timestamps
# plus a heap for the top K
# Idea: log appends the timestamp to the message's list
# (timestamps arrive in order, so lists stay sorted). A
# window count for one message is bisect_right(end) -
# bisect_left(start).
class Logger:
```

```
    def __init__(self):
        # message -> its timestamps, in order of arrival
        self.times = defaultdict(list)

    def log(self, ts, msg):
        self.times[msg].append(ts)

    def top_k(self, k, start=None, end=None):
        # the k most frequent messages in [start, end]
        counts = {}
        for msg, ts in self.times.items():
            lo, hi = 0, len(ts)
            if start is not None:
                lo = bisect_left(ts, start)
            if end is not None:
                hi = bisect_right(ts, end)
            if hi > lo:
                counts[msg] = hi - lo
        # ties are broken alphabetically
        def rank(m):
            return (-counts[m], m)
        return heapq.nsmallest(k, counts, key=rank)

    def expire(self, before):
        # bounded memory: forget timestamps older than
        # `before`
        for msg in list(self.times):
            ts = self.times[msg]
            self.times[msg] = ts[bisect_left(ts, before):]
            if not self.times[msg]:
                del self.times[msg]
```

87. Hit Counter [M]

```
from collections import deque


# HELPERS used by the code below
WINDOW = 300   # seconds


# BASE QUESTION (optimal): Queue of timestamps with
# eviction
# Idea: Keep a deque of timestamps in arrival order.
# getHits pops from the front while the oldest is outside
# the window, and the answer is the deque length. Each
# timestamp is pushed and popped once.
class QueueCounter:
    """One entry per hit; old entries are dropped when
    asked. Timestamps must not go backwards."""

    def __init__(self):
        self.hits = deque()

    def hit(self, ts):
        self.hits.append(ts)

    def get_hits(self, ts):
        # keep only the hits in (ts - 300, ts]
        while self.hits and self.hits[0] <= ts - WINDOW:
            self.hits.popleft()
        return len(self.hits)
```

88. Website Activity Analytics: Unique Users + Session Time [M]

```
from collections import defaultdict


# HELPERS for the base question
FORMAT = "%Y-%m-%d %H:%M:%S"


# BASE QUESTION (optimal): One pass to group, then
# sessions per user
# Idea: Parse each record once. unique[activity] collects
# user ids in a set.
def unique_users_by_activity(records):
    users = defaultdict(set)
    for r in records:
        # a set counts every user once per activity
        users[r["activity"]].add(r["user_id"])
    return {a: len(u) for a, u in sorted(users.items())}


# BASE QUESTION (optimal): One pass to group, then
# sessions per user
# Idea: Parse each record once. unique[activity] collects
# user ids in a set.
# Answer: Sort each user's events by time before pairing;
# a second login before a logout is ignored (or closes the
# old session, depending on the contract you agree).
def session_seconds(events):
    # events: one user's (time, activity), any order
    total, start = 0, None
    # FOLLOW-UP 1: sorted: rows may arrive out of order
    for t, act in sorted(events):
        if act == "login" and start is None:
            # a second login before a logout is ignored
            start = t
        elif act == "logout" and start is not None:
            total += (t - start).total_seconds()
            start = None
    # a login with no logout ends at the last event seen
    if start is not None:
        last = max(t for t, _ in events)
        total += (last - start).total_seconds()
    return total
```

GROUP 11 OF 43 | 8 QUESTIONS, ONE CODE TEMPLATE
Binary search on a yes/no condition

89. First Index Reaching a Threshold (Binary Search) [E]

```
# BASE QUESTION (optimal): Binary search for the lower
# bound
# Idea: Halve the range, keeping the boundary between
# values below k and values at least k.
def first_index_at_least(arr, k):
    lo, hi = 0, len(arr)
    while lo < hi:
        mid = (lo + hi) // 2
        if arr[mid] >= k:
            hi = mid
        else:
            lo = mid + 1
    return lo if lo < len(arr) else -1
```

90. Cutting Ribbons (LC 1891) [M]

```
# HELPERS for the base question
def pieces(ribbons, length):
    # how many pieces of this length can we cut?
    return sum(r // length for r in ribbons)


# Answer: No piece can be longer than the longest ribbon,
# and K pieces of length L need K * L total length, so L
# <= sum // K. Starting hi at min(max(ribbons), sum // K)
# saves a few probes and also returns 0 immediately when
# sum < K.
def upper_bound(ribbons, k):
    # no piece is longer than the longest ribbon, and
    # k pieces of length L need k * L total length
    return min(max(ribbons), sum(ribbons) // k)


# BASE QUESTION (optimal): Binary search the answer
# (integer lengths)
# Idea: If you can cut k pieces of length L you can cut k
# pieces of any shorter length, so feasibility is
# monotone.
def max_length(ribbons, k):
    # the best whole-number length (0 if impossible)
    lo, hi, best = 1, upper_bound(ribbons, k), 0
    while lo <= hi:
        mid = (lo + hi) // 2
        if pieces(ribbons, mid) >= k:
            # mid works: remember it and try longer
            best, lo = mid, mid + 1
        else:
            hi = mid - 1
    return best
```

91. First Bad Version with Parallel Search Follow-up [M]

```
# BASE QUESTION (optimal): problem: binary search
# Idea: isBad is monotone (all good, then all bad), so
# keep the invariant 'the first bad version is in [lo,
# hi]' and halve the interval. Use lo + (hi - lo) // 2 in
# languages with fixed-width integers.
def first_bad_version(n, is_bad):
    # the base question: binary search on versions 1..n
    lo, hi = 1, n
    while lo < hi:
        mid = lo + (hi - lo) // 2
        if is_bad(mid):
            # the first bad version is mid or earlier
            hi = mid
        else:
            lo = mid + 1
```

```
    return lo
```

92. Solve Banana Speed and Interval Merge [M]

```
# HELPERS for the base question
def hours_needed(piles, speed):
    # each hour she works on ONE pile, so every pile
    # rounds up
    return sum(-(-p // speed) for p in piles)


# BASE QUESTION (optimal): Bananas: binary search the
# speed
# Idea: Hours needed are non-increasing in the speed, so
# keep lo = 1 and hi = max(piles) and halve: if speed mid
# finishes within h, try slower, otherwise faster. Use
# ceiling division (an hour finishes at most one pile).
def min_eating_speed(piles, h):
    # fewer hours than piles can never work
    if h < len(piles):
        return -1
    lo, hi = 1, max(piles)
    # more speed never takes more hours: binary search
    while lo < hi:
        mid = (lo + hi) // 2
        if hours_needed(piles, mid) <= h:
            hi = mid
        else:
            lo = mid + 1
    return lo
```

93. Given a list with response time for each server, calculate how long time it will take to process n requests. [M]

```
# HELPERS for the base question
# Answer: Add the startup time to the check: server i
# completes max(0, (T - start_i) // r_i) requests. The
# check stays monotone in T, so binary search still
# applies.
def done_by(times, t, starts=None):
    # how many requests all servers finish within time t.
    # starts[i] is a startup delay of server i (default
    # 0).
    total = 0
    for i, r in enumerate(times):
        begin = starts[i] if starts else 0
        total += max(0, (t - begin) // r)
    return total


# BASE QUESTION (optimal): Binary search on the total time
# Idea: In time T server i completes T // r_i requests, so
# the servers finish at least n requests exactly when
# sum(T // r_i) >= n, which is monotone in T.
def time_for(times, n, starts=None):
    # smallest t with done_by(t) >= n. done_by never goes
    # down as t grows, so binary search works.
    if n <= 0:
        return 0
    lo = 0
    first = starts[0] if starts else 0
    hi = first + times[0] * n
    for i, r in enumerate(times):
        begin = starts[i] if starts else 0
        hi = min(hi, begin + r * n)
    while lo < hi:
        mid = (lo + hi) // 2
        if done_by(times, mid, starts) >= n:
            hi = mid
        else:
            lo = mid + 1
    return lo
```

94. Median of Two Sorted Arrays (LC 4) with a Follow-up Variant [H]

```
# HELPERS for the base question
INF = float("inf")


# BASE QUESTION (optimal): Binary search on the partition
# Idea: Make a the shorter array. Binary search i in a,
# set j = half - i in b so the left side holds (m + n + 1)
# // 2 elements.
def median_two(a, b):
    """Both arrays are sorted: binary search the cut. The
    left parts must hold half of all the elements."""
    if len(a) > len(b):
        a, b = b, a
    m, n = len(a), len(b)
    if m + n == 0:
        raise ValueError("no elements")
    lo, hi, half = 0, m, (m + n + 1) // 2
    while lo <= hi:
        i = (lo + hi) // 2
        j = half - i
        # values just left and right of each cut
        a_left = a[i - 1] if i > 0 else -INF
        a_right = a[i] if i < m else INF
        b_left = b[j - 1] if j > 0 else -INF
        b_right = b[j] if j < n else INF
        if a_left <= b_right and b_left <= a_right:
            # a valid cut: read the median off its edges
            if (m + n) % 2:
                return max(a_left, b_left)
            return (max(a_left, b_left)
                    + min(a_right, b_right)) / 2
        if a_left > b_right:
            hi = i - 1
        else:
            lo = i + 1
```

95. Find the median of two sorted arrays. [H]

```
# HELPERS for the base question
INF = float("inf")


# BASE QUESTION (optimal): Binary search on the partition
# Idea: Binary search how many elements of the shorter
# array go in the left half. A partition is right when
# every left element is <= every right element, which only
# needs the four border values to be checked.
# Answer: The same partition search: choose i elements
# from the shorter array and k - i from the other, and
# move i until the borders are in order. The median is
# just k = (m + n + 1) // 2.
def kth_of_two(a, b, k):
    # FOLLOW-UP 3: any k-th smallest, not only the median
    # k-th smallest (k starts at 1) of two sorted arrays.
    # Take i items from a and k - i from b so that the
    # borders are in order. Binary search on i: O(log n).
    if len(a) > len(b):
        a, b = b, a
    lo, hi = max(0, k - len(b)), min(k, len(a))
    while lo <= hi:
        i = (lo + hi) // 2
        j = k - i
        a_left = a[i - 1] if i > 0 else -INF
        a_right = a[i] if i < len(a) else INF
        b_left = b[j - 1] if j > 0 else -INF
        b_right = b[j] if j < len(b) else INF
        if a_left > b_right:
            hi = i - 1
        elif b_left > a_right:
            lo = i + 1
        else:
            return max(a_left, b_left)
```

```
    raise ValueError("k is out of range")


# BASE QUESTION (optimal): Binary search on the partition
# Idea: Binary search how many elements of the shorter
# array go in the left half. A partition is right when
# every left element is <= every right element, which only
# needs the four border values to be checked.
def median_of_two(a, b):
    # the median is the middle item, or the mean of the
    # two
    # middle items when the total is even
    total = len(a) + len(b)
    if total == 0:
        raise ValueError("no data")
    if total % 2:
        return kth_of_two(a, b, total // 2 + 1)
    low = kth_of_two(a, b, total // 2)
    high = kth_of_two(a, b, total // 2 + 1)
    return (low + high) / 2
```

96. Find longest duplicated substring [H]

```
# HELPERS for the base question
MOD = (1 << 61) - 1


BASE = 131


def find_length(s, length):
    # a repeated window of this length? (rolling hash)
    codes = [ord(c) for c in s]
    power = pow(BASE, length, MOD)
    h = 0
    for i in range(length):
        h = (h * BASE + codes[i]) % MOD
    seen = {h: [0]}
    for i in range(1, len(s) - length + 1):
        # slide the window: drop the left, add the right
        h = (h * BASE - codes[i - 1] * power
             + codes[i + length - 1]) % MOD
        # a hash match may be a collision: compare the
        # text
        for j in seen.get(h, ()):
            if s[j:j + length] == s[i:i + length]:
                return s[i:i + length]
        seen.setdefault(h, []).append(i)
    return None


# BASE QUESTION (optimal): Binary search on the length
# with a rolling hash (Rabin-Karp)
# Idea: For a fixed length L compute the polynomial hash
# of every window in O(1) each by sliding it, store hashes
# with their start positions, and when a hash repeats
# compare the actual substrings (to rule out a collision).
def longest_dup(s):
    # binary search the length: if length L repeats, so do
    # all shorter lengths
    lo, hi, best = 1, len(s) - 1, ""
    while lo <= hi:
        mid = (lo + hi) // 2
        found = find_length(s, mid)
        if found:
            best, lo = found, mid + 1
        else:
            hi = mid - 1
    return best
```

GROUP 12 OF 43 | 8 QUESTIONS, ONE CODE TEMPLATE
Greedy: scan once, carry the best choice so far, act only when forced

97. Maximum Profit [E]

```
# BASE QUESTION (optimal): One pass for the cheapest and
# the dearest city
# Idea: Find the city with the lowest price and the city
# with the highest price in one scan (ties by name). They
# are the best pair because profit is sell minus buy and
# the order of cities does not matter.
def max_profit_prices(prices):
    # a dictionary city -> price: buy at the cheapest city
    # and
    # sell at the dearest, order does not matter
    if len(prices) < 2:
        return 0
    values = list(prices.values())
    return max(values) - min(values)
```

98. Alarm for the Next Upcoming Birthday [E]

```
# HELPERS used by the code below
MONTH_DAYS = [
    31,
    28,
    31,
    30,
    31,
    30,
    31,
    31,
    30,
    31,
    30,
    31,
]


# BASE QUESTION (optimal): Day of the year and a cyclic
# minimum
# Idea: Days until the birthday is (doy - today) mod 365;
# take the smallest, ties by name.
def day_of_year(month, day):
    return sum(MONTH_DAYS[: month - 1]) + day


# BASE QUESTION (optimal): Day of the year and a cyclic
# minimum
# Idea: Days until the birthday is (doy - today) mod 365;
# take the smallest, ties by name.
def next_birthday(today, people):
    now = day_of_year(*today)
    return min(
        people,
        key=lambda p: (
            (day_of_year(p[1], p[2]) - now) % 365,
            p[0],
        ),
    )[0]
```

99. Coins on the Table [E]

```
# BASE QUESTION (optimal): One clamped counter
# Idea: total = max(0, total + delta) for every event.
def coins_on_table(events):
    total = 0
    for e in events:
        total = (
            total + 1 if e == "+" else max(0, total - 1)
        )
    return total
```

100. Visible People in a Line (Last Person, Then Everyone) [M]

```
# HELPERS for the base question
INF = float("inf")


# BASE QUESTION (optimal): One pass with a running maximum
# (last person)
# Idea: Walk from the person next to the last one towards
# the front, keeping the largest height seen strictly
# between the last person and the current one.
def last_sees(a):
    """Part 1: walk away from the last person keeping the
    tallest height seen strictly between us and them."""
    if not a:
        return 0
    last, between, count = a[-1], -INF, 0
    for k in range(len(a) - 2, -1, -1):
        # nobody between is taller than both of us
        if between <= max(last, a[k]):
            count += 1
        between = max(between, a[k])
    return count
```

101. Bounded Knapsack Variant [M]

```
# BASE QUESTION (optimal): Sort by value and take greedily
# Idea: Sort the item types by value, highest first. For
# each type take min(copies, slots left) and add value
# times that. Stop when the bag is full.
def max_value_sorted(items, W):
    """Unit weights: every slot is worth the same, so the
    best value always goes in first."""
    total = 0
    for value, count in sorted(items, reverse=True):
        # take as many copies as still fit in the bag
        take = min(count, W)
        total += value * take
        W -= take
        if W == 0:
            break
    return total
```

102. Find the Largest Monotone Increasing Number [M]

```
# BASE QUESTION (optimal): Right-to-left scan, then fill
# with nines
# Idea: Walk i from the last digit to the first. When
# digit[i - 1] > digit[i], decrease digit[i - 1] by one
# and remember i as the start of the nines.
def largest_monotone(n):
    # largest number <= n whose digits never decrease
    d = list(str(n))
    mark = len(d)
    # scan from the right; a drop forces the left digit
    # down by
    # one and everything after it up to 9
    for i in range(len(d) - 1, 0, -1):
        if d[i - 1] > d[i]:
            d[i - 1] = str(int(d[i - 1]) - 1)
            mark = i
    for i in range(mark, len(d)):
        d[i] = "9"
```

```
    return int("".join(d))
```

103. Min boards to cover roof holes [M]

```
# BASE QUESTION (optimal): Greedy: start a board at the
# leftmost uncovered hole
# Idea: Sort the holes. Place a board starting at the
# first uncovered hole, which covers every hole below
# start + L; skip those and repeat.
def min_boards(holes, length):
    # greedy: the leftmost uncovered hole starts a board,
    # which
    # then covers `length` positions
    count, covered_to = 0, float("-inf")
    for p in sorted(set(holes)):
        if p > covered_to:
            count += 1
            covered_to = p + length - 1
    return count
```

104. Jump Game II (Minimum Jumps) [M]

```
# BASE QUESTION (optimal): Greedy by ranges
# Idea: Each jump extends the covered range to the
# farthest index seen so far.
def jump(nums):
    jumps, current_end, farthest = 0, 0, 0
    for i in range(len(nums) - 1):
        farthest = max(farthest, i + nums[i])
        if i == current_end:
            # the range of the current jump is used up:
            # take another
            jumps += 1
            current_end = farthest
    return jumps
```

GROUP 13 OF 43 | 8 QUESTIONS, ONE CODE TEMPLATE
Sets and counters: remember the first collection, check the second against it

105. Exclude First-K-Unique From Another List [E]

```
# BASE QUESTION (optimal): A set of the first k distinct
# values
# Idea: Membership in a set makes the filter linear.
def exclude_first_k_unique(list1, list2, k):
    blocked = set()
    for x in list1:
        if len(blocked) == k:
            break
        blocked.add(x)
    return [y for y in list2 if y not in blocked]
```

106. Find the Duplicates [E]

```
from bisect import bisect_left


from math import log2


# CASE 1: M close to N (two pointers)
def common_two_pointers(a, b):
    # M close to N: walk both sorted arrays together and
    # advance the smaller side. O(N + M) time, O(1) space
    # besides the output.
    out, i, j = [], 0, 0
    while i < len(a) and j < len(b):
        if a[i] < b[j]:
            i += 1
        elif a[i] > b[j]:
            j += 1
        else:
            if not out or out[-1] != a[i]:
                out.append(a[i])
            i += 1
            j += 1
    return out


# CASE 2: M much bigger than N (binary search)
def common_binary_search(small, big):
    # M much bigger than N: binary search every item of
    # the
    # short array in the long one. O(N log M); the search
    # starts where the previous one stopped.
    out, lo = [], 0
    for x in small:
        lo = bisect_left(big, x, lo)
        if lo == len(big):
            break
        if big[lo] == x and (not out or out[-1] != x):
            out.append(x)
    return out


# BASE QUESTION (optimal): Case 1 (M close to N): two
# pointers
# Idea: Advance whichever pointer points at the smaller
# value; when equal, record the value once (skip repeats)
# and advance both. Each element is passed at most once.
def common(a, b):
    # choose by cost: N log M against N + M
    small, big = (a, b) if len(a) <= len(b) else (b, a)
    if len(small) * log2(len(big) + 2) < len(a) + len(b):
        return common_binary_search(small, big)
    return common_two_pointers(a, b)
```

107. Positional Matches Between Two Lists [E]

```
# BASE QUESTION (optimal): zip and a filter
# Idea: Walk both lists together and keep the agreeing
# values.
def positional_matches(first, second):
    return [a for a, b in zip(first, second) if a == b]
```

108. Contains Duplicate [E]

```
# BASE QUESTION (optimal): A set of seen values
# Idea: Add each value to a set; seeing one again means a
# duplicate.
def contains_duplicate(nums):
    seen = set()
    for x in nums:
        if x in seen:
            return True
        seen.add(x)
    return False
```

109. Filter Common Elements Per Window (Two Lists) [M]

```
# BASE QUESTION (optimal): A set per window
# Idea: Membership in the window of a is a set lookup.
def filter_windows(a, b, k):
    out = []
    for i in range(0, len(a), k):
        seen = set(a[i : i + k])
        out.extend(
            x for x in b[i : i + k] if x not in seen
        )
    return out
```

110. Reconstruct original array from doubled shuffle [M]

```
from collections import Counter


# BASE QUESTION (optimal): Counter processed in order of
# absolute value
# Idea: Count the values. For each value x in order of
# absolute value with a remaining count, it needs count[x]
# copies of 2x: if count[2x] is smaller, fail; otherwise
# consume them and append x that many times.
def find_original(changed):
    # `changed` holds every original x and its double 2x,
    # in
    # any order. Smallest absolute value first: it can
    # only be
    # an original, never someone's double.
    if len(changed) % 2:
        return []
    count, out = Counter(changed), []
    for x in sorted(count, key=lambda v: (abs(v), v)):
        if count[x] == 0:
            continue
        if x == 0:
            # zeros pair with each other
            if count[0] % 2:
                return []
            out += [0] * (count[0] // 2)
            count[0] = 0
            continue
        # every x needs its own 2x
        if count[2 * x] < count[x]:
            return []
        out += [x] * count[x]
        count[2 * x] -= count[x]
        count[x] = 0
    return out
```

111. Min deletions to avoid overlap in first k [M]

```
from collections import Counter


# BASE QUESTION (optimal): Sum of min(count in the first
# k, count in the rest) over shared values
# Idea: Count each value in the first k elements and in
# the rest. For every value with a positive count on both
# sides, one side has to lose all of its copies, so add
# the smaller count.
def min_deletions(a, k):
    # the values in a[:k] and a[k:] must not overlap; for
    # each
    # shared value delete the copies on the cheaper side
    left, right = Counter(a[:k]), Counter(a[k:])
    return sum(min(left[v], right[v])
               for v in left.keys() & right.keys())
```

112. Smallest Missing Positive Integer in Linear Time and Constant Extra Space [H]

```
# BASE QUESTION (optimal): Cyclic placement: put each
# value v at index v - 1
# Idea: For each position, while nums[i] is in 1..n and
# not already at its home index, swap it to index nums[i]
# - 1. Each swap puts one value in its final place, so the
# total number of swaps is at most n.
def first_missing_positive(nums):
    # put each value v in 1..n at index v - 1 by swapping
    a, n = list(nums), len(nums)
    for i in range(n):
        while 1 <= a[i] <= n and a[a[i] - 1] != a[i]:
            j = a[i] - 1
            a[i], a[j] = a[j], a[i]
    # the first index that does not hold index + 1
    for i in range(n):
        if a[i] != i + 1:
            return i + 1
    return n + 1
```

GROUP 14 OF 43 | 8 QUESTIONS, ONE CODE TEMPLATE
bisect on a sorted list: find a position, count a range, keep it sorted

113. Count People in a Postal Code Range [E]

```
from bisect import bisect_left, bisect_right


# BASE QUESTION (optimal): Sorted codes and two bisects
# Idea: The codes in the range form one block of the
# sorted list.
class PostalRegistry:
    def __init__(self, people):
        self.codes = sorted(code for _, code in people)

    def count_in_range(self, low, high):
        return max(
            0,
            bisect_right(self.codes, high)
            - bisect_left(self.codes, low),
        )
```

114. Sliding-Window Mean After Removing the Largest K Values [M]

```
from bisect import bisect_left, bisect_right


from fractions import Fraction


# HELPERS used by the code below
def as_text(fr):
    return f"{fr.numerator}/{fr.denominator}"


# HELPERS for the base question
# Answer: Maintain the sum of the K smallest alongside the
# K largest with the mirror-image boundary updates; the
# mean of the middle values is (total - top - bottom) / (w
# - 2K). Checked against sorting every window.
class Window:
    """A sorted window that keeps the sum of its k largest
    and k smallest values while values come and go."""

    def __init__(self, k):
        self.k, self.s = k, []
        self.total = self.top = self.bottom = 0

    def insert(self, x):
        k, s = self.k, self.s
        old_len = len(s)
        p = bisect_right(s, x)
        s.insert(p, x)
        self.total += x
        # top-k region is the last k places (new list)
        if old_len < k:
            self.top += x
        elif p >= old_len + 1 - k:
            # x entered it: the value at old_len - k left
            # it
            self.top += x - s[old_len - k]
        # bottom-k region is the first k places
        if old_len < k:
            self.bottom += x
        elif p < k:
            self.bottom += x - s[k]

    def remove(self, x):
        k, s = self.k, self.s
        old_len = len(s)
        p = bisect_left(s, x)
        self.total -= x
```

```
        if old_len <= k:
            self.top -= x
            self.bottom -= x
        else:
            if p >= old_len - k:
                # the value just below the region joins it
                self.top += s[old_len - k - 1] - x
            if p < k:
                self.bottom += s[k] - x
        del s[p]


# BASE QUESTION (optimal): Sorted window with an
# incrementally maintained top-K sum
# Idea: Keep the window values in a sorted list, the total
# window sum, and top = the sum of its K largest elements.
def trimmed_window_means(nums, window, k, both=False):
    # mean of each window after dropping the k largest
    # (and, with both=True, the k smallest as well)
    w, out = Window(k), []
    for i, x in enumerate(nums):
        w.insert(x)
        if i >= window:
            w.remove(nums[i - window])
        if i >= window - 1:
            kept = window - (2 * k if both else k)
            drop = w.top + (w.bottom if both else 0)
            mean = Fraction(w.total - drop, kept)
            out.append(as_text(mean))
    return out
```

115. Russian Doll Envelopes [M]

```
from bisect import bisect_left


# BASE QUESTION (optimal): Sort, then LIS with binary
# search
# Idea: Sort by (width ascending, height descending).
def max_envelopes(envelopes):
    # sort by width; for equal widths put TALLER first so
    # a
    # strictly increasing height run can use only one of
    # them
    ordered = sorted(envelopes,
                     key=lambda e: (e[0], -e[1]))
    # tails[k]: smallest ending height of a chain of k + 1
    tails = []
    for _, h in ordered:
        i = bisect_left(tails, h)
        if i == len(tails):
            tails.append(h)
        else:
            tails[i] = h
    return len(tails)
```

116. Count Sorted Strings With a Given Prefix [M]

```
from bisect import bisect_left


# BASE QUESTION (optimal): Two binary searches
# Idea: The matching words form one contiguous block of
# the sorted list; find both ends.
def count_with_prefix(words, prefix):
    lo = bisect_left(words, prefix)
    hi = bisect_left(words, prefix + "\uffff")
    return hi - lo
```

117. Streaming Insert + Find K-th Largest [M]

```
from bisect import bisect_left, insort


# BASE QUESTION (optimal): Sorted list with bisect (k
# chosen per query)
# Idea: Keep all values in an ascending list with insort.
# The k-th largest (0-indexed as in the examples) is a[-1
# - k].
# Answer: The heap can no longer answer because removing
# an arbitrary element is O(k).
# Answer: A heap cannot delete an arbitrary value
# efficiently.
class StreamKth:
    """Any k, with deletes: a sorted list."""

    def __init__(self):
        self.a = []

    def insert(self, x):
        # keep the list sorted: O(log n) search, O(n)
        # shift
        insort(self.a, x)

    # FOLLOW-UP 1, 3: values can also be deleted
    def delete(self, x):
        # remove one copy of x if present
        i = bisect_left(self.a, x)
        if i < len(self.a) and self.a[i] == x:
            del self.a[i]
            return True
        return False

    def find_largest(self, k):
        # k = 0 is the largest value
        if 0 <= k < len(self.a):
            return self.a[-1 - k]
        return None
```

118. Given positions of N kids and M ice cream sellers on a 1D line, find the smallest distance E such that every kid lies within the range [P - E, P + E] of at least one seller. [M]

```
from bisect import bisect_left


# BASE QUESTION (optimal): Sort the sellers, binary search
# each kid
# Idea: With the sellers sorted, bisect_left gives the
# first seller at or after the kid; the nearest seller is
# that one or the one before it. E is the largest of these
# nearest distances.
def smallest_e(kids, sellers):
    # each kid goes to the nearest seller; the answer is
    # the
    # worst such distance. The nearest seller is next to
    # kid's place in the sorted list of sellers.
    sellers = sorted(sellers)
    worst = 0
    for k in kids:
        i = bisect_left(sellers, k)
        near = INF = float("inf")
        if i < len(sellers):
            near = sellers[i] - k
        if i > 0:
            near = min(near, k - sellers[i - 1])
        worst = max(worst, near)
    return worst
```

119. Find Maximum Envelope Nesting [H]

```
from bisect import bisect_left


# BASE QUESTION (optimal): Sort by width ascending and
# height descending, then LIS by binary search
# Idea: Sorting equal widths by decreasing height means
# two envelopes of the same width can never both appear in
# a strictly increasing height sequence.
def max_envelopes(envelopes):
    # sort by width, taller first on ties, then find the
    # longest
    # strictly increasing run of heights
    ordered = sorted(envelopes,
                     key=lambda e: (e[0], -e[1]))
    tails = []
    for _, h in ordered:
        i = bisect_left(tails, h)
        if i == len(tails):
            tails.append(h)
        else:
            tails[i] = h
    return len(tails)
```

120. Count of Smaller Numbers After Self [H]

```
# HELPERS for the base question
class Fenwick:
    """Counts of seen values; prefix(i) = how many seen
    values have a rank below i."""

    def __init__(self, size):
        self.size = size
        self.tree = [0] * (size + 1)

    def add(self, i):
        i += 1
        while i <= self.size:
            self.tree[i] += 1
            i += i & -i

    def prefix(self, i):
        total = 0
        while i > 0:
            total += self.tree[i]
            i -= i & -i
        return total


def ranks(nums):
    # coordinate compression: value -> 0, 1, 2 ... by size
    return {v: i for i, v in enumerate(sorted(set(nums)))}


# BASE QUESTION (optimal): Fenwick tree over compressed
# values, scanning right to left
# Idea: Compress values to ranks 1..m. Scan from the
# right: the answer for x is the number of already
# inserted values with rank less than rank(x) (a prefix
# sum query), then insert rank(x). Both operations are
# O(log m).
def count_smaller_after(nums):
    # go from the right; before inserting x ask how many
    # already-seen values are strictly smaller than x
    rank = ranks(nums)
    tree = Fenwick(len(rank))
    out = []
    for x in reversed(nums):
        out.append(tree.prefix(rank[x]))
        tree.add(rank[x])
    return out[::-1]
```

GROUP 15 OF 43 | 8 QUESTIONS, ONE CODE TEMPLATE
Prefix sums, Tier 1: precompute once, answer ranges and balance points in O(1)

121. Parity Alternating Subarray Query [E]

```
# BASE QUESTION (optimal): Prefix count of equal-parity
# neighbour pairs
# Idea: bad[i] is the number of positions j < i with
# nums[j] and nums[j + 1] of the same parity. The pairs
# inside nums[l..r] are j = l .. r - 1, so the subarray
# alternates when bad[r] - bad[l] == 0.
class ParityQueries:
    """Static array. bad[i] = number of neighbouring pairs
    with equal parity among the first i pairs. A range
    alternates when it contains no such pair."""

    def __init__(self, nums):
        self.bad = [0]
        for a, b in zip(nums, nums[1:]):
            same = a % 2 == b % 2
            self.bad.append(self.bad[-1] + same)

    def alternates(self, l, r):
        # pairs l .. r - 1 are inside the range [l, r]
        return self.bad[r] - self.bad[l] == 0
```

122. Cake Balance Line [E]

```
# BASE QUESTION (optimal): Running left sum
# Idea: The right side is total - left - heights[i].
def cake_balance_index(heights):
    total, left = sum(heights), 0
    for i, h in enumerate(heights):
        if left == total - left - h:
            return i
        left += h
    return -1
```

123. Find the Pivot Index (Balanced Split) in O(n) [E]

```
# BASE QUESTION (optimal): Running left sum
# Idea: The right sum is total - left - nums[i], so one
# pass is enough.
def pivot_index(nums):
    total, left = sum(nums), 0
    for i, x in enumerate(nums):
        if left == total - left - x:
            return i
        left += x
    return -1
```

124. Range Duplicate Check with Constant-Time Queries [M]

```
# HELPERS for the base question
INF = float("inf")


# BASE QUESTION (optimal): Next-occurrence array with
# suffix minimum
# Idea: Scan right to left with a map from value to its
# latest index to fill next[i] (infinity if none). Then
# reach[i] = min(next[i], reach[i + 1]). Query(l, r) is
# reach[l] > r.
# Answer: Rebuild next and reach in O(n) after a batch of
# changes when reads dominate.
class RangeUnique:
    """O(n) build, then O(1) per query."""

    def __init__(self, a):
        self.a = list(a)
        self.rebuild()

    def rebuild(self):
```

```
        a, n = self.a, len(self.a)
        # nxt[i] = next index holding the same value
        nxt, last = [INF] * n, {}
        for i in range(n - 1, -1, -1):
            nxt[i] = last.get(a[i], INF)
            last[a[i]] = i
        # reach[i] = earliest repeat from index i onward
        self.reach = [INF] * (n + 1)
        for i in range(n - 1, -1, -1):
            self.reach[i] = min(nxt[i], self.reach[i + 1])

    def unique(self, l, r):
        # no repeat starts in [l, r] and ends inside it
        return self.reach[l] > r

    # FOLLOW-UP 1: the array changes between queries
    def update(self, i, value):
        # reads dominate, so a linear rebuild is fine
        self.a[i] = value
        self.rebuild()
```

125. Find a Horizontal Cut That Bisects Rectangle Area [M]

```
from collections import defaultdict


from fractions import Fraction


# HELPERS for the base question
def total_area(rects):
    # rects are disjoint (x1, y1, x2, y2)
    return sum((x2 - x1) * (y2 - y1)
               for x1, y1, x2, y2 in rects)


# BASE QUESTION (optimal): Sweep the y events and
# interpolate inside one slab
# Idea: Make events (y, +width) at each rectangle's bottom
# and (y, -width) at its top and sort them. Between
# consecutive event heights the slope is the running sum
# of widths.
# Answer: Replace 'half of the total' by p * total in the
# slab test; the interpolation formula inside the slab is
# the same. The code returns an exact Fraction and is
# checked by computing the area below the returned cut.
def cut_height(rects, share=Fraction(1, 2)):
    # FOLLOW-UP 4: share= leaves any fraction below the
    # cut
    # the height with `share` of the total area below it
    # (1/2 = bisect). Exact Fractions, never floats.
    want = share * total_area(rects)
    # width gained when the sweep passes each height
    events = defaultdict(int)
    for x1, y1, x2, y2 in rects:
        events[y1] += x2 - x1
        events[y2] -= x2 - x1
    ys = sorted(events)
    width, so_far = 0, Fraction(0)
    for y0, y1 in zip(ys, ys[1:]):
        # inside this slab the covered width is constant
        width += events[y0]
        gain = width * (y1 - y0)
        if width and so_far + gain >= want:
            # the cut is inside this slab: interpolate
            return y0 + (want - so_far) / width
        so_far += gain
    return Fraction(ys[-1])
```

126. Check pivot after removing one element [M]

```
# BASE QUESTION (optimal): Closed-form value to remove,
# found in a dictionary of positions
# Idea: Let P be the prefix sum before i and S the total.
def pivot_after_removal(a):
    # remove ONE element, then is there a pivot: an
    # element
    # whose left sum equals its right sum? Returns
    # (removed
    # index, pivot index) or None. O(n) with two scans.
    n, total = len(a), sum(a)
    prefix = [0]
    for x in a:
        prefix.append(prefix[-1] + x)
    # key[j]: left + left + a[j] when nothing is removed
    key = [2 * prefix[j] + a[j] for j in range(n)]
    # pivot j AFTER the removed index i:
    #   2 * prefix[j] - a[i] = total - a[j]  -> a[i] = key
    # - total
    where, ok = {}, None
    for j in range(n):
        i = where.get(key[j] - total)
        if i is not None:
            return i, j
        where[a[j]] = j
    # pivot j BEFORE the removed index i:
    #   2 * prefix[j] + a[j] = total - a[i]  -> a[i] =
    # total - key
    where = {}
    for j in range(n - 1, -1, -1):
        i = where.get(total - key[j])
        if i is not None:
            return i, j
        where[a[j]] = j
    return ok
```

127. Gold Chain Split into Two Equal Halves [M]

```
# HELPERS for the base question
def prefix_sums(a):
    # P[j] = sum of the first j segments
    P = [0]
    for x in a:
        P.append(P[-1] + x)
    return P


# BASE QUESTION (optimal): Prefix sums with first and last
# occurrence maps
# Idea: Let P[j] be the sum of the first j segments.
def gold_chain(a):
    # remove exactly one segment, then cut once into
    # two non-empty halves of equal weight
    n = len(a)
    if n < 3:
        return False
    P = prefix_sums(a)
    total = P[-1]
    # first and last position of every prefix sum (cuts
    # sit between segments, so positions 1..n-1)
    first, last = {}, {}
    for j in range(1, n):
        first.setdefault(P[j], j)
        last[P[j]] = j
    for r in range(n):
        left_over = total - a[r]
        if left_over % 2:
            continue
        half = left_over // 2
        # cut before the removed segment: P[j] == half
        if half in first and first[half] <= min(r, n - 2):
            return True
        # cut after it: P[j] - a[r] == half
```

```
        if half + a[r] in last and last[half + a[r]] > r:
            return True
    return False
```

128. Count the number of ways to split a string. [M]

```
from math import comb


# HELPERS for the base question
MOD = 10 ** 9 + 7


# BASE QUESTION (optimal): Count the ones, multiply the
# gap sizes
# Idea: Let ones be the total number of ones. If it is not
# divisible by 3 the answer is 0; if it is 0, any two of
# the n - 1 cut positions work, C(n - 1, 2).
# Answer: The same argument: each of the m - 1 cut points
# sits in the gap after a multiple of k ones, so multiply
# the gap sizes; with no ones at all it is C(n - 1, m -
# 1).
# Answer: Return the answer modulo 10^9 + 7 and apply the
# modulus after every multiplication; in languages with
# fixed-width integers use 64-bit intermediates. Cost:
# O(n) with modular arithmetic; no big integers are
# needed.
def ways_to_split(s, parts=3, mod=MOD):
    # a binary string cut into `parts` non-empty pieces
    # that
    # hold the same number of 1s (LeetCode 1573 for
    # parts=3)
    n = len(s)
    if n < parts:
        return 0
    # FOLLOW-UP 1: parts=m: m pieces with equal ones
    ones = [i for i, ch in enumerate(s) if ch == "1"]
    if len(ones) % parts:
        return 0
    if not ones:
        # no ones: any choice of parts - 1 cut points
        # works
        total = comb(n - 1, parts - 1)
        return total % mod if mod else total
    each = len(ones) // parts
    answer = 1
    for cut in range(1, parts):
        # the cut lies in the run of zeros between the
        # last
        # 1 of piece `cut` and the first 1 of the next
        # piece
        gap = ones[cut * each] - ones[cut * each - 1]
        answer *= gap
        # FOLLOW-UP 3: huge counts: answer modulo 10^9 + 7
        if mod:
            answer %= mod
    return answer
```

GROUP 16 OF 43 | 8 QUESTIONS, ONE CODE TEMPLATE
Sorted intervals: merge while they touch, then read the gaps or search

129. Check an Interval Against Existing Intervals for Conflicts [E]

```
from bisect import bisect_right


# BASE QUESTION (optimal): Sorted starts with bisect:
# check only two neighbours
# Idea: Keep the booked intervals sorted by start.
class Calendar:
    """A sorted calendar: O(log n) per check."""

    def __init__(self):
        self.starts, self.ends = [], []

    def book(self, start, end):
        i = bisect_right(self.starts, start)
        # the booking before must end by `start`
        if i > 0 and self.ends[i - 1] > start:
            return False
        # the booking after must start at or after `end`
        if i < len(self.starts) and self.starts[i] < end:
            return False
        self.starts.insert(i, start)
        self.ends.insert(i, end)
        return True
```

130. Find Containing Range [M]

```
from bisect import bisect_left, bisect_right


# BASE QUESTION (optimal): Merge once, then binary search
# the starts
# Idea: After sorting and merging, the starts are
# increasing, so bisect_right(starts, x) - 1 is the only
# candidate; it contains x if its end is at least x.
# Preprocessing is paid once.
# Answer: Sort by start and consider only ranges starting
# at or before x (one bisect); among those, keep the ones
# whose end is at least x. With many overlaps use an
# interval tree for O(log n + k) per query.
class RangeIndex:
    """Merged, sorted, disjoint closed ranges."""

    def __init__(self, ranges=()):
        self.starts, self.ends = [], []
        self.original = sorted(ranges)
        for s, e in self.original:
            self.insert(s, e)

    # FOLLOW-UP 2: ranges can be inserted over time
    def insert(self, s, e):
        # first range ending at or after s ...
        i = bisect_left(self.ends, s)
        # ... up to the last range starting at or before e
        j = bisect_right(self.starts, e)
        if i < j:
            s = min(s, self.starts[i])
            e = max(e, self.ends[j - 1])
        # replace everything the new range touches
        self.starts[i:j] = [s]
        self.ends[i:j] = [e]

    def find(self, x):
        # the merged range containing x, or None
        i = bisect_right(self.starts, x) - 1
        if i >= 0 and self.ends[i] >= x:
            return self.starts[i], self.ends[i]
        return None
```

```
    # FOLLOW-UP 1: every ORIGINAL range containing x
    def find_original(self, x):
        # every ORIGINAL range that contains x
        k = bisect_right([s for s, _ in self.original], x)
        return [r for r in self.original[:k] if r[1] >= x]

    # FOLLOW-UP 3: queries given in bulk
    def find_many(self, queries):
        # bulk: sort the queries and sweep once
        order = sorted(range(len(queries)),
                       key=lambda i: queries[i])
        out, j = [None] * len(queries), 0
        for i in order:
            x = queries[i]
            # skip ranges that end before x
            while j < len(self.ends) and self.ends[j] < x:
                j += 1
            inside = j < len(self.starts)
            if inside and self.starts[j] <= x:
                out[i] = (self.starts[j], self.ends[j])
        return out
```

131. Union of Sorted Interval Lists [M]

```
import heapq


# HELPERS for the base question
# Answer: Read each list through an iterator (a file or a
# paged API). union_k only ever holds one interval per
# list plus the current merged interval, so memory is O(K)
# and it can emit results as it goes.
def union_stream(lists, touching=True):
    # lists: sorted iterables of (start, end)
    merged = heapq.merge(*lists)
    cur = None
    for start, end in merged:
        if cur is None:
            cur = [start, end]
            continue
        # touching: [1, 3] and [3, 5] share the point 3
        joined = start < cur[1]
        if touching:
            joined = start <= cur[1]
        if joined:
            cur[1] = max(cur[1], end)
        else:
            # finished one block: emit it as we go
            yield tuple(cur)
            cur = [start, end]
    if cur is not None:
        yield tuple(cur)


# BASE QUESTION (optimal): Two-pointer merge
# Idea: Always take the interval with the smaller start
# next. If it touches or overlaps the last output interval
# (start <= last end, closed intervals) extend that one,
# otherwise append it.
def union_two(a, b, touching=True):
    return list(union_stream([a, b], touching))
```

132. Range Module (Insert and Point Query) [M]

```
from bisect import bisect_left, bisect_right


# BASE QUESTION (optimal): Merged intervals with bisect
# Idea: Disjoint sorted intervals mean one bisect locates
# the only candidate for a point.
class RangeModule:
    def __init__(self):
        self.lefts = []
        self.rights = []
```

```
    def add_range(self, left, right):
        i = bisect_left(self.rights, left)
        j = bisect_right(self.lefts, right)
        if i < j:
            left = min(left, self.lefts[i])
            right = max(right, self.rights[j - 1])
        self.lefts[i:j] = [left]
        self.rights[i:j] = [right]

    def query_point(self, x):
        i = bisect_right(self.lefts, x) - 1
        return i >= 0 and x < self.rights[i]
```

133. Meetings Minus a Do-Not-Schedule Block [M]

```
# BASE QUESTION (optimal): Merge, then cut
# Idea: After merging, every interval keeps at most a left
# and a right piece around the block.
def merge_intervals(meetings):
    merged = []
    for s, e in sorted(meetings):
        if merged and s <= merged[-1][1]:
            merged[-1] = (
                merged[-1][0],
                max(merged[-1][1], e),
            )
        else:
            merged.append((s, e))
    return merged


# BASE QUESTION (optimal): Merge, then cut
# Idea: After merging, every interval keeps at most a left
# and a right piece around the block.
def subtract_dns(meetings, dns):
    lo, hi = dns
    out = []
    for s, e in merge_intervals(meetings):
        if s < lo:
            out.append((s, min(e, lo)))
        if e > hi:
            out.append((max(s, hi), e))
    return out
```

134. Merge Intervals [M]

```
# BASE QUESTION (optimal): Sort by start and sweep
# Idea: Sort by start. If an interval starts at or before
# the end of the last output interval, extend that end
# with max; otherwise append it. The sort guarantees
# nothing earlier can overlap the new interval.
def merge_intervals(intervals, touching_ints=False):
    out = []
    # sort by start, then extend the last interval while
    # they overlap
    for s, e in sorted(intervals):
        gap = 1 if touching_ints else 0
        if out and s <= out[-1][1] + gap:
            out[-1][1] = max(out[-1][1], e)
        else:
            out.append([s, e])
    return out
```

135. Find safe travel intervals between planet influences [H]

```
# HELPERS used by the code below
INF = float("inf")


# HELPERS for the base question
def unsafe_blocks(planets):
    # planet (c, r) makes [c - r, c + r] unsafe; merge
    # overlaps
    merged = []
    spans = sorted((c - r, c + r) for c, r in planets)
    for a, b in spans:
        # sharing even one point joins two blocks
        if merged and a <= merged[-1][1]:
            merged[-1][1] = max(merged[-1][1], b)
        else:
            merged.append([a, b])
    return merged


# BASE QUESTION (optimal): Merge the unsafe intervals,
# then read off the gaps
# Idea: Sort the unsafe intervals by start and merge any
# that overlap or touch (they share a point, which is
# unsafe).
def safe_intervals(planets):
    # the gaps between unsafe blocks (open intervals)
    merged = unsafe_blocks(planets)
    if not merged:
        return [(-INF, INF)]
    out = [(-INF, merged[0][0])]
    for prev, nxt in zip(merged, merged[1:]):
        out.append((prev[1], nxt[0]))
    out.append((merged[-1][1], INF))
    return out
```

136. Implement a function that processes a stream of interval data. [H]

```
from bisect import bisect_left, bisect_right


# BASE QUESTION (optimal): Sorted disjoint intervals with
# binary search
# Idea: Keep the merged intervals sorted by start. A new
# interval only interacts with the intervals found by two
# binary searches; replace that slice by the single merged
# interval. Touching closed intervals merge.
# Answer: The sorted disjoint intervals already support
# both: a point is covered if the interval with the
# largest start at or before it ends at or after it (one
# bisect), and the total length is the sum of the interval
# lengths (or keep it updated on each add).
class IntervalSet:
    """Keeps closed intervals merged and sorted.
    integer=True: [1, 3] and [4, 5] cover 1..5 and merge.
    integer=False: only overlap or a shared end merges."""

    def __init__(self, integer=True):
        self.starts, self.ends = [], []
        self.gap = 1 if integer else 0
        # length of [a, b] is b - a + extra
        self.extra = 1 if integer else 0
        self.covered = 0

    def length(self, start, end):
        return end - start + self.extra

    def add(self, start, end):
        if end < start:
            raise ValueError("bad interval")
        # first interval that touches the new one, and the
        # first one that is completely to its right
        i = bisect_left(self.ends, start - self.gap)
```

```
        j = bisect_right(self.starts, end + self.gap)
        for k in range(i, j):
            # the swallowed intervals leave the total ...
            self.covered -= self.length(self.starts[k],
                                        self.ends[k])
        if i < j:
            start = min(start, self.starts[i])
            end = max(end, self.ends[j - 1])
        # ... and the merged one joins it
        self.starts[i:j] = [start]
        self.ends[i:j] = [end]
        self.covered += self.length(start, end)

    # FOLLOW-UP 3: is this point covered?
    def contains(self, point):
        # the interval with the largest start <= point
        i = bisect_right(self.starts, point) - 1
        return i >= 0 and self.ends[i] >= point

    # FOLLOW-UP 3: how much is covered in total?
    def total(self):
        return self.covered

    def intervals(self):
        return list(zip(self.starts, self.ends))
```

GROUP 17 OF 43 | 8 QUESTIONS, ONE CODE TEMPLATE
Trie: nested dicts, one character per level

137. URL Request Handler Router [M]

```
# HELPERS used by the code below
def segments(path):
    # "/users/7/photo" -> ["users", "7", "photo"]
    return [s for s in path.split("/") if s]


# HELPERS for the base question
class Node:
    def __init__(self):
        # fixed segment -> child node
        self.literal = {}
        # child for a {placeholder} segment
        self.wild = None
        self.wild_name = None
        # handler when the path ends here
        self.handler = None
        # handler for a trailing "*" (rest of the path)
        self.rest = None


# BASE QUESTION (optimal): Segment trie with literal-first
# fallback
# Idea: Each trie node has literal children and one
# wildcard child; the pattern is stored at its last node.
# Lookup tries the literal child first, and if that whole
# branch fails it backs up and tries the wildcard child.
# Answer: Make the method the first segment of the path (a
# tiny trick) or keep a dict method to trie. Lookup uses
# the method as the first step.
class Router:
    def __init__(self):
        self.root = Node()

    def add(self, pattern, handler=None, method="GET"):
        node = self.root
        # FOLLOW-UP 1: the HTTP method is the first
        # segment
        # the method is simply the first segment
        for seg in [method] + segments(pattern):
            # FOLLOW-UP 2: a trailing * matches the rest
            # of the path
            if seg == "*":
                # all deeper paths use this handler
                node.rest = handler or pattern
                return
            if seg.startswith("{") and seg.endswith("}"):
                node.wild = node.wild or Node()
                node.wild_name = seg[1:-1]
                node = node.wild
            else:
                lit = node.literal
                node = lit.setdefault(seg, Node())
        node.handler = handler or pattern

    def match(self, url, method="GET"):
        segs = [method] + segments(url)
        params = {}

        def walk(node, i):
            if i == len(segs):
                return node.handler
            # 1) exact segment first
            child = node.literal.get(segs[i])
            if child is not None:
                found = walk(child, i + 1)
                if found is not None:
```

```
                return found
            # 2) then a placeholder, remembering its value
            if node.wild is not None:
                # FOLLOW-UP 3: remember the value (id = 7)
                params[node.wild_name] = segs[i]
                found = walk(node.wild, i + 1)
                if found is not None:
                    return found
                # undo the value when backtracking
                params.pop(node.wild_name, None)
            # 3) finally a trailing wildcard
            return node.rest

        handler = walk(self.root, 0)
        return handler, dict(params)
```

138. Trie Prefix Search / Autocomplete [M]

```
# HELPERS for the base question
class Node:
    def __init__(self):
        self.children = {}
        # how often the word ending here was inserted
        self.freq = 0
        # best (negative freq, word) pairs under this node
        self.top = []


# BASE QUESTION (optimal): Trie with a DFS over sorted
# children
# Idea: Insert each word, marking the last node. For a
# query walk the prefix; if a step is missing return an
# empty list.
class Trie:
    def __init__(self, k=3):
        self.root, self.k = Node(), k

    def insert(self, word, freq=1):
        node = self.root
        path = [node]
        for ch in word:
            node = node.children.setdefault(ch, Node())
            path.append(node)
        node.freq += freq
        # refresh the top-k list on every node of the path
        for n in path:
            n.top = [x for x in n.top if x[1] != word]
            n.top.append((-node.freq, word))
            n.top.sort()
            del n.top[self.k:]

    def find(self, prefix):
        # walk down the prefix; None if it is missing
        node = self.root
        for ch in prefix:
            node = node.children.get(ch)
            if node is None:
                return None
        return node

    def search_prefix(self, prefix):
        # every word with this prefix, in alphabetical
        # order
        node = self.find(prefix)
        out = []
        if node is None:
            return out
        stack = [(node, prefix)]
        while stack:
            n, text = stack.pop()
            if n.freq:
                out.append(text)
            for ch in n.children:
```

```
                stack.append((n.children[ch], text + ch))
        return sorted(out)

    def suggest(self, prefix):
        # best k completions, read from one node: O(L + k)
        node = self.find(prefix)
        return [w for _, w in node.top] if node else []

    def fuzzy(self, prefix):
        # words whose first letters differ in at most one
        # place
        out, stack = [], [(self.root, "", 0)]
        while stack:
            n, text, wrong = stack.pop()
            if len(text) == len(prefix):
                out += self.search_below(n, text)
                continue
            for ch, child in n.children.items():
                miss = wrong + (ch != prefix[len(text)])
                if miss <= 1:
                    stack.append((child, text + ch, miss))
        return sorted(out)

    def search_below(self, node, text):
        out, stack = [], [(node, text)]
        while stack:
            n, t = stack.pop()
            if n.freq:
                out.append(t)
            for ch, child in n.children.items():
                stack.append((child, t + ch))
        return out
```

139. Build Prefix Lookup with a Trie [M]

```
# HELPERS for the base question
class Node:
    def __init__(self):
        self.children = {}
        # does a word end here?
        self.is_word = False
        # how many words pass through this node
        self.count = 0


# BASE QUESTION (optimal): Trie with a DFS below the
# prefix node
# Idea: Insert every word into nested dicts, marking word
# ends. To answer a prefix, walk down one character at a
# time (no match means no completions), then DFS the
# subtree collecting words.
# Answer: Store a counter in each node that is incremented
# when a word passes through it; the answer is the counter
# at the end of the prefix walk, O(L) with no DFS.
class PrefixTrie:
    def __init__(self, words=()):
        self.root = Node()
        for w in words:
            self.insert(w)

    def insert(self, word):
        node = self.root
        for ch in word:
            node = node.children.setdefault(ch, Node())
            node.count += 1
        # a repeated word is only counted as one word
        if not node.is_word:
            node.is_word = True
        else:
            self.undo_count(word)

    def undo_count(self, word):
        node = self.root
```

```
        for ch in word:
            node = node.children[ch]
            node.count -= 1

    def walk(self, prefix):
        node = self.root
        for ch in prefix:
            node = node.children.get(ch)
            if node is None:
                return None
        return node

    # FOLLOW-UP 1: how many words start with the prefix
    def count_prefix(self, prefix):
        # how many words start with the prefix: O(len)
        node = self.walk(prefix)
        if prefix == "":
            return self.root_count()
        return node.count if node else 0

    def root_count(self):
        kids = self.root.children.values()
        return sum(c.count for c in kids)

    # FOLLOW-UP 2: only the top k completions
    def suggest(self, prefix, k=3):
        # first k completions in alphabetical order; the
        # DFS
        # visits children in order and stops after k words
        node = self.walk(prefix)
        out = []
        if node is None:
            return out
        stack = [(node, prefix)]
        while stack and len(out) < k:
            n, text = stack.pop()
            if n.is_word:
                out.append(text)
            # reversed so the smallest letter is popped
            # first
            for ch in sorted(n.children, reverse=True):
                stack.append((n.children[ch], text + ch))
        return out

    def completions(self, prefix):
        # every word with the prefix
        return self.suggest(prefix, k=10 ** 9)
```

140. Greedy Longest-Match Dictionary Tokenizer [M]

```
# BASE QUESTION (optimal): Trie walk keeping the deepest
# (or best-priority) terminal
# Idea: Insert every key into a trie with its id at the
# last node. From each text position walk down while the
# next character exists, remembering the last terminal
# passed.
# Answer: A flat map of every key wastes memory because
# keys share prefixes. A compressed (radix) trie stores
# each shared stretch once, or shard the keys by first
# characters across machines.
class Tokenizer:
    def __init__(self, table, priority=None):
        # table: key -> id. priority(key): lower wins;
        # with
        # None the longest match wins
        self.priority = priority
        self.root = {}
        for key, ident in table.items():
            node = self.root
            for ch in key:
                node = node.setdefault(ch, {})
            # the end of a key stores the key and its id
            node["$"] = (key, ident)
```

```
    def better(self, new, old):
        # should the match `new` replace the match `old`?
        if old is None:
            return True
        if self.priority is None:
            # deeper (longer) matches come later: keep
            # them
            return True
        rank = self.priority
        return rank(new[0]) < rank(old[0])

    def longest_at(self, text, i):
        # walk the trie from position i
        node, best, j = self.root, None, i
        while j < len(text) and text[j] in node:
            node = node[text[j]]
            j += 1
            end = node.get("$")
            if end and self.better(end, best):
                best = end
        return best

    def tokenize(self, text):
        out, i = [], 0
        while i < len(text):
            match = self.longest_at(text, i)
            if match is None:
                # no key starts here: keep the character
                out.append(text[i])
                i += 1
            else:
                key, ident = match
                out.append(ident)
                i += len(key)
        return out
```

141. Check if all substrings are dictionary words [M]

```
# HELPERS for the base question
# Answer: Same loops, but return the substring instead of
# False, which is what a user wants to see. Check the
# shorter ones first so the report is the smallest
# counterexample.
def first_failing(s, words, min_len=3):
    # the SHORTEST substring (length >= min_len) that is
    # not a
    # word, or None. Short lengths first gives the
    # smallest
    # counterexample, and cheap checks run before
    # expensive ones.
    words = set(words)
    longest = max(map(len, words), default=0)
    for length in range(min_len, len(s) + 1):
        # longer than every word: the first window already
        # fails
        if length > longest:
            return s[:length]
        for i in range(len(s) - length + 1):
            piece = s[i:i + length]
            if piece not in words:
                return piece
    return None


# BASE QUESTION (optimal): Walk a trie from each start
# position
# Idea: For each start i descend the trie along s[i:]. At
# depth 1 and 2 only walk; from depth 3 on the current
# node must be marked as a word end.
def all_substrings_words(s, words, min_len=3):
    return first_failing(s, words, min_len) is None
```

142. Return dictionary words matching a prefix [M]

```
from bisect import bisect_left, bisect_right, insort


# BASE QUESTION (optimal): Sort once, binary search the
# range
# Idea: With the words sorted, the matches are the words
# from bisect_left(prefix) until the first word that no
# longer starts with the prefix.
# Answer: Stop the forward walk after K words, as the
# limit parameter does; with the sorted list the first K
# in alphabetical order are already the first K in the
# block.
# Answer: Keep the list sorted with bisect.insort (O(n)
# per insert, fast in practice) or use a balanced tree; a
# trie handles inserts in O(L).
class PrefixIndex:
    def __init__(self, words):
        # a sorted copy: all words with a prefix sit
        # together
        self.words = sorted(words)

    # FOLLOW-UP 1: limit= returns only the first K
    def matches(self, prefix, limit=None):
        # first possible position, then walk while it
        # matches
        lo = bisect_left(self.words, prefix)
        # one past the last word that starts with the
        # prefix
        hi = bisect_right(self.words, prefix + "■")
        found = self.words[lo:hi]
        return found if limit is None else found[:limit]

    # FOLLOW-UP 2: words added while queries run
    def add(self, word):
        # words can arrive while queries run: keep it
        # sorted
        insort(self.words, word)
```

143. Implement Trie Insert and Exact-Word Search [M]

```
# HELPERS for the base question
class Node:
    def __init__(self):
        self.children = {}
        self.end = False


# BASE QUESTION (optimal): Trie of nodes with an end-of-
# word flag
# Idea: Each node has children by character and a flag.
# insert creates missing nodes and marks the last one.
# search walks the word and requires the flag at the end;
# startsWith only requires that the walk succeeds.
# Answer: On '.', try every child at that level and
# recurse; other characters follow the single matching
# child. Worst case O(26^L) per query on adversarial
# patterns, so mention the bound.
class Trie:
    def __init__(self):
        self.root = Node()

    def walk(self, text):
        # follow the letters; None if the path breaks
        node = self.root
        for ch in text:
            node = node.children.get(ch)
            if node is None:
                return None
        return node

    def insert(self, word):
        node = self.root
```

```
        for ch in word:
            node = node.children.setdefault(ch, Node())
        node.end = True

    def search(self, word):
        # the whole word must have been inserted
        node = self.walk(word)
        return node is not None and node.end

    def starts_with(self, prefix):
        return self.walk(prefix) is not None

    # FOLLOW-UP 1: '.' matches any one letter
    def search_wild(self, pattern):
        # '.' matches any one letter (LeetCode 211)
        def go(node, i):
            if i == len(pattern):
                return node.end
            ch = pattern[i]
            if ch == ".":
                # try every child at this level
                kids = node.children.values()
                return any(go(c, i + 1) for c in kids)
            child = node.children.get(ch)
            return child is not None and go(child, i + 1)

        return go(self.root, 0)

    # FOLLOW-UP 2: delete a word
    def delete(self, word):
        # unmark the word, then prune nodes nothing needs
        path = [self.root]
        for ch in word:
            nxt = path[-1].children.get(ch)
            if nxt is None:
                return False
            path.append(nxt)
        if not path[-1].end:
            return False
        path[-1].end = False
        # walk back up removing empty, non-word nodes
        for i in range(len(word), 0, -1):
            node = path[i]
            if node.end or node.children:
                break
            del path[i - 1].children[word[i - 1]]
        return True
```

144. Implement Longest-Match Text Replacement [H]

```
# BASE QUESTION (optimal): Trie walk that remembers the
# last token end
# Idea: Build a trie of the tokens (each token's last node
# stores its id). At position i walk down the trie along
# the text; whenever the current node stores an id, record
# it and its length.
# Answer: Collect every token that matches at the position
# (walk the trie and gather all token ends along the path,
# not just the last) and choose with the comparator. The
# trie gives the candidates; the policy is separate.
class Replacer:
    def __init__(self, mapping):
        # mapping: token -> id. A trie shares the
        # prefixes.
        self.root = {}
        for token, ident in mapping.items():
            node = self.root
            for ch in token:
                node = node.setdefault(ch, {})
            node["$"] = (token, ident)

    def candidates(self, text, i):
        # every token that matches at position i, shortest
```

```
        # first (all ends along the trie path, not just
        # the last)
        out, node, j = [], self.root, i
        while j < len(text) and text[j] in node:
            node = node[text[j]]
            j += 1
            if "$" in node:
                out.append(node["$"])
        return out

    # FOLLOW-UP 1: choose= picks by priority, not length
    def replace(self, text, choose=None,
                word_boundary=False):
        # choose(candidates) picks the winner; the default
        # is the
        # longest token
        choose = choose or (lambda cs: cs[-1])
        out, i = [], 0
        while i < len(text):
            found = self.candidates(text, i)
            # FOLLOW-UP 3: only whole-word matches
            if word_boundary:
                found = [c for c in found if
                         self.edge(text, i, len(c[0]))]
            if found:
                token, ident = choose(found)
                out.append(ident)
                i += len(token)
            else:
                # nothing matches here: keep the character
                out.append(text[i])
                i += 1
        return out

    @staticmethod
    def edge(text, i, size):
        # letters must not touch the match on either side
        before = i == 0 or not text[i - 1].isalnum()
        end = i + size
        after = end == len(text)
        after = after or not text[end].isalnum()
        return before and after
```

GROUP 18 OF 43 | 8 QUESTIONS, ONE CODE TEMPLATE
A dict for lookup plus an ordered container for who is next (lazy deletion)

145. Implement Batched Undo/Redo Layer [E]

```
from collections import deque


# HELPERS for the base question
class Change:
    """One key in one batch: value before and after."""

    def __init__(self, key, old, has_old, new):
        self.key, self.old, self.has_old, self.new = (
            key, old, has_old, new)


# BASE QUESTION (optimal): Store only the changed keys per
# batch
# Idea: While a batch is open keep {key: (old, new)} with
# the old value recorded only the first time a key is
# touched. endBatch pushes it on the undo stack and clears
# the redo stack.
# Answer: Keep the undo stack in a deque with maxlen N so
# the oldest batch is dropped automatically. Redo is
# unaffected.
# Answer: Keep a stack of open batches and merge a child
# into its parent on endBatch (keep the parent's old value
# if the key was already touched). Only the outermost
# endBatch commits an undoable action.
class Layer:
    def __init__(self, history=None):
        self.props = {}
        # FOLLOW-UP 1, 4: history=N keeps the last N
        # batches
        # committed batches; a deque with a maximum length
        # forgets the oldest ones when the history is
        # capped
        self.undo_stack = deque(maxlen=history)
        self.redo_stack = []
        # open batches (a stack, so batches can nest):
        # each is
        # a dict key -> Change, so a key appears only once
        self.open = []

    def begin_batch(self):
        self.open.append({})

    def set_property(self, key, value):
        if not self.open:
            # outside a batch: a batch of one change
            self.begin_batch()
            self.set_property(key, value)
            self.end_batch()
            return
        batch = self.open[-1]
        if key in batch:
            # touched again: keep the ORIGINAL old value
            # and
            # only update the final value
            batch[key].new = value
        else:
            batch[key] = Change(key, self.props.get(key),
                                key in self.props, value)
        self.props[key] = value

    def end_batch(self):
        batch = self.open.pop()
        if self.open:
            # FOLLOW-UP 2: nested batches fold into the
            # parent
```

```
            # a nested batch folds into its parent
            self.merge(self.open[-1], batch)
        elif batch:
            # only the outermost batch is one undoable
            # action;
            # a new action clears the redo history
            self.undo_stack.append(list(batch.values()))
            self.redo_stack.clear()

    @staticmethod
    def merge(parent, child):
        for key, change in child.items():
            if key in parent:
                parent[key].new = change.new
            else:
                parent[key] = change

    def undo(self):
        if not self.undo_stack:
            return False
        changes = self.undo_stack.pop()
        for c in changes:
            if c.has_old:
                self.props[c.key] = c.old
            else:
                # the key did not exist before: remove it
                self.props.pop(c.key, None)
        self.redo_stack.append(changes)
        return True

    def redo(self):
        if not self.redo_stack:
            return False
        changes = self.redo_stack.pop()
        for c in changes:
            self.props[c.key] = c.new
        self.undo_stack.append(changes)
        return True
```

146. Design a restaurant waitlist system [M]

```
# HELPERS for the base question
INF = float("inf")


# BASE QUESTION (optimal): Segment tree of minimum size
# over arrival order
# Idea: Each arrival gets the next position. The tree
# stores, for each range, the smallest waiting party size
# (infinity for empty or seated slots).
class Waitlist:
    """Earliest party that fits: a segment tree of the
    smallest size over arrival slots."""

    def __init__(self, capacity=1024):
        self.cap = capacity
        self.tree = [INF] * (2 * capacity)
        self.count = 0
        self.slot, self.who = {}, {}

    def update(self, i, value):
        i += self.cap
        self.tree[i] = value
        while i > 1:
            i //= 2
            self.tree[i] = min(self.tree[2 * i],
                               self.tree[2 * i + 1])

    def add(self, party_id, name, size):
        self.slot[party_id] = self.count
        self.who[self.count] = party_id
        self.update(self.count, size)
        self.count += 1
```

```
    def cancel(self, party_id):
        i = self.slot.get(party_id)
        if i is None or self.tree[i + self.cap] == INF:
            return False
        self.update(i, INF)
        return True

    def seat(self, capacity):
        # no party small enough: nobody is seated
        if self.tree[1] > capacity:
            return None
        i = 1
        # go left whenever the left half holds a party
        # that fits
        while i < self.cap:
            left = self.tree[2 * i] <= capacity
            i = 2 * i if left else 2 * i + 1
        self.update(i - self.cap, INF)
        return self.who[i - self.cap]
```

147. Implement Python's itertools.tee [M]

```
from collections import deque


# BASE QUESTION (optimal): One deque per consumer, filled
# lazily
# Idea: When a consumer's deque is empty it pulls the next
# item from the source and appends it to every consumer's
# deque, then pops its own.
def tee_queues(iterable, n=2):
    source = iter(iterable)
    # one waiting line of values per consumer
    queues = [deque() for _ in range(n)]

    def consumer(mine):
        while True:
            if not mine:
                try:
                    value = next(source)
                except StopIteration:
                    return
                # give the new value to everybody
                for q in queues:
                    q.append(value)
            yield mine.popleft()

    return tuple(consumer(q) for q in queues)
```

148. Store With Cheapest-Item Removal [M]

```
import heapq


# BASE QUESTION (optimal): A heap keyed by price and
# timestamp
# Idea: The smallest (price, timestamp) is always at the
# top of the heap.
class Store:
    def __init__(self):
        self.heap = []

    def add(self, value, price, timestamp):
        heapq.heappush(
            self.heap, (price, timestamp, value)
        )

    def remove_cheapest(self):
        if not self.heap:
            return None
        return heapq.heappop(self.heap)[2]
```

149. Round-Robin Job Scheduler [M]

```
from collections import deque


# BASE QUESTION (optimal): A queue of jobs
# Idea: Run the front job for one slice and send it to the
# back if it is not finished.
def round_robin(jobs, quantum):
    queue = deque([jid, burst] for jid, burst in jobs)
    clock, done = 0, []
    while queue:
        job = queue.popleft()
        run = min(quantum, job[1])
        clock += run
        job[1] -= run
        if job[1] > 0:
            # not finished: back of the line
            queue.append(job)
        else:
            done.append((job[0], clock))
    return done
```

150. Aggregate FizzBuzz Offer Matches [M]

```
from collections import deque


# BASE QUESTION (optimal): Two FIFO queues and an ordered
# total
# Idea: Match against the opposite queue's front until the
# offer or the queue is empty.
def fizzbuzz_matches(offers):
    queues = {"Fizz": deque(), "Buzz": deque()}
    totals = {}
    for offer in offers:
        mine = queues[offer["type"]]
        other = queues[
            (
                "Buzz"
                if offer["type"] == "Fizz"
                else "Fizz"
            )
        ]
        left = offer["quantity"]
        while left and other:
            front = other[0]
            m = min(left, front[1])
            fizz_name = (
                offer["name"]
                if offer["type"] == "Fizz"
                else front[0]
            )
            buzz_name = (
                front[0]
                if offer["type"] == "Fizz"
                else offer["name"]
            )
            totals[(fizz_name, buzz_name)] = (
                totals.get((fizz_name, buzz_name), 0)
                + m
            )
            left -= m
            front[1] -= m
            if front[1] == 0:
                other.popleft()
        if left:
            mine.append([offer["name"], left])
    return [
        f"FizzBuzz! {q} {f}:{b}"
        for (f, b), q in totals.items()
    ]
```

151. Restaurant Waitlist: join / delete / find_first_match [M]

```
from collections import defaultdict, deque


# BASE QUESTION (optimal): Queue per party size with lazy
# deletion
# Idea: join stores (sequence number, name) in the queue
# for its size and in a map from name to its record.
# delete just removes the record from the map.
class Waitlist:
    def __init__(self):
        # arrival counter: lower means joined earlier
        self.seq = 0
        # person -> their arrival number (people still
        # waiting)
        self.live = {}
        # party size -> line of (arrival number, person)
        self.queues = defaultdict(deque)

    def join(self, name, size):
        self.seq += 1
        self.live[name] = self.seq
        self.queues[size].append((self.seq, name))

    def delete(self, name):
        # lazy: the entry in its queue is skipped later
        self.live.pop(name, None)

    def front(self, queue):
        # drop deleted people from the front of a queue
        while queue:
            seq, name = queue[0]
            if self.live.get(name) == seq:
                break
            queue.popleft()
        return queue[0] if queue else None

    def find_first_match(self, table):
        best = None
        for size, queue in self.queues.items():
            if size > table:
                continue
            head = self.front(queue)
            # the earliest head among all sizes that fit
            if head and (not best or head[0] < best[0]):
                best = head
        return best[1] if best else None
```

152. Implement LRU Cache. [H]

```
# HELPERS for the base question
class Node:
    __slots__ = ("key", "value", "prev", "next")

    def __init__(self, key=None, value=None):
        self.key, self.value = key, value
        self.prev = self.next = None


# BASE QUESTION (optimal): Hash map plus doubly linked
# list (built by hand)
# Idea: Each key maps to a node of a doubly linked list
# ordered by recency, with sentinel head and tail nodes so
# no operation has to special-case an empty list.
class LRUCache:
    """get and put in O(1): a hash map plus a doubly
    linked list, most recently used item at the front."""

    def __init__(self, capacity):
        self.cap, self.map = capacity, {}
        # sentinel nodes mean no special cases for the
        # ends
        self.head, self.tail = Node(), Node()
```

```
        self.head.next = self.tail
        self.tail.prev = self.head

    def unlink(self, node):
        node.prev.next = node.next
        node.next.prev = node.prev

    def push_front(self, node):
        node.next, node.prev = self.head.next, self.head
        self.head.next.prev = node
        self.head.next = node

    def get(self, key):
        node = self.map.get(key)
        if node is None:
            return -1
        # reading makes it the most recently used
        self.unlink(node)
        self.push_front(node)
        return node.value

    def put(self, key, value):
        node = self.map.get(key)
        if node:
            node.value = value
            self.unlink(node)
            self.push_front(node)
            return
        if len(self.map) == self.cap:
            # full: evict the least recently used (at the
            # back)
            oldest = self.tail.prev
            self.unlink(oldest)
            del self.map[oldest.key]
        node = Node(key, value)
        self.map[key] = node
        self.push_front(node)
```

GROUP 19 OF 43 | 7 QUESTIONS, ONE CODE TEMPLATE
Rolling state: the next state comes only from the previous one

153. Climbing Stairs [E]

```
# BASE QUESTION (optimal): Two running values
# Idea: Keep ways for the previous two steps and advance:
# (a, b) = (b, a + b). Start with ways(0) = ways(1) = 1.
# No table and no recursion.
def climb(n):
    # ways(n) = ways(n - 1) + ways(n - 2): O(n), O(1)
    # space
    a, b = 1, 1
    for _ in range(n - 1):
        a, b = b, a + b
    return b if n >= 1 else 1
```

154. Fibonacci Numbers [E]

```
# BASE QUESTION (optimal): Iteration with two variables
# Idea: Keep the previous two values and advance n times.
# Memoising the recursion gives the same time with O(n)
# memory; the loop needs no memory and no recursion.
def fib(n):
    # bottom-up with two variables: O(n) time, O(1) memory
    a, b = 1, 1
    for _ in range(n):
        a, b = b, a + b
    return a
```

155. Maximize sum without choosing adjacent elements [M]

```
# BASE QUESTION (optimal): Two running values (take, skip)
# Idea: take is the best total that includes the current
# element, skip the best that excludes it. For the next
# element: new take = skip + x (the previous element must
# have been skipped), new skip = max(take, skip).
def rob_line(nums):
    # take: best when the current element is taken,
    # skip: best when it is not
    take = skip = 0
    for x in nums:
        take, skip = skip + x, max(take, skip)
    return max(take, skip)
```

156. Given n houses in a line with money in each, find the maximum amount a robber can steal, without stealing from two adjacent houses. [M]

```
# BASE QUESTION (optimal): Two running values (take, skip)
# Idea: take is the best total that includes the current
# element, skip the best that excludes it. For the next
# element: new take = skip + x (the previous element must
# have been skipped), new skip = max(take, skip).
def rob_line(nums):
    # take = best sum if we take this house; skip = if we
    # do not
    take = skip = 0
    for x in nums:
        take, skip = skip + x, max(take, skip)
    return max(take, skip)
```

157. Paint House II [M]

```
# BASE QUESTION (optimal): Two smallest values of the
# previous row
# Idea: A cell only needs the cheapest previous colour
# that differs from its own.
def min_cost_paint_houses(costs):
    prev = [0] * len(costs[0])
    for row in costs:
        best = sorted(
            range(len(prev)), key=lambda j: prev[j]
```

```
        )[:2]
        first = best[0]
        second = best[1] if len(best) > 1 else best[0]
        prev = [
            row[j]
            + (
                prev[second]
                if j == first
                else prev[first]
            )
            for j in range(len(row))
        ]
    return min(prev)
```

158. Write SQL code to publish the Fibonacci series. [M]

```
import sqlite3


# HELPERS for the base question
# a recursive query: each row is computed from the
# previous one
FIB_SQL = """
WITH RECURSIVE fib(n, a, b) AS (
    SELECT 1, 0, 1
    UNION ALL
    SELECT n + 1, b, a + b FROM fib WHERE n < ?
)
SELECT a FROM fib
"""


# BASE QUESTION (optimal): Recursive CTE carrying (n, a,
# b)
# Idea: The anchor row is (1, 0, 1): the first term is 0
# and the next pair is (0, 1). Each recursive step
# produces (n + 1, b, a + b) while n is below the limit,
# so the term in column a is the Fibonacci number.
# Answer: Bind the limit as a query parameter (never build
# the SQL by string concatenation) and fetch the single
# column. The recursion lives inside the database, so
# there is one round trip regardless of n.
def fibonacci(count):
    # FOLLOW-UP 4: from app code: count is a parameter
    # one round trip: the recursion runs inside the
    # database;
    # the limit is a bound parameter, never glued into the
    # SQL
    with sqlite3.connect(":memory:") as db:
        rows = db.execute(FIB_SQL, (count,)).fetchall()
    return [r[0] for r in rows]
```

159. Constrained Dice Roll Sequences [H]

```
# BASE QUESTION (optimal): DP over the last value and its
# run length
# Idea: dp[v][r] counts prefixes ending in value v
# repeated r times.
def count_dice_sequences(n, roll_max):
    mod = 10**9 + 7
    dp = [[0] * (roll_max[v] + 1) for v in range(6)]
    for v in range(6):
        dp[v][1] = 1
    for _ in range(n - 1):
        total = sum(sum(row) for row in dp) % mod
        new = [
            [0] * (roll_max[v] + 1) for v in range(6)
        ]
        for v in range(6):
            # a different previous value starts a run of
            # length one
            new[v][1] = (total - sum(dp[v])) % mod
            for r in range(1, roll_max[v]):
                new[v][r + 1] = dp[v][r]
```

```
    dp = new
return sum(sum(row) for row in dp) % mod
```

GROUP 20 OF 43 | 7 QUESTIONS, ONE CODE TEMPLATE
Two pointers: move i and j toward each other (or along two lists)

160. Reverse a linked list. [E]

```
# BASE QUESTION (optimal): Flip the pointers in one pass
# Idea: Walk with prev and cur. Save cur.next, point cur
# at prev, then move both forward. When cur is None, prev
# is the new head. Saving next before the overwrite is the
# step people forget.
def reverse_list(head):
    # point every node back at the one before it
    prev = None
    while head:
        head.next, prev, head = prev, head, head.next
    return prev
```

161. Sort a binary array in linear time. [E]

```
# BASE QUESTION (optimal): Two pointers swapping from both
# ends
# Idea: Keep lo at the next position for a 0 and hi at the
# next position for a 1. Advance lo past zeros, retreat hi
# past ones, and when both are stuck swap them. One pass,
# in place.
def sort_binary(arr):
    # two pointers: zeros collect on the left, ones on the
    # right; every step swaps or moves one pointer: O(n)
    lo, hi = 0, len(arr) - 1
    while lo < hi:
        if arr[lo] == 0:
            lo += 1
        elif arr[hi] == 1:
            hi -= 1
        else:
            # a 1 on the left and a 0 on the right: swap
            # them
            arr[lo], arr[hi] = arr[hi], arr[lo]
            lo += 1
            hi -= 1
    return arr
```

162. Is this a valid palindrome? [E]

```
import unicodedata


# HELPERS for the base question
# Answer: Normalise first (unicodedata.normalize and
# casefold) and decide what counts as a letter.
# str.isalnum handles many scripts, but combining marks
# need normalisation.
def clean(s):
    # follow-up: unicode. Decompose accents, drop the
    # combining marks, fold case, keep letters and digits.
    text = unicodedata.normalize("NFKD", s)
    text = "".join(ch for ch in text
                   if not unicodedata.combining(ch))
    return [ch for ch in text.casefold() if ch.isalnum()]


# BASE QUESTION (optimal): Two pointers that skip ignored
# characters
# Idea: Move left and right toward each other, skipping
# characters that are not letters or digits, and compare
# the lowercase forms. The first mismatch ends it. Nothing
# is copied.
def is_palindrome(s):
    # compare the cleaned characters from both ends inward
    chars = clean(s)
    lo, hi = 0, len(chars) - 1
    while lo < hi:
```

```
        if chars[lo] != chars[hi]:
            return False
        lo += 1
        hi -= 1
    return True
```

163. Reverse a Sentence [E]

```
# HELPERS for the base question
def reverse_range(arr, lo, hi):
    # reverse arr[lo..hi] in place with two pointers
    while lo < hi:
        arr[lo], arr[hi] = arr[hi], arr[lo]
        lo += 1
        hi -= 1


def each_word(arr):
    # yield the (start, end) of every word: a run of
    # characters without a space
    i = 0
    while i < len(arr):
        if arr[i] == " ":
            i += 1
            continue
        j = i
        while j + 1 < len(arr) and arr[j + 1] != " ":
            j += 1
        yield i, j
        i = j + 1


# BASE QUESTION (optimal): Reverse the whole array, then
# reverse each word in place
# Idea: Reverse every character; each word is now
# backwards but in the right place. Walk the array and
# reverse each maximal run of non-space characters.
# Everything happens inside the same array.
def reverse_words(arr):
    # reverse the whole array, so the words are in the
    # right
    # order but spelled backwards; then reverse each word
    # back. O(n) time and O(1) extra space.
    reverse_range(arr, 0, len(arr) - 1)
    for start, end in list(each_word(arr)):
        reverse_range(arr, start, end)
    return arr
```

164. Find median of merged RLE arrays [M]

```
# HELPERS for the base question
def kth_smallest(runs_a, runs_b, k):
    # zero-based rank in the merged order, walking the
    # runs
    # of two sorted run-length lists; nothing is expanded
    i = j = seen = 0
    while True:
        take_a = j >= len(runs_b)
        if not take_a and i < len(runs_a):
            take_a = runs_a[i][0] <= runs_b[j][0]
        value, count = runs_a[i] if take_a else runs_b[j]
        seen += count
        if seen > k:
            return value
        if take_a:
            i += 1
        else:
            j += 1


# BASE QUESTION (optimal): Merge the runs and walk to the
# two middle ranks
# Idea: Let N be the sum of all counts.
def median_rle(a, b):
```

```
    # a and b: [(value, count)] sorted by value
    total = sum(c for _, c in a) + sum(c for _, c in b)
    if total % 2:
        return kth_smallest(a, b, total // 2)
    low = kth_smallest(a, b, total // 2 - 1)
    high = kth_smallest(a, b, total // 2)
    return (low + high) / 2
```

165. Run-Length Vector Storage + Dot Product [M]

```
# BASE QUESTION (optimal): Two-pointer over runs
# Idea: Each run is (value, count). Compare the remaining
# lengths of the two current runs, multiply the values by
# the overlap, then advance whichever run was exhausted
# (or both).
def rle_dot(a, b):
    # two pointers over the runs: never expand anything
    i = j = 0
    left_a = a[0][1] if a else 0
    left_b = b[0][1] if b else 0
    total = 0
    while i < len(a) and j < len(b):
        # both runs overlap for min(left_a, left_b)
        # positions
        take = min(left_a, left_b)
        total += a[i][0] * b[j][0] * take
        left_a -= take
        left_b -= take
        if left_a == 0:
            i += 1
            left_a = a[i][1] if i < len(a) else 0
        if left_b == 0:
            j += 1
            left_b = b[j][1] if j < len(b) else 0
    return total
```

166. Count Unique Triplets That Sum to Zero [M]

```
# BASE QUESTION (optimal): Sort and two pointers
# Idea: Fix the smallest value and sweep the other two
# inward, skipping duplicates.
def count_zero_sum_triplets(nums):
    nums = sorted(nums)
    count = 0
    for i in range(len(nums) - 2):
        if i and nums[i] == nums[i - 1]:
            continue
        left, right = i + 1, len(nums) - 1
        while left < right:
            total = nums[i] + nums[left] + nums[right]
            if total < 0:
                left += 1
            elif total > 0:
                right -= 1
            else:
                count += 1
                left += 1
                right -= 1
                while (
                    left < right
                    and nums[left] == nums[left - 1]
                ):
                    left += 1
    return count
```

GROUP 21 OF 43 | 7 QUESTIONS, ONE CODE TEMPLATE
Walk two strings at once with two indices

167. Find the First Occurrence of a Substring [E]

```
# BASE QUESTION (optimal): Naive scan with early exit
# Idea: Compare character by character and stop at the
# first mismatch.
def str_str(haystack, needle):
    n, m = len(haystack), len(needle)
    for i in range(n - m + 1):
        j = 0
        while j < m and haystack[i + j] == needle[j]:
            j += 1
        if j == m:
            return i
    return -1
```

168. Determine if two words are sorted in lexicographic order. [E]

```
# HELPERS for the base question
ALPHABET = "abcdefghijklmnopqrstuvwxyz"


# BASE QUESTION (optimal): Compare character by character,
# optionally with a custom alphabet
# Idea: Find the first position where the words differ and
# compare the two characters' ranks. If one word is a
# prefix of the other, the shorter one comes first, so
# 'app' is before 'apple' but not the reverse.
def in_order(a, b, rank):
    # compare letter by letter by rank; the first
    # difference
    # decides. If one word is a prefix of the other, the
    # shorter one comes first. A character outside the
    # alphabet is ranked after all the known ones.
    def key(ch):
        return rank.get(ch, len(rank) + ord(ch))

    for x, y in zip(a, b):
        if x != y:
            return key(x) < key(y)
    return len(a) <= len(b)
```

169. Return all unique substrings of a given string. [E]

```
# BASE QUESTION (optimal): Count with a suffix array and
# LCP
# Idea: Sort the suffixes; the number of distinct
# substrings is sum over suffixes of (length) minus sum of
# the LCP of neighbours in sorted order, because each new
# suffix adds only the prefixes that the previous one did
# not already contribute.
def unique_substrings(s):
    # simplest: every slice into a set, then sort. O(n^3).
    return sorted({s[i:j] for i in range(len(s))
                   for j in range(i + 1, len(s) + 1)})
```

170. Can board states be transformed? [M]

```
# HELPERS for the base question
def pieces(board):
    # the non-blank symbols with their positions
    return [(ch, i) for i, ch in enumerate(board)
            if ch != "X"]


# BASE QUESTION (optimal): Two pointers over the pieces
# Idea: Skip blanks in both strings and compare the next
# piece in each: they must be the same letter.
def can_transform(start, end):
    # L moves left, R moves right, X is blank. Pieces
```

```
    # never pass
    # each other, so the order of the non-X symbols is
    # fixed;
    # a '#' is a wall: it never moves and no piece crosses
    # it.
    if len(start) != len(end):
        return False
    a, b = pieces(start), pieces(end)
    if len(a) != len(b):
        return False
    for (c1, i), (c2, j) in zip(a, b):
        if c1 != c2:
            return False
        # an L may only end up further left, an R further
        # right
        if c1 == "L" and i < j:
            return False
        if c1 == "R" and i > j:
            return False
        # a wall must stay exactly where it is
        if c1 == "#" and i != j:
            return False
    return True
```

171. Decide string transform with directional tokens and walls [M]

```
# HELPERS for the base question
def pieces(board):
    # everything except the empty '.' slots, with
    # positions
    return [(ch, i) for i, ch in enumerate(board)
            if ch != "."]


# BASE QUESTION (optimal): Compare the non-dot sequences
# with position rules
# Idea: Walk both strings skipping dots. The next symbol
# must be equal; a wall must be at the same index; an L
# must satisfy start index >= target index; an R must
# satisfy start index <= target index.
def can_transform(start, end):
    # 'L' moves left, 'R' moves right, 'X' is an immovable
    # wall. Pieces never pass each other, so their order
    # is
    # fixed; each one only has a direction it may move.
    if len(start) != len(end):
        return False
    a, b = pieces(start), pieces(end)
    if len(a) != len(b):
        return False
    for (c1, i), (c2, j) in zip(a, b):
        if c1 != c2:
            return False
        if c1 == "X" and i != j:
            return False
        if c1 == "L" and i < j:
            return False
        if c1 == "R" and i > j:
            return False
    return True
```

172. Determine if two sentences are similar. [M]

```
# BASE QUESTION (optimal): Set of pairs (not transitive)
# Idea: Put each pair into a set in both directions, so a
# word pair is similar if the words are equal or the pair
# is in the set. Sentences of different length are never
# similar.
def similar_direct(s1, s2, pairs):
    # same length, and every pair of words is equal or a
    # listed pair (in either direction)
    if len(s1) != len(s2):
        return False
    allowed = {frozenset(p) for p in pairs}
```

```
    return all(a == b or frozenset((a, b)) in allowed
               for a, b in zip(s1, s2))
```

173. Find Dictionary Matches for a Jammed String [H]

```
from itertools import groupby


# HELPERS for the base question
def runs(word):
    # "heello" -> [("h", 1), ("e", 2), ("l", 2), ("o", 1)]
    return [(ch, len(list(g))) for ch, g in groupby(word)]


# BASE QUESTION (optimal): Index the dictionary by its run
# letters
# Idea: Group words by the string of their distinct run
# letters ('hello' -> 'helo'). A query can only match
# words in its own group, so compare run counts only
# there. Build the index once and reuse it for many
# queries.
# Answer: Only the comparison changes. The index (grouping
# by run letters) is the same, so make the minimum stretch
# a parameter, as the code does with min_stretch.
class JammedIndex:
    """A query run may be longer than the word's run
    when it is at least min_stretch long."""

    def __init__(self, words):
        # words grouped by their run letters ("hello" ->
        # "helo")
        self.groups = {}
        for w in words:
            r = runs(w)
            letters = "".join(ch for ch, _ in r)
            self.groups.setdefault(letters, []).append(
                (w, [n for _, n in r]))

    # FOLLOW-UP 1: min_stretch=1: any run may be longer
    def matches(self, query, min_stretch=3):
        r = runs(query)
        letters = "".join(ch for ch, _ in r)
        counts = [n for _, n in r]
        found = []
        # only words with the same run letters can match
        group = self.groups.get(letters, [])
        for word, word_counts in group:
            if all(self.fits(q, w, min_stretch)
                   for q, w in zip(counts, word_counts)):
                found.append(word)
        return sorted(found)

    @staticmethod
    def fits(q, w, min_stretch):
        # the same length, or a stretch of at least
        # min_stretch
        return q == w or (q > w and q >= min_stretch)
```

GROUP 22 OF 43 | 7 QUESTIONS, ONE CODE TEMPLATE
Simulate a machine: a class with state and one branch per kind of event

174. Mario-Style Score Tracker (OOP) [E]

```
# BASE QUESTION (optimal): A small state class
# Idea: One method owns the rules; the game ends when
# lives hit zero.
class MarioGame:
    def __init__(self):
        self.score = 0
        self.lives = 3
        self.big = False

    def process(self, event):
        if self.lives == 0:
            return
        if event == "coin":
            self.score += 100
        elif event == "mushroom":
            self.score += 1000
            self.big = True
        elif event == "enemy":
            self.score += 200
        elif event == "damage":
            if self.big:
                self.big = False
            else:
                self.lives -= 1

    def run(self, events):
        for event in events:
            self.process(event)

    def get_score(self):
        return self.score

    def is_alive(self):
        return self.lives > 0
```

175. Tic-Tac-Toe System with K Players and 3-in-a-Row Win [M]

```
# HELPERS used by the code below
DIRECTIONS = ((0, 1), (1, 0), (1, 1), (1, -1))


# BASE QUESTION (optimal): Check the four lines through
# the new mark
# Idea: Place the mark, then for each of the four
# directions walk outward both ways while the cells hold
# the same player and add the two counts plus one. A total
# of at least 3 wins immediately.
# Answer: Replace the constant 3 by w in the run check:
# the cost per move becomes O(w) because each of the four
# directions walks at most w - 1 cells each way. The board
# size does not matter any more, which is the point of
# checking locally.
class TicTacToe:
    def __init__(self, n, k, target=3):
        self.n, self.k, self.target = n, k, target
        self.board = [[0] * n for _ in range(n)]
        self.moves = 0
        self.over = False

    def inside(self, r, c):
        return 0 <= r < self.n and 0 <= c < self.n

    def run_length(self, r, c, dr, dc):
        # marks of the same player in a line through (r,
        # c)
        player, run = self.board[r][c], 1
        for sign in (1, -1):
```

```
            a, b = r + sign * dr, c + sign * dc
            while self.inside(a, b) and \
                    self.board[a][b] == player:
                run += 1
                a, b = a + sign * dr, b + sign * dc
        return run

    def move(self, player, r, c):
        # whose turn is it? players go 1, 2, ..., k, 1,
        # ...
        turn = self.moves % self.k + 1
        if self.over or player != turn:
            raise ValueError("not this player's turn")
        if not self.inside(r, c) or self.board[r][c]:
            raise ValueError("illegal cell")
        self.board[r][c] = player
        self.moves += 1
        # only lines through the new mark can have changed
        for dr, dc in DIRECTIONS:
            run = self.run_length(r, c, dr, dc)
            # FOLLOW-UP 1: win with `target` in a row
            if run >= self.target:
                self.over = True
                return f"player {player} wins"
        if self.moves == self.n * self.n:
            self.over = True
            return "draw"
        return "continue"
```

176. Design Tic-Tac-Toe With K Players [M]

```
# HELPERS for the base question
DIRECTIONS = ((0, 1), (1, 0), (1, 1), (1, -1))


# BASE QUESTION (optimal): Counters per player and per
# line
# Idea: rows[p][r], cols[p][c], diag[p] and anti[p] count
# each player's marks on those lines. A move increments
# the counters for its row, column and, if applicable, the
# diagonals; the player wins when one reaches n.
# Answer: Counters per line no longer work, because a line
# can hold a run anywhere. After each move look along the
# four directions from that cell, counting same-player
# neighbours up to m in both directions. O(m) per move.
class TicTacToe:
    def __init__(self, n, k, target=3):
        self.n, self.k, self.target = n, k, target
        # FOLLOW-UP 3: a huge or unbounded board
        # only occupied cells are stored (works for huge
        # boards)
        self.cells = {}
        self.history = []
        self.winner = None

    def turn(self):
        # players go 1, 2, ..., k, then 1 again
        return len(self.history) % self.k + 1

    def run_length(self, r, c, dr, dc, player):
        # marks of `player` in a line through (r, c)
        run = 1
        for sign in (1, -1):
            a, b = r + sign * dr, c + sign * dc
            while self.cells.get((a, b)) == player:
                run += 1
                a, b = a + sign * dr, b + sign * dc
        return run

    def move(self, player, r, c):
        if self.winner is not None:
            return "Game Over"
        # an invalid move changes nothing
```

```
        if not 1 <= player <= self.k:
            return "Invalid move"
        if player != self.turn():
            return "Invalid move"
        if not (0 <= r < self.n and 0 <= c < self.n):
            return "Invalid move"
        if (r, c) in self.cells:
            return "Invalid move"
        self.cells[(r, c)] = player
        self.history.append((r, c))
        # only the four lines through the new mark can win
        for dr, dc in DIRECTIONS:
            run = self.run_length(r, c, dr, dc, player)
            # FOLLOW-UP 1: win with `target` in a row
            if run >= self.target:
                self.winner = player
                return f"{player} won"
        return "Continue"

    # FOLLOW-UP 2: undo the last move
    def undo(self):
        # take the last move back (also reopens a finished
        # game)
        if not self.history:
            return False
        del self.cells[self.history.pop()]
        self.winner = None
        return True
```

177. Infinite Deck Card Duel [M]

```
import random


# BASE QUESTION (optimal): A seedable deck and pure game
# rules
# Idea: The deck owns the Random; the rules are plain
# functions of hands and win counts.
class InfiniteDeck:
    def __init__(self, seed=None):
        self.rng = random.Random(seed)

    def draw(self):
        return self.rng.randint(1, 13)


# BASE QUESTION (optimal): A seedable deck and pure game
# rules
# Idea: The deck owns the Random; the rules are plain
# functions of hands and win counts.
def round_winner(hand1, hand2):
    if max(hand1) == max(hand2):
        return 0
    return 1 if max(hand1) > max(hand2) else 2


# BASE QUESTION (optimal): A seedable deck and pure game
# rules
# Idea: The deck owns the Random; the rules are plain
# functions of hands and win counts.
def decide_champion(wins1, wins2):
    if wins1 - wins2 >= 2:
        return 1
    return 2 if wins2 - wins1 >= 2 else 0


# BASE QUESTION (optimal): A seedable deck and pure game
# rules
# Idea: The deck owns the Random; the rules are plain
# functions of hands and win counts.
def play_game(deck, hand_size=5, num_rounds=100):
    wins = {1: 0, 2: 0}
    for _ in range(num_rounds):
```

```
        h1 = [deck.draw() for _ in range(hand_size)]
        h2 = [deck.draw() for _ in range(hand_size)]
        winner = round_winner(h1, h2)
        if winner:
            wins[winner] += 1
    return decide_champion(wins[1], wins[2])
```

178. Implement iterator merging lists with filtering [M]

```
# BASE QUESTION (optimal): Lazy iterator with one pointer
# per list and a lookahead
# Idea: Keep a pointer per list and a rotation index.
# _advance() scans round robin for the next value that is
# not blocked and not already yielded and stores it as the
# lookahead (or None when everything is exhausted).
class FilteredZigzag:
    """Take one item from each list in turn; skip blocked
    ids and ids already returned. Has hasNext / next."""

    def __init__(self, lists, blocked=()):
        self.lists = lists
        self.blocked = set(blocked)
        self.pos = [0] * len(lists)
        self.turn = 0
        self.seen = set()
        # one item of lookahead makes hasNext easy
        self.ahead = None
        self.advance()

    def advance(self):
        self.ahead = None
        misses = 0
        # stop when every list had nothing left in a full
        # round
        while misses < len(self.lists):
            i = self.turn
            self.turn = (self.turn + 1) % len(self.lists)
            if self.pos[i] >= len(self.lists[i]):
                misses += 1
                continue
            value = self.lists[i][self.pos[i]]
            self.pos[i] += 1
            misses = 0
            blocked = value in self.blocked
            skip = blocked or value in self.seen
            if not skip:
                self.ahead = value
                return

    def has_next(self):
        return self.ahead is not None

    def next(self):
        value = self.ahead
        self.seen.add(value)
        self.advance()
        return value
```

179. Design an elevator control system [M]

```
# HELPERS for the base question
INF = float("inf")


class Request:
    def __init__(self, floor, direction):
        # direction: +1 up, -1 down
        self.floor, self.direction = floor, direction


# BASE QUESTION (optimal): Elevator (SCAN) per car and
# cost-based assignment of calls
# Idea: Each car keeps a set of stops and a direction.
# step() moves one floor in its direction and removes a
```

```
# stop it reaches; when no stops remain ahead it reverses,
# and when none remain at all it goes idle.
class Elevator:
    def __init__(self, car_id, floor=0):
        self.id, self.floor = car_id, floor
        # 0 = idle, +1 = going up, -1 = going down
        self.direction = 0
        self.stops = set()
        self.in_service = True

    def cost(self, req):
        # how bad is it to give this car the request?
        if not self.in_service:
            return INF
        dist = abs(self.floor - req.floor)
        if self.direction == 0:
            return dist
        # on the way: the car is moving toward the floor
        # and
        # the call goes the same way
        gap = req.floor - self.floor
        ahead = gap * self.direction >= 0
        if ahead and req.direction == self.direction:
            return dist
        # otherwise it finishes its sweep first
        span = max(self.stops) - min(self.stops) + 1
        return dist + 2 * span + 10

    def step(self):
        # one tick: serve a stop here, then move one floor
        served = None
        if self.floor in self.stops:
            self.stops.discard(self.floor)
            served = self.floor
        if not self.stops:
            self.direction = 0
            return served
        up = [s for s in self.stops if s > self.floor]
        down = [s for s in self.stops if s < self.floor]
        # keep the direction while there are stops ahead
        # (SCAN)
        if (self.direction >= 0 and up) or (
                self.direction < 0 and not down):
            self.direction = 1
        else:
            self.direction = -1
        self.floor += self.direction
        return served


# BASE QUESTION (optimal): Elevator (SCAN) per car and
# cost-based assignment of calls
# Idea: Each car keeps a set of stops and a direction.
# step() moves one floor in its direction and removes a
# stop it reaches; when no stops remain ahead it reverses,
# and when none remain at all it goes idle.
# Answer: Add modes: during the morning rush park idle
# cars in the lobby and let them go up with a full load;
# in the evening do the opposite.
class Controller:
    def __init__(self, cars, lobby=0):
        self.cars, self.lobby = cars, lobby

    def assign(self, req):
        # the cheapest in-service car takes the call
        car = min(self.cars, key=lambda c: c.cost(req))
        if car.cost(req) == INF:
            return None
        car.stops.add(req.floor)
        return car.id

    def tick(self):
        return {c.id: c.step() for c in self.cars}
```

```
    # FOLLOW-UP 3: a failed car hands its stops over
    def fail(self, car_id):
        # a broken car: take it out and hand its stops to
        # others
        car = next(c for c in self.cars if c.id == car_id)
        car.in_service = False
        orphans, car.stops = car.stops, set()
        for floor in orphans:
            self.assign(Request(floor, 0))

    # FOLLOW-UP 1, 4: morning rush: idle cars wait in the
    # lobby
    def park_idle(self):
        # morning rush: idle cars wait in the lobby
        for car in self.cars:
            if car.in_service and not car.stops:
                car.stops.add(self.lobby)
```

180. Simulate In-Place Cellular State Updates [H]

```
# HELPERS for the base question
AROUND = [(dr, dc) for dr in (-1, 0, 1)
          for dc in (-1, 0, 1) if (dr, dc) != (0, 0)]


def live_neighbors(board, r, c, wrap=False):
    m, n = len(board), len(board[0])
    count = 0
    for dr, dc in AROUND:
        a, b = r + dr, c + dc
        if wrap:
            # a torus: the edges are glued together
            a, b = a % m, b % n
        elif not (0 <= a < m and 0 <= b < n):
            continue
        # bit 0 is the OLD state, even for updated cells
        count += board[a][b] & 1
    return count


# BASE QUESTION (optimal): Two-bit encoding in the same
# board
# Idea: Bit 0 holds the current state and bit 1 holds the
# next state. First pass: count neighbors using only bit 0
# (value & 1) and set bit 1 by the rules.
def step_in_place(board, wrap=False):
    m, n = len(board), len(board[0])
    for r in range(m):
        for c in range(n):
            live = live_neighbors(board, r, c, wrap)
            alive = board[r][c] & 1
            # bit 1 stores the NEW state
            if live == 3 or (alive and live == 2):
                board[r][c] |= 2
    for r in range(m):
        for c in range(n):
            # shift the new state into place
            board[r][c] >>= 1
    return board
```

GROUP 23 OF 43 | 6 QUESTIONS, ONE CODE TEMPLATE
Top-K by frequency: Counter plus nsmallest

181. Top-K Frequent in Large Logs [M]

```
import heapq


from collections import Counter


# HELPERS for the base question
def speaker(line):
    # "A says hi" -> "A"
    return line.split(" ", 1)[0]


def rank_key(item):
    # more messages first; ties by name
    return (-item[1], item[0])


# BASE QUESTION (optimal): Count, then keep a size-K heap
# Idea: Parse the speaker from each line and count in a
# hash map. nsmallest keeps a heap of size k, so it never
# sorts all d distinct users. Ties break by user name so
# the output is deterministic.
def top_k_speakers(chat_log, k):
    counts = Counter(speaker(line) for line in chat_log)
    # a heap of size k: O(n + d log k)
    items = counts.items()
    best = heapq.nsmallest(k, items, key=rank_key)
    return [user for user, _ in best]
```

182. Rearrange Tasks With Cooldown [M]

```
from collections import Counter


# BASE QUESTION (optimal): Closed form from the most
# frequent task
# Idea: Let m be the highest frequency and k the number of
# tasks that have that frequency. The skeleton needs (m -
# 1) * (n + 1) + k slots.
def least_interval(tasks, n):
    # closed form: the most frequent task sets the frame
    counts = Counter(tasks).values()
    most = max(counts, default=0)
    # how many tasks are tied for the most
    tied = sum(1 for c in counts if c == most)
    if not tasks:
        return 0
    # (most - 1) full frames of n + 1 slots, plus the last
    # row
    return max(len(tasks), (most - 1) * (n + 1) + tied)
```

183. Implement frequency + distance top■K queries [M]

```
import heapq


from collections import Counter


# BASE QUESTION (optimal): Q1: heap with the full key
# Idea: Count with a Counter, then heapq.nsmallest(k, ...)
# with the key (-frequency, squared distance, x, y).
# Squared distances avoid square roots and are exact for
# integers.
def top_points(points, k):
    # more frequent first, then closer to the origin, then
    # x, y
```

```
    counts = Counter(map(tuple, points))

    def rank(p):
        dist = p[0] ** 2 + p[1] ** 2
        return (-counts[p], dist, p[0], p[1])

    best = heapq.nsmallest(k, counts, key=rank)
    return [list(p) for p in best]
```

184. Solve chat, grid paths, and car rentals [M]

```
import heapq


from collections import Counter


# BASE QUESTION (optimal): Q1: top-K active users with a
# size-K heap
# Idea: Count messages in a hash map, then keep a min-heap
# of the best k (count, user). That beats sorting all d
# users when k is small. Ties break by smaller user id so
# the answer is deterministic.
def top_k_users(messages, k):
    # Q1, heap: O(n + d log k); sorted order breaks ties
    counts = Counter(messages)
    best = heapq.nsmallest(
        k, counts.items(), key=lambda kv: (-kv[1], kv[0]))
    return [user for user, _ in best]


# BASE QUESTION (optimal): Q1: top-K active users with a
# size-K heap
# Idea: Count messages in a hash map, then keep a min-heap
# of the best k (count, user). That beats sorting all d
# users when k is small. Ties break by smaller user id so
# the answer is deterministic.
def count_paths(m, n, blocked=(), mod=None):
    # Q2: start (m-1, 0), end (m-1, n-1), moves right-up,
    # right, right-down. One column of ways is enough:
    # O(m).
    wall = set(blocked)
    ways = [0] * m
    ways[m - 1] = 1
    for c in range(1, n):
        nxt = [0] * m
        for r in range(m):
            # FOLLOW-UP 4: blocked cells (obstacles) give
            # no paths
            if (r, c) in wall:
                # a blocked cell contributes no paths
                continue
            total = ways[r]
            if r > 0:
                total += ways[r - 1]
            if r + 1 < m:
                total += ways[r + 1]
            nxt[r] = total % mod if mod else total
        ways = nxt
    return ways[m - 1]


# BASE QUESTION (optimal): Q1: top-K active users with a
# size-K heap
# Idea: Count messages in a hash map, then keep a min-heap
# of the best k (count, user). That beats sorting all d
# users when k is small. Ties break by smaller user id so
# the answer is deterministic.
def min_cars(rentals):
    # Q3: rentals are (pickup, return); how many cars are
    # needed?
    # A heap holds the return times of the cars on the
    # road.
```

```
    cars = []
    for pickup, back in sorted(rentals):
        if cars and cars[0] <= pickup:
            # a car came back in time: reuse it
            heapq.heapreplace(cars, back)
        else:
            heapq.heappush(cars, back)
    return len(cars)
```

185. Top-K Chatters by Word Count [M]

```
import heapq


from collections import Counter


# BASE QUESTION (optimal): Counter and a bounded selection
# Idea: Sum the words per author, then pick the k best
# with a heap keyed by (-count, name).
def top_k_chatters(lines, k):
    counts = Counter()
    for line in lines:
        name, body = line.split(": ", 1)
        counts[name] += len(body.split())
    return heapq.nsmallest(
        k,
        counts.items(),
        key=lambda kv: (-kv[1], kv[0]),
    )
```

186. Find top/bottom-k words in list or stream [M]

```
import heapq


from collections import Counter


# BASE QUESTION (optimal): Counter plus two heaps of size
# K
# Idea: One pass counts the words. heapq.nsmallest with
# key (-count, word) gives the K most frequent and with
# key (count, word) the K least frequent, each in O(d log
# k) without a full sort.
def top_bottom(words, k):
    # ties are broken alphabetically
    c = Counter(words)
    top = heapq.nsmallest(k, c, key=lambda w: (-c[w], w))
    low = heapq.nsmallest(k, c, key=lambda w: (c[w], w))
    bottom = low
    return top, bottom
```

GROUP 24 OF 43 | 6 QUESTIONS, ONE CODE TEMPLATE
Digits and characters: peel one at a time and carry

187. Caesar Cipher: Recover Key, Encrypt, and Decrypt [E]

```
# BASE QUESTION (optimal): Key from one letter pair,
# shared shift helper
# Idea: The difference of one letter pair is the key; the
# same helper shifts forward or backward.
def find_shift(plaintext, ciphertext):
    for p, c in zip(plaintext, ciphertext):
        if "a" <= p <= "z":
            return (ord(c) - ord(p)) % 26
    return 0


# BASE QUESTION (optimal): Key from one letter pair,
# shared shift helper
# Idea: The difference of one letter pair is the key; the
# same helper shifts forward or backward.
def shift_text(text, k):
    out = []
    for ch in text:
        if "a" <= ch <= "z":
            ch = chr((ord(ch) - 97 + k) % 26 + 97)
        out.append(ch)
    return "".join(out)


# BASE QUESTION (optimal): Key from one letter pair,
# shared shift helper
# Idea: The difference of one letter pair is the key; the
# same helper shifts forward or backward.
def encrypt(text, k):
    return shift_text(text, k)


# BASE QUESTION (optimal): Key from one letter pair,
# shared shift helper
# Idea: The difference of one letter pair is the key; the
# same helper shifts forward or backward.
def decrypt(text, k):
    return shift_text(text, -k)
```

188. Add two big integers from digit lists [M]

```
# BASE QUESTION (optimal): Digit by digit with a carry
# Idea: Walk both lists from the right with indices i and
# j; at each step add a[i] (or 0 once exhausted), b[j] (or
# 0) and the carry, append sum % 10, and carry = sum //
# 10.
def add_digits(a, b):
    # arrays with the most significant digit FIRST
    i, j, carry, out = len(a) - 1, len(b) - 1, 0, []
    while i >= 0 or j >= 0 or carry:
        total = carry
        total += a[i] if i >= 0 else 0
        total += b[j] if j >= 0 else 0
        out.append(total % 10)
        carry = total // 10
        i, j = i - 1, j - 1
    return out[::-1] or [0]
```

189. Solve several coding interview problems [M]

```
# BASE QUESTION (optimal): Big numbers: digit by digit
# from the right with a carry
# Idea: Walk both strings from the right, add the two
# digits and the carry, append total % 10, carry total //
# 10; after the loop add a final carry digit if needed and
# reverse.
def add_strings(a, b):
    # big numbers kept as digit strings, added from the
    # right
    i, j, carry, out = len(a) - 1, len(b) - 1, 0, []
    while i >= 0 or j >= 0 or carry:
        total = carry
        if i >= 0:
            total += int(a[i])
        if j >= 0:
            total += int(b[j])
        out.append(str(total % 10))
        carry = total // 10
        i, j = i - 1, j - 1
    return "".join(out[::-1])


# BASE QUESTION (optimal): Big numbers: digit by digit
# from the right with a carry
# Idea: Walk both strings from the right, add the two
# digits and the carry, append total % 10, carry total //
# 10; after the loop add a final carry digit if needed and
# reverse.
def intersection(head_a, head_b):
    # two chains meeting: each pointer walks its chain and
    # then
    # the other one, so both walk the same total length
    a, b = head_a, head_b
    while a is not b:
        a = a.next if a else head_b
        b = b.next if b else head_a
    return a


# BASE QUESTION (optimal): Big numbers: digit by digit
# from the right with a carry
# Idea: Walk both strings from the right, add the two
# digits and the carry, append total % 10, carry total //
# 10; after the loop add a final carry digit if needed and
# reverse.
def overlap_depth(rects):
    # rects (x1, y1, x2, y2), half-open: the largest
    # number of
    # rectangles sharing one point. The best point can be
    # a
    # rectangle's lower-left corner, so test those.
    best = 0
    for x, y, _, _ in rects:
        inside = [r for r in rects if r[0] <= x < r[2]
                  and r[1] <= y < r[3]]
        best = max(best, len(inside))
    return best
```

190. Check Whether Digits Are Monotone Increasing [M]

```
# BASE QUESTION (optimal): Peel digits from the right and
# compare with the previous one
# Idea: Take the last digit with n % 10, then the next
# one, and so on. Because we move right to left, the digit
# just read must not exceed the one read before it. A
# single digit and zero are monotone.
def is_monotone_digits(n):
    # non-decreasing digits, read from the right with n %
    # 10
    prev = 9
    while n:
        d = n % 10
```

```
        # a digit bigger than the one to its right: a
        # descent
        if d > prev:
            return False
        prev = d
        n //= 10
    return True
```

191. Check Digits and Combine Ranges [H]

```
# BASE QUESTION (optimal): reverse only the second half
# arithmetically
# Idea: Reject negatives and positive numbers ending in 0.
def is_palindrome(x, base=10):
    # no string: build the reversed second half
    if x < 0 or (x % base == 0 and x != 0):
        return False
    rev = 0
    # stop when the reversed half catches up with the rest
    while x > rev:
        rev = rev * base + x % base
        x //= base
    # an odd number of digits has a middle digit to drop
    return x == rev or x == rev // base
```

192. Permutation Sequence [H]

```
from math import factorial


# BASE QUESTION (optimal): Factorial number system
# Idea: Each digit is chosen by dividing the remaining
# index by the size of a block of permutations.
def get_permutation(n, k):
    digits = list(range(1, n + 1))
    k -= 1
    out = []
    for i in range(n, 0, -1):
        block = factorial(i - 1)
        out.append(digits.pop(k // block))
        k %= block
    return "".join(map(str, out))
```

GROUP 25 OF 43 | 6 QUESTIONS, ONE CODE TEMPLATE
Look at the input once and keep only a small summary (and count your calls)

193. Two-Day Log Intersection under Memory Limit [M]

```
from collections import defaultdict


# BASE QUESTION (optimal): Hash map of client sets
# (everything fits in memory)
# Idea: Build object -> set of clients for each file, keep
# the objects that appear in both, and count the union of
# their client sets.
def log_objects(day1, day2):
    # everything fits in memory: object -> set of clients
    c1, c2 = defaultdict(set), defaultdict(set)
    for _, obj, client in day1:
        c1[obj].add(client)
    for _, obj, client in day2:
        c2[obj].add(client)
    # in both files, with 2+ clients over both days
    both = c1.keys() & c2.keys()
    return sorted(o for o in both
                  if len(c1[o] | c2[o]) >= 2)
```

194. Find shared objects across two log files [M]

```
from collections import defaultdict


# BASE QUESTION (optimal): Hash set of the smaller file's
# ids
# Idea: Load the object ids of the smaller file into a
# set, then stream the other file and keep ids that are in
# the set. Each line is read once; memory depends only on
# the smaller file.
def shared_objects(day1, day2):
    # rows are [timestamp, obj_id, client_id]. Remember,
    # per
    # object, which days it appeared on and its clients.
    days, clients = defaultdict(set), defaultdict(set)
    for number, rows in ((1, day1), (2, day2)):
        for _, obj, client in rows:
            days[obj].add(number)
            clients[obj].add(client)
    return sorted(o for o in days
                  if len(days[o]) == 2
                  and len(clients[o]) >= 2)
```

195. Streaming Points: Emit K Within Distance [M]

```
from bisect import bisect_left, insort


# BASE QUESTION (optimal): Sorted list, check only windows
# touching the change
# Idea: Insert with bisect. Because no window existed
# before, a new window must contain the new point, so test
# the K windows that include its position.
class PointStream:
    def __init__(self, k, d):
        # emit when k points fit in a span of d
        self.k, self.d = k, d
        # all points still waiting, kept sorted
        self.points = []

    def window_start(self, pos):
        # first window of k neighbours that includes pos
        lo = max(0, pos - self.k + 1)
        hi = min(pos, len(self.points) - self.k)
        for i in range(lo, hi + 1):
            last = self.points[i + self.k - 1]
```

```
            span = last - self.points[i]
            if span <= self.d:
                return i
        return None

    def add(self, x):
        # insert in order, then emit every window made by
        # x
        insort(self.points, x)
        out, pos = [], bisect_left(self.points, x)
        while True:
            i = self.window_start(pos)
            if i is None:
                return out
            # remove the group so it is emitted only once
            out.append(self.points[i:i + self.k])
            del self.points[i:i + self.k]
            pos = i
```

196. Minimize calls to find all bad test pairs [M]

```
# HELPERS for the base question
def shortest_failing_prefix(items, extra, run_test):
    # binary search: the smallest prefix of `items` that
    # fails
    # together with `extra`; its last item is a partner
    lo, hi = 1, len(items)
    while lo < hi:
        mid = (lo + hi) // 2
        if run_test(set(items[:mid]) | extra):
            lo = mid + 1
        else:
            hi = mid
    return items[lo - 1]


def partners(group, t, run_test):
    # every member of the group that is bad with t
    rest, found = list(group), []
    while rest and not run_test(set(rest) | {t}):
        x = shortest_failing_prefix(rest, {t}, run_test)
        found.append(x)
        rest.remove(x)
    return found


# BASE QUESTION (optimal): Independent groups plus binary
# search for partners
# Idea: Process tests one by one. Keep groups of tests
# that pass runTest.
def find_bad_pairs(tests, run_test):
    # keep groups of tests that pass together; a new test
    # is
    # checked against each group and joins one with no
    # partner
    groups, pairs = [], set()
    for t in tests:
        placed = False
        for group in groups:
            found = partners(group, t, run_test)
            for x in found:
                pairs.add((min(x, t), max(x, t)))
            if not found and not placed:
                group.append(t)
                placed = True
        if not placed:
            groups.append([t])
    return pairs
```

197. Find All Bad Pairs Using runTest Oracle [H]

```
# HELPERS for the base question
def first_partner(group, t, run_test):
    # binary search: the shortest prefix of the group that
    # fails together with t; its last member is a partner
    lo, hi = 1, len(group)
    while lo < hi:
        mid = (lo + hi) // 2
        if run_test(set(group[:mid]) | {t}):
            lo = mid + 1
        else:
            hi = mid
    return group[lo - 1]


def partners(group, t, run_test):
    # all members of the group that are bad with t
    rest, found = list(group), []
    while rest and not run_test(set(rest) | {t}):
        x = first_partner(rest, t, run_test)
        found.append(x)
        # take it out and test the others again
        rest.remove(x)
    return found


# BASE QUESTION (optimal): Group the tests and binary
# search for partners
# Idea: Keep groups of tests that pass together.
def find_bad_pairs(items, run_test):
    # groups of items that pass together (no bad pair
    # inside)
    groups, pairs = [], set()
    for t in items:
        placed = False
        for group in groups:
            found = partners(group, t, run_test)
            for x in found:
                pairs.add((min(x, t), max(x, t)))
            # t joins the first group with no partner for
            # it
            if not found and not placed:
                group.append(t)
                placed = True
        if not placed:
            groups.append([t])
    return pairs
```

198. Find maximum value in huge text file [H]

```
# BASE QUESTION (optimal): Single machine: stream the
# file, keep one running maximum
# Idea: Never load the file. Read it through a buffered
# iterator and keep one variable, the maximum so far,
# started at None so that an all-negative file or a one-
# line file is handled correctly (do not start at 0).
def max_in_file(path):
    # one sequential pass, one number in memory
    best = None
    with open(path, "r", buffering=1 << 20) as f:
        for line in f:
            line = line.strip()
            # blank lines are ignored (state this
            # contract)
            if not line:
                continue
            value = int(line)
            # start from None, not 0: all numbers may be
            # negative
            if best is None or value > best:
                best = value
    return best
```

GROUP 26 OF 43 | 5 QUESTIONS, ONE CODE TEMPLATE
Grid DP: each cell from its neighbours above and to the left

199. Grid Path Count with Diagonal Moves and Checkpoints [M]

```
# HELPERS for the base question
def step(ways):
    # next column from the current column only
    n = len(ways)
    return [(ways[r - 1] if r else 0) + ways[r]
            + (ways[r + 1] if r + 1 < n else 0)
            for r in range(n)]


def keep_row(ways, row):
    # a checkpoint forces the path through one row
    return [x if r == row else 0
            for r, x in enumerate(ways)]


# Answer: No path can visit both, because a path is in
# exactly one row per column, so the answer is 0 (the code
# returns it before the DP). Checkpoints at the start or
# end cell are fine: they only filter rows of the first or
# last column.
def checkpoint_rows(checkpoints, ordered):
    # column -> required row; None if impossible
    rows = {}
    for r, c in checkpoints:
        # two different checkpoints in one column: no path
        if rows.setdefault(c, r) != r:
            return None
    if len(rows) != len(checkpoints):
        return None
    cols = [c for _, c in checkpoints]
    # ordered: they must be listed left to right
    if ordered and cols != sorted(cols):
        return None
    return rows


# BASE QUESTION (optimal): Column DP with a rolling array
# and checkpoint filters
# Idea: ways[r] is the number of paths that are in row r
# at the current column. The next column is ways'[r] =
# ways[r - 1] + ways[r] + ways[r + 1]. At a checkpoint
# column keep only the checkpoint's row.
def count_paths(n, m, checkpoints=(), ordered=False):
    rows = checkpoint_rows(list(checkpoints), ordered)
    if rows is None:
        return 0
    # one rolling column is enough: O(n) space
    ways = [0] * n
    ways[n - 1] = 1
    if 0 in rows:
        ways = keep_row(ways, rows[0])
    for c in range(1, m):
        ways = step(ways)
        if c in rows:
            ways = keep_row(ways, rows[c])
    return ways[n - 1]
```

200. Longest Same-Color Grid Sequence [M]

```
# BASE QUESTION (optimal): Four run lengths per cell
# Idea: Keep, for the previous row, the run length ending
# at each column in the vertical and both diagonal
# directions, and a running value for the horizontal run
# in the current row.
def longest_same_color_line(g):
    if not g or not g[0]:
        return 0
    n, best = len(g[0]), 0
    # run lengths ending in the previous row:
    # straight down, down-left, down-right
    up, dl, dr = [0] * n, [0] * n, [0] * n
    for i, row in enumerate(g):
        nu, nl, nr = [1] * n, [1] * n, [1] * n
        left = 0
        for j, color in enumerate(row):
            # horizontal run from the cell on the left
            same = j and row[j - 1] == color
            left = left + 1 if same else 1
            if i:
                # extend the run from the cell above
                if g[i - 1][j] == color:
                    nu[j] = up[j] + 1
                # from above-left (down-right diagonal)
                if j and g[i - 1][j - 1] == color:
                    nr[j] = dr[j - 1] + 1
                # from above-right (down-left diagonal)
                if j + 1 < n and g[i - 1][j + 1] == color:
                    nl[j] = dl[j + 1] + 1
            best = max(best, left, nu[j], nl[j], nr[j])
        up, dl, dr = nu, nl, nr
    return best
```

201. Count Same-Color Squares in Matrix [M]

```
# HELPERS for the base question
def square_sides(g):
    # side[i][j]: the biggest same-colour square whose
    # bottom-right corner is (i, j); one row at a time
    if not g:
        return []
    prev, out = [0] * len(g[0]), []
    for i, row in enumerate(g):
        cur = [1] * len(row)
        for j, c in enumerate(row):
            # all three neighbours must match this cell
            if i and j and g[i - 1][j] == c \
                    and g[i][j - 1] == c \
                    and g[i - 1][j - 1] == c:
                cur[j] = 1 + min(prev[j], cur[j - 1],
                                 prev[j - 1])
        out.append(cur)
        prev = cur
    return out


# BASE QUESTION (optimal): dp on the bottom-right corner
# Idea: dp[j] holds the best square side ending in the
# current row at column j.
def count_same_color_squares(g):
    # every cell contributes one square per side up to its
    # side
    return sum(sum(row) for row in square_sides(g))
```

202. Find the Longest Straight Run of One Color [H]

```
# HELPERS for the base question
DIRECTIONS = ((0, 1), (1, 0), (1, 1), (1, -1))


def runs(grid):
    # yield (length, start, direction) for every maximal
    # run;
    # a run is walked only from its first cell, so the
    # total
    # work is linear in the number of cells
    rows, cols = len(grid), len(grid[0])
    for r in range(rows):
        for c in range(cols):
            for dr, dc in DIRECTIONS:
                pr, pc = r - dr, c - dc
                before = 0 <= pr < rows and 0 <= pc < cols
                if before and grid[pr][pc] == grid[r][c]:
                    # not the first cell of its run
                    continue
                n, a, b = 0, r, c
                while 0 <= a < rows and 0 <= b < cols \
                        and grid[a][b] == grid[r][c]:
                    n += 1
                    a, b = a + dr, b + dc
                yield n, (r, c), (dr, dc)


# BASE QUESTION (optimal): DP per direction with a rolling
# row
# Idea: run[d][i][j] is the length of the same-color run
# ending at (i, j) in direction d. It is the previous
# cell's run plus one when the colors match, else 1.
def longest_run(grid):
    if not grid or not grid[0]:
        return 0
    return max(n for n, _, _ in runs(grid))
```

203. Count same-color squares in a character grid [H]

```
# HELPERS for the base question
def square_table(grid):
    # side[i][j]: the biggest same-colour square with its
    # bottom-right corner at (i, j)
    prev, rows = [0] * len(grid[0]), []
    for i, row in enumerate(grid):
        cur = [1] * len(row)
        for j, ch in enumerate(row):
            # the up, left and up-left cells must match
            # too
            if i and j and ch == grid[i - 1][j] \
                    and ch == grid[i][j - 1] \
                    and ch == grid[i - 1][j - 1]:
                cur[j] = 1 + min(prev[j], cur[j - 1],
                                 prev[j - 1])
        rows.append(cur)
        prev = cur
    return rows


# BASE QUESTION (optimal): DP: side of the largest square
# ending at each cell, summed
# Idea: If the cell matches its top, left and top-left
# neighbours then dp = 1 + min(dp top, dp left, dp top-
# left); otherwise 1. Summing dp over all cells counts
# every square exactly once, at its bottom-right corner.
def count_same_color_squares(grid):
    # each cell adds one square for every size up to its
    # side
    return sum(sum(row) for row in square_table(grid))
```

GROUP 27 OF 43 | 5 QUESTIONS, ONE CODE TEMPLATE
Nested structures: a stack of contexts, or recursion with a cache

204. Decode String [M]

```
# BASE QUESTION (optimal): Stack of (prefix, repeat)
# frames
# Idea: On '[' push the text built so far and the repeat
# count, then start fresh. On ']' pop and append the
# current text repeated count times to the saved prefix.
def decode(s):
    # stack of (text so far, repeat count) for each open [
    stack, cur, num = [], [], 0
    for ch in s:
        if ch.isdigit():
            num = num * 10 + int(ch)
        elif ch == "[":
            # remember what we have, start the inner text
            stack.append((cur, num))
            cur, num = [], 0
        elif ch == "]":
            prev, times = stack.pop()
            prev.append("".join(cur) * times)
            cur = prev
        else:
            cur.append(ch)
    return "".join(cur)
```

205. Template Variable Expansion with Follow-Ups [M]

```
# HELPERS for the base question
def tokens(s):
    # split text into ("text", char) and ("var", name)
    i = 0
    while i < len(s):
        if s[i] != "%":
            yield "text", s[i]
            i += 1
        elif s[i:i + 2] == "%%":
            # %% is a literal percent sign
            yield "text", "%"
            i += 2
        else:
            j = s.find("%", i + 1)
            if j < 0:
                raise ValueError("unmatched %")
            yield "var", s[i + 1:j]
            i = j + 1


# BASE QUESTION (optimal): Depth-first expansion with memo
# and a visiting set
# Idea: expand(text) scans the text once: %% becomes %,
# %NAME% becomes the resolved value of NAME.
def expand_template(text, mapping):
    done, visiting = {}, set()

    def resolve(name):
        # values may hold placeholders: expand them first
        if name in done:
            return done[name]
        if name in visiting:
            raise ValueError("cycle at " + name)
        if name not in mapping:
            raise KeyError(name)
        visiting.add(name)
        done[name] = scan(mapping[name])
        visiting.discard(name)
        return done[name]

    def scan(s):
        return "".join(resolve(x) if kind == "var" else x
```

```
                    for kind, x in tokens(s))

    return scan(text)
```

206. Expand Recursive Key-Value Placeholders in a Template [M]

```
import re


# HELPERS for the base question
PATTERN = re.compile(r"%%|%(\w+)%")


# BASE QUESTION (optimal): Memoised recursive expansion
# with a visiting set
# Idea: expand(key) substitutes every placeholder in the
# key's value by expand(inner), caches the result, and
# uses a visiting set to raise on a cycle. Unknown keys
# stay as written.
# Answer: Choose an escape such as %% and handle it in the
# same scan: when the text has %%, output one % and skip
# both characters; otherwise try to match a placeholder.
# Test percent signs next to placeholders.
def expand(template, mapping, unknown="keep",
           limit=10 ** 6):
    # unknown: "keep" the text, "empty" it, or "error"
    cache, visiting = {}, set()

    def expand_text(text):
        def one(m):
            key = m.group(1)
            # FOLLOW-UP 1: %% is a literal percent sign
            if key is None:
                # %% is a literal percent sign
                return "%"
            if key not in mapping:
                # FOLLOW-UP 2: unknown key: keep, empty or
                # error
                if unknown == "error":
                    raise KeyError(key)
                if unknown == "empty":
                    return ""
                return m.group(0)
            return expand_key(key)
        out = PATTERN.sub(one, text)
        # FOLLOW-UP 3: huge values: cap the expansion
        # stop explosive expansions early
        if len(out) > limit:
            raise ValueError("expansion too large")
        return out

    def expand_key(key):
        # each key is expanded once and then cached
        if key in cache:
            return cache[key]
        if key in visiting:
            raise ValueError(f"cycle at {key}")
        visiting.add(key)
        cache[key] = expand_text(mapping[key])
        visiting.discard(key)
        return cache[key]

    return expand_text(template)
```

207. Nested List Weight Sum (LC 339) with Follow-Ups [M]

```
# HELPERS for the base question
def with_depths(nested):
    # every integer with its depth (top level = 1),
    # using a stack so deep input cannot overflow
    stack = [(nested, 1)]
    while stack:
        item, depth = stack.pop()
        if isinstance(item, list):
            for child in item:
                stack.append((child, depth + 1))
        else:
            yield item, depth - 1


# BASE QUESTION (optimal): Single pass over the string
# with a depth counter
# Idea: Every integer is weighted by the number of '['
# currently open, so no tree needs to be built. Accumulate
# digits in a buffer and flush the buffer when a non-digit
# arrives or the string ends.
def weight_sum(nested):
    # deeper integers weigh more (LeetCode 339)
    return sum(v * d for v, d in with_depths(nested))


# BASE QUESTION (optimal): Single pass over the string
# with a depth counter
# Idea: Every integer is weighted by the number of '['
# currently open, so no tree needs to be built. Accumulate
# digits in a buffer and flush the buffer when a non-digit
# arrives or the string ends.
def weight_sum_string(s):
    # one pass over the text, O(1) extra space
    depth = total = 0
    num, sign, have = 0, 1, False
    # a closing sentinel flushes a trailing number
    for ch in s + "]":
        if ch.isdigit():
            num, have = num * 10 + int(ch), True
            continue
        # any other character ends the number being read
        if have:
            total += sign * num * depth
            num, sign, have = 0, 1, False
        if ch == "[":
            depth += 1
        elif ch == "]":
            depth -= 1
        elif ch == "-":
            sign = -1
    return total
```

208. Nested Function Expression Evaluator [M]

```
import math


import re


# HELPERS for the base question
TOKEN = re.compile(r"-?\d+|[A-Za-z_]\w*|[(),]")


# Answer: State the contract: integer or float, truncate
# toward zero or floor (Python's // floors, which differs
# for negatives), and divide by zero handling. The
# implementation above truncates toward zero.
def divide(a, b):
    # integer division that truncates toward zero
    if b == 0:
        raise ZeroDivisionError("division by zero")
```

```
    return int(a / b)


OPS = {
    "add": lambda *a: sum(a),
    "sub": lambda a, b: a - b,
    "mul": lambda *a: math.prod(a),
    "pow": lambda a, b: a ** b,
    "div": divide,
    "min": min,
    "max": max,
}


# BASE QUESTION (optimal): Tokenizer + recursive descent
# over name(arg, arg, ...)
# Idea: Tokenize into identifiers, integers, commas and
# parentheses.
def evaluate(expr, env=None):
    env = env or {}
    tokens = TOKEN.findall(expr)

    def parse(i):
        # returns (value, index after this expression)
        tok = tokens[i]
        if re.fullmatch(r"-?\d+", tok):
            return int(tok), i + 1
        # FOLLOW-UP 4: variables come from env
        # a name not followed by "(" is a variable
        if i + 1 >= len(tokens) or tokens[i + 1] != "(":
            if tok not in env:
                raise ValueError(f"undefined {tok!r}")
            return env[tok], i + 1
        if tok not in OPS:
            raise ValueError(f"unknown function {tok!r}")
        i += 2
        args = []
        # read arguments until the closing bracket
        while tokens[i] != ")":
            value, i = parse(i)
            args.append(value)
            if tokens[i] == ",":
                i += 1
        return OPS[tok](*args), i + 1

    value, end = parse(0)
    if end != len(tokens):
        raise ValueError("trailing tokens")
    return value
```

GROUP 28 OF 43 | 5 QUESTIONS, ONE CODE TEMPLATE
Frontier search (BFS or DFS), Tier 2: free steps, early exit, neighbours worked out on the fly

209. Shortest Path Nodes and Escaping a Pursuer [M]

```
from collections import deque


# HELPERS used by the code below
def bfs(adj, s):
    # distance from s to every reachable node
    d = {s: 0}
    q = deque([s])
    while q:
        u = q.popleft()
        for v in adj[u]:
            if v not in d:
                d[v] = d[u] + 1
                q.append(v)
    return d


# HELPERS for the base question
FAR = 10 ** 9


# BASE QUESTION (optimal): Three BFS runs and a safe
# shortest-path DAG
# Idea: dA, dT and dB come from BFS. The nodes on some
# shortest path are those with dA[v] + dT[v] == dA[t].
def shortest_length(adj, a, t):
    # the base question: edges on the shortest path
    return bfs(adj, a).get(t)
```

210. Radio Broadcast Reachability [M]

```
from collections import deque


# BASE QUESTION (optimal): BFS over the implicit graph
# Idea: Compare squared distances; every radio enters the
# queue once.
def can_reach(radios, radius, source, target):
    limit = radius * radius
    seen = [False] * len(radios)
    seen[source] = True
    queue = deque([source])
    while queue:
        u = queue.popleft()
        if u == target:
            return True
        for v, (x, y) in enumerate(radios):
            dx, dy = radios[u][0] - x, radios[u][1] - y
            if (
                not seen[v]
                and dx * dx + dy * dy <= limit
            ):
                seen[v] = True
                queue.append(v)
    return False
```

211. Minimum Direction Violations in a Directed Graph [M]

```
from collections import deque


# HELPERS used by the code below
INF = float("inf")


# BASE QUESTION (optimal): 0-1 BFS on the doubled graph
# Idea: Add every edge twice: forward with weight 0,
# backward with weight 1. Shortest path with weights in
# {0, 1} is a deque BFS: push a zero-weight neighbour to
# the front and a one-weight neighbour to the back.
def min_violations(n, edges, start, end):
    # 0-1 BFS: following an edge is free, reversing costs
    # 1
    graph = [[] for _ in range(n)]
    for u, v in edges:
        graph[u].append((v, 0))
        graph[v].append((u, 1))
    dist = [INF] * n
    dist[start] = 0
    dq = deque([start])
    while dq:
        u = dq.popleft()
        for v, w in graph[u]:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                # cost 0 goes to the front, cost 1 to the
                # back
                if w == 0:
                    dq.appendleft(v)
                else:
                    dq.append(v)
    return -1 if dist[end] == INF else dist[end]
```

212. Compute shortest paths with blocked nodes [M]

```
from collections import defaultdict, deque


# BASE QUESTION (optimal): BFS that never enters a blocked
# node
# Idea: Build adjacency lists (one direction only if
# directed). If the source is blocked return all -1.
def blocked_distances(n, edges, source, blocked,
                      directed=False):
    # BFS that treats blocked nodes as removed;
    # unreachable
    # and blocked nodes get -1
    bad = set(blocked)
    if source in bad:
        return [-1] * n
    adj = defaultdict(list)
    for u, v in edges:
        adj[u].append(v)
        if not directed:
            adj[v].append(u)
    dist = [-1] * n
    dist[source] = 0
    queue = deque([source])
    while queue:
        u = queue.popleft()
        for v in adj[u]:
            if v not in bad and dist[v] == -1:
                dist[v] = dist[u] + 1
                queue.append(v)
    return dist
```

213. Shortest Directed Cycle Through a Node [M]

```
from collections import deque


# HELPERS for the base question
def make_graph(n, edges):
    adj = [[] for _ in range(n)]
    for a, b, *w in edges:
        # edges may carry a weight (default 1)
        adj[a].append((b, w[0] if w else 1))
    return adj


# BASE QUESTION (optimal): BFS from v, close the cycle on
# the first edge back to v
# Idea: A directed cycle through v is a path v to u
# followed by an edge u to v. BFS visits nodes in
# nondecreasing distance, so the first time we see an edge
# back to v its length is dist[u] + 1 and is the shortest.
def shortest_cycle_through(n, edges, v):
    # BFS from v: the first edge that comes back to v
    # closes
    # the shortest cycle through v
    adj = make_graph(n, edges)
    dist, queue = {v: 0}, deque([v])
    while queue:
        u = queue.popleft()
        for w, _ in adj[u]:
            if w == v:
                return dist[u] + 1
            if w not in dist:
                dist[w] = dist[u] + 1
                queue.append(w)
    return -1
```

GROUP 29 OF 43 | 5 QUESTIONS, ONE CODE TEMPLATE
Topological sort (Kahn) and cycle detection

214. Course Scheduling and Dependency Variants [M]

```
from collections import defaultdict, deque


# HELPERS for the base question
def edges_from_sequences(*sequences):
    # each neighbouring pair in a sequence is a rule
    return [(s[i], s[i + 1]) for s in sequences
            for i in range(len(s) - 1)]


def merge_two_orders(a, b):
    # shared values are the same vertex
    edges = edges_from_sequences(a, b)
    # a cycle (conflict) gives None
    return topological_order(set(a) | set(b), edges)


# BASE QUESTION (optimal): Kahn's algorithm (course order,
# or can two sequences merge)
# Idea: Build the graph (prerequisite -> dependent,
# including isolated vertices), count indegrees and keep a
# queue of vertices with indegree 0. Pop one, append it to
# the order and lower the indegree of its dependents.
def topological_order(vertices, edges):
    # edge (u, v) means u must come before v
    adj = defaultdict(list)
    indeg = {v: 0 for v in vertices}
    for u, v in edges:
        adj[u].append(v)
        # count unmet prerequisites of v
        indeg[v] = indeg.get(v, 0) + 1
        indeg.setdefault(u, 0)
    # start with everything that has no prerequisite
    queue = deque(v for v, d in indeg.items() if d == 0)
    order = []
    while queue:
        u = queue.popleft()
        order.append(u)
        # finishing u frees its dependents
        for v in adj[u]:
            indeg[v] -= 1
            if indeg[v] == 0:
                queue.append(v)
    # fewer than all vertices means a cycle: no order
    return order if len(order) == len(indeg) else None


# BASE QUESTION (optimal): Kahn's algorithm (course order,
# or can two sequences merge)
# Idea: Build the graph (prerequisite -> dependent,
# including isolated vertices), count indegrees and keep a
# queue of vertices with indegree 0. Pop one, append it to
# the order and lower the indegree of its dependents.
def can_merge(a, b):
    # True when some order keeps both sequences in order
    return merge_two_orders(a, b) is not None
```

215. Validate Race Results (Cycle Detection) [M]

```
from collections import deque


# BASE QUESTION (optimal): Kahn's algorithm
# Idea: Peel off nodes with no incoming edge; everything
# peels off exactly when there is no cycle.
def results_are_valid(results):
    graph, indeg = {}, {}
    for w, l in set(results):
        graph.setdefault(w, []).append(l)
        graph.setdefault(l, [])
        indeg[l] = indeg.get(l, 0) + 1
        indeg.setdefault(w, 0)
    queue = deque(u for u in graph if indeg[u] == 0)
    removed = 0
    while queue:
        u = queue.popleft()
        removed += 1
        for v in graph[u]:
            indeg[v] -= 1
            if indeg[v] == 0:
                queue.append(v)
    return removed == len(graph)
```

216. Solve tree, graph, sliding-window problems [M]

```
from collections import defaultdict, deque


# HELPERS for the base question
# Answer: The same Kahn loop records the order in which
# courses are removed; if fewer than n courses are
# removed, return an empty list.
def course_order(n, prereqs):
    # (a, b): to take a you must take b first. Kahn's
    # algorithm;
    # if some course never frees up there is a cycle
    adj, indeg = defaultdict(list), [0] * n
    for a, b in prereqs:
        adj[b].append(a)
        indeg[a] += 1
    queue = deque(c for c in range(n) if indeg[c] == 0)
    order = []
    while queue:
        c = queue.popleft()
        order.append(c)
        for nxt in adj[c]:
            indeg[nxt] -= 1
            if indeg[nxt] == 0:
                queue.append(nxt)
    return order if len(order) == n else []


# BASE QUESTION (optimal): Course schedule: Kahn's
# topological sort
# Idea: Count prerequisites per course and start from
# courses with none. Remove a course, decrease the count
# of the courses that depend on it, and queue those that
# reach zero.
def zigzag(root):
    # level order, alternating direction
    out, left_to_right = [], True
    queue = deque([root] if root else [])
    while queue:
        level = deque()
        for _ in range(len(queue)):
            node = queue.popleft()
            # append for left-to-right, appendleft
            # otherwise
            if left_to_right:
                level.append(node.val)
            else:
```

```
                level.appendleft(node.val)
            for child in (node.left, node.right):
                if child:
                    queue.append(child)
        out.append(list(level))
        left_to_right = not left_to_right
    return out


# BASE QUESTION (optimal): Course schedule: Kahn's
# topological sort
# Idea: Count prerequisites per course and start from
# courses with none. Remove a course, decrease the count
# of the courses that depend on it, and queue those that
# reach zero.
def can_finish(n, prereqs):
    return len(course_order(n, prereqs)) == n


# BASE QUESTION (optimal): Course schedule: Kahn's
# topological sort
# Idea: Count prerequisites per course and start from
# courses with none. Remove a course, decrease the count
# of the courses that depend on it, and queue those that
# reach zero.
def longest_at_most_k(s, k):
    # sliding window with a count of each letter inside it
    count, left, best = defaultdict(int), 0, 0
    for right, ch in enumerate(s):
        count[ch] += 1
        # too many distinct letters: shrink from the left
        while len(count) > k:
            count[s[left]] -= 1
            if count[s[left]] == 0:
                del count[s[left]]
            left += 1
        best = max(best, right - left + 1)
    return best
```

217. Validate course catalog dependencies [M]

```
from collections import defaultdict, deque


# HELPERS for the base question
# Answer: Compute strongly connected components (Tarjan or
# Kosaraju); every component with more than one course (or
# a self-loop) is a cycle group to report.
def find_cycle(known, prereqs, indeg):
    # every leftover course still has a leftover
    # prerequisite,
    # so walking backwards must eventually repeat a course
    left = {c for c in known if indeg[c] > 0}
    before = defaultdict(list)
    for u, v in prereqs:
        if u in left and v in left:
            before[v].append(u)
    x = min(left, key=str)
    walk, pos = [], {}
    while x not in pos:
        pos[x] = len(walk)
        walk.append(x)
        x = before[x][0]
    cycle = walk[pos[x]:] + [x]
    # walked against the edges: reverse to list
    # prerequisites first
    return cycle[::-1]


# BASE QUESTION (optimal): Missing references, then Kahn's
# algorithm, then a cycle from the leftovers
# Idea: Report unknown courses (any referenced id not in
# the set). If none, run Kahn's algorithm: repeatedly
```

```
# remove courses with no remaining prerequisites, building
# an order.
# Answer: The function above returns the sorted unknown
# ids when any exist and otherwise a cycle (a list of
# courses starting and ending at the same course), found
# by walking backwards along prerequisites inside the set
# of courses Kahn's algorithm could not remove (every such
# course has a leftover prerequisite, so the walk cannot
# dead-end).
def validate_catalog(courses, prereqs):
    # prereqs (u, v): course v requires course u.
    # Returns (ok, missing ids, order or one cycle)
    known = set(courses)
    missing = sorted({x for pair in prereqs for x in pair
                      if x not in known}, key=str)
    if missing:
        return False, missing, None
    adj, indeg = defaultdict(list), {c: 0 for c in known}
    for u, v in prereqs:
        adj[u].append(v)
        indeg[v] += 1
    ready = [c for c in known if indeg[c] == 0]
    queue = deque(sorted(ready, key=str))
    order = []
    while queue:
        c = queue.popleft()
        order.append(c)
        for nxt in adj[c]:
            indeg[nxt] -= 1
            if indeg[nxt] == 0:
                queue.append(nxt)
    if len(order) == len(known):
        return True, [], order
    return False, [], find_cycle(known, prereqs, indeg)
```

218. Course Schedule [M]

```
from collections import defaultdict, deque


# HELPERS for the base question
def build(n, prereqs):
    # (a, b): course b must be taken before course a
    adj, indeg = defaultdict(list), [0] * n
    for a, b in prereqs:
        adj[b].append(a)
        indeg[a] += 1
    return adj, indeg


# Answer: The same Kahn loop records the order; if fewer
# than n courses are taken return an empty list. O(V + E).
def find_order(n, prereqs):
    # Kahn's algorithm: take courses with no unmet
    # prerequisite
    adj, indeg = build(n, prereqs)
    queue = deque(c for c in range(n) if indeg[c] == 0)
    order = []
    while queue:
        c = queue.popleft()
        order.append(c)
        for nxt in adj[c]:
            indeg[nxt] -= 1
            if indeg[nxt] == 0:
                queue.append(nxt)
    # fewer than n taken: a cycle blocks the rest
    return order if len(order) == n else []


# BASE QUESTION (optimal): Kahn's algorithm (topological
# sort)
# Idea: Count each course's unmet prerequisites and queue
# those with none. Take a course, reduce the counts of the
```

```
# courses that need it, and queue any that reach zero.
def can_finish(n, prereqs):
    return len(find_order(n, prereqs)) == n
```

GROUP 30 OF 43 | 5 QUESTIONS, ONE CODE TEMPLATE
Counting pairs by looking up each partner in a Counter or a bucket

219. Progressive Array Partitioning [M]

```
# BASE QUESTION (optimal): Counter, then compare the set
# of counts with the number of values
# Idea: All counts are different exactly when the set of
# counts has as many elements as there are distinct
# values. One pass to count, one pass over the counts.
def stage1(counts):
    # true when every value occurs a different number of
    # times: the counts have no repeats
    return len(set(counts.values())) == len(counts)
```

220. K-diff Pairs in an Array [M]

```
from collections import Counter


# BASE QUESTION (optimal): Count the values
# Idea: A value v starts a pair when v + k is present; for
# k = 0 it must appear twice.
def count_k_diff_pairs(nums, k):
    if k < 0:
        return 0
    counts = Counter(nums)
    if k == 0:
        return sum(1 for c in counts.values() if c >= 2)
    return sum(1 for v in counts if v + k in counts)
```

221. Count Axis-Aligned Squares Among Grid Points [M]

```
# BASE QUESTION (optimal): Opposite corners and a set
# Idea: Two opposite corners decide the other two, so one
# set lookup per pair is enough.
def count_axis_aligned_squares(points):
    seen = set(points)
    total = 0
    for i, (x1, y1) in enumerate(points):
        for x2, y2 in points[i + 1 :]:
            if abs(x1 - x2) == abs(y1 - y2) != 0:
                if (x1, y2) in seen and (
                    x2,
                    y1,
                ) in seen:
                    total += 1
    return total // 2
```

222. Sum of Two Squares in Two Ways [M]

```
from collections import Counter


# BASE QUESTION (optimal): Count every pair once in a hash
# map
# Idea: Loop a from 1 while 2a^2 < n and b from a while
# a^2 + b^2 < n, adding each sum to a Counter. There are
# about pi * n / 8 pairs. The answer is the sorted list of
# sums whose count is exactly 2.
# Answer: Change the comparison from == 2 to == k; the
# counting loop is the same. For k = 3 the smallest is
# 325.
# Answer: Start a at 0; then perfect squares like 25 gain
# a representation, so some answers appear or disappear.
# Check against the examples to see which convention the
# question uses (here, positive only).
def sums_in_k_ways(n, k=2, zero=False):
    # every s < n is counted once per pair a <= b with
    # a*a + b*b = s. Keep the sums seen exactly k times.
    ways = Counter()
    # FOLLOW-UP 2: zero=True allows 0^2 + b^2
```

```
    a = 0 if zero else 1
    while 2 * a * a < n:
        b = a
        while a * a + b * b < n:
            ways[a * a + b * b] += 1
            b += 1
        a += 1
    # FOLLOW-UP 1: k ways instead of exactly two
    return sorted(s for s, c in ways.items()
                  if c == k and s > 0)
```

223. Maximum-Area Rectangle from Planar Points [H]

```
from collections import defaultdict


from itertools import combinations


# HELPERS used by the code below
def cross(o, p, q):
    # twice the signed area of triangle o, p, q
    ax, ay = p[0] - o[0], p[1] - o[1]
    bx, by = q[0] - o[0], q[1] - o[1]
    return ax * by - ay * bx


# HELPERS for the base question
# Answer: Not in general: the answer can depend on all
# pairs, and even deciding whether any rectangle exists is
# believed to need about n^2 work.
def all_rectangle_areas(points):
    # duplicates cannot help, so drop them
    pts = list(set(map(tuple, points)))
    buckets = defaultdict(list)
    # every pair is a possible diagonal
    for p, q in combinations(pts, 2):
        # same centre (sum of corners) and same length
        key = (p[0] + q[0], p[1] + q[1],
               (p[0] - q[0]) ** 2 + (p[1] - q[1]) ** 2)
        buckets[key].append((p, q))
    # two diagonals in one bucket form a rectangle
    for pairs in buckets.values():
        for (p, q), (r, s) in combinations(pairs, 2):
            # area = |cross| of two sides leaving p
            yield abs(cross(p, r, s))


# BASE QUESTION (optimal): Bucket pairs by diagonal
# midpoint and length
# Idea: For every pair (p, q) use the key (p.x + q.x, p.y
# + q.y, squared length). All pairs with the same key are
# diagonals of one circle around the same centre, so any
# two of them form a rectangle.
def max_rect_area(points):
    # best area, or 0 when no rectangle exists
    return max(all_rectangle_areas(points), default=0)
```

GROUP 31 OF 43 | 5 QUESTIONS, ONE CODE TEMPLATE
Tree DP: build a parent order once, then add up from the leaves

224. Compute File System Total Size [M]

```
# HELPERS used by the code below
class File:
    def __init__(self, size):
        self.size, self.parent = size, None


# BASE QUESTION (optimal): Iterative traversal with a
# visited set
# Idea: An explicit stack removes the recursion limit, and
# a set of visited node ids makes each node count once, so
# shared directories are not double counted and cycles
# terminate.
# Answer: Count a file once per identity (a visited set of
# ids, as coded) if the question asks for disk usage, but
# once per path if it asks for the sum of the sizes shown.
# Ask which.
def total_size(root, count_once=True):
    # iterative walk; count_once handles hard links
    total, seen, stack = 0, set(), [root]
    while stack:
        node = stack.pop()
        # FOLLOW-UP 2: hard links: count a shared file
        # once
        if count_once:
            if id(node) in seen:
                continue
            seen.add(id(node))
        if isinstance(node, File):
            total += node.size
        else:
            stack.extend(node.children)
    return total
```

225. Minimum-Cost Tree Disconnection [M]

```
from collections import defaultdict


# HELPERS for the base question
INF = float("inf")


def tree_costs(n, edges):
    # edges are (parent, child, cost); node 0 is the root
    kids = defaultdict(list)
    for u, v, w in edges:
        kids[u].append((v, w))
    # FOLLOW-UP 2: a loop, not recursion: safe on a long
    # chain
    # breadth-first order: parents come before children
    order = [0]
    for u in order:
        for v, _ in kids[u]:
            order.append(v)
    # best[v]: cheapest way to cut v off from its leaves
    # free[v]: sum of every negative cost below v
    best, free = [0] * n, [0] * n
    for v in reversed(order):
        for c, w in kids[v]:
            # 1) cut this edge and the negatives below
            cut = w + free[c]
            # 2) keep it and cut deeper (a leaf cannot)
            keep = best[c] if kids[c] else INF
            best[v] += min(cut, keep)
            free[v] += min(0, w) + free[c]
    return kids, best, free
```

```
# BASE QUESTION (optimal): Tree DP: best(v) = cheapest
# cuts inside v's subtree that disconnect v from its
# leaves
# Idea: For a child c with edge weight w there are two
# options: cut the edge, paying w plus whatever extra
# negative-weight edges you can still cut below c (free(c)
# = sum of min(0, weight) over its subtree), or keep the
# edge and require best(c), which is impossible when c is
# a leaf.
# Answer: Then free(c) is always 0 and the recurrence
# collapses to best(v) = sum over children of (w if the
# child is a leaf, else min(w, best(child))).
# Answer: Compute nodes in reverse BFS order (children
# before parents) and store best and free in arrays, no
# recursion needed. State the space as O(n).
def disconnect_cost(n, edges):
    # works for negative costs and needs no recursion
    return tree_costs(n, edges)[1][0]
```

226. Given a large n-ary tree where the root node can pass information to one child per iteration, and in subsequent iterations, nodes can pass information to one child while the root continues with others, find the minimum number of iterations to reach all nodes. [M]

```
# HELPERS used by the code below
def order_children(tree, node):
    # children of `node` as (child, cost) pairs
    return tree.get(node, [])


# HELPERS for the base question
# Answer: Replace the unit step by the cost: the child
# served in order position i can only start after the sum
# of the costs of the edges before it, so T(v) = max over
# i of (prefix cost up to child_i + T(child_i)).
# Answer: The order still matters, and the right rule is
# Jackson's: serve children in decreasing order of their
# own finishing time T, whatever each call costs, because
# the finish of the child served at prefix cost P is P + T
# and the largest tail should start earliest.
def finish_times(tree, root):
    # time[v] = rounds until v and everything below has
    # the
    # news, once v has it. v calls its children one after
    # another; the call to the i-th child costs the edges
    # so
    # far, and that child then needs time[child] more.
    # Best order (Jackson's rule): the child with the
    # LARGEST time[child] first.
    order, stack = [], [root]
    while stack:
        node = stack.pop()
        order.append(node)
        kids = order_children(tree, node)
        stack.extend(c for c, _ in kids)
    time, plan = {}, {}
    # process children before parents
    for node in reversed(order):
        kids = sorted(order_children(tree, node),
                      key=lambda kc: -time[kc[0]])
        spent, worst = 0, 0
        for child, cost in kids:
            spent += cost
            worst = max(worst, spent + time[child])
        time[node] = worst
        plan[node] = [c for c, _ in kids]
    return time, plan


# BASE QUESTION (optimal): Post-order DP: serve the
# slowest subtree first
# Idea: Let T(v) be the rounds needed to inform the
# subtree of v once v knows. v informs its children one
# per round.
```

```
def min_rounds(tree, root):
    return finish_times(tree, root)[0][root]
```

227. Compute distance-sum from every tree node [H]

```
# HELPERS for the base question
def build(n, edges):
    adj = [[] for _ in range(n)]
    for e in edges:
        u, v = e[0], e[1]
        w = e[2] if len(e) > 2 else 1
        adj[u].append((v, w))
        adj[v].append((u, w))
    return adj


def distances_from(adj, s):
    # distance from s to every node (a tree: one pass)
    dist, stack = {s: 0}, [s]
    while stack:
        u = stack.pop()
        for v, w in adj[u]:
            if v not in dist:
                dist[v] = dist[u] + w
                stack.append(v)
    return dist


# BASE QUESTION (optimal): Re-rooting with subtree sizes
# Idea: Root the tree at 0 and compute size[v] (subtree
# sizes) and ans[0] = the sum of depths.
def distance_sums(n, edges):
    # sum of distances from every node; edges may be
    # weighted
    adj = build(n, edges)
    parent, order, up = [-1] * n, [0], [0] * n
    for u in order:
        for v, w in adj[u]:
            if v != parent[u]:
                parent[v], up[v] = u, w
                order.append(v)
    size = [1] * n
    for u in reversed(order):
        if parent[u] >= 0:
            size[parent[u]] += size[u]
    # the root's sum is a plain sum of distances
    ans = [0] * n
    ans[0] = sum(distances_from(adj, 0).values())
    # root moves across an edge of weight w: size[u] nodes
    # get
    # w closer, the other n - size[u] get w farther
    for u in order[1:]:
        shift = up[u] * (n - 2 * size[u])
        ans[u] = ans[parent[u]] + shift
    return ans
```

228. Tree Distance Sum (Re-root DP) [H]

```
# HELPERS for the base question
def build(n, edges):
    adj = [[] for _ in range(n)]
    for a, b in edges:
        adj[a].append(b)
        adj[b].append(a)
    return adj


# BASE QUESTION (optimal): Two passes: subtree sizes, then
# re-root
# Idea: Root the tree at 0 and get a BFS order. In reverse
# order compute size[u] and the sum of distances inside
# u's subtree. For the root that sum is already the
# answer.
def tree_distance_sums(n, edges):
```

```
if n == 0:
    return []
adj = build(n, edges)
# order: parents before children (no recursion)
parent, order = [-1] * n, [0]
for u in order:
    for v in adj[u]:
        if v != parent[u]:
            parent[v] = u
            order.append(v)
# size[u]: nodes in u's subtree
# inside[u]: distances from u to its own subtree
size, inside = [1] * n, [0] * n
for u in reversed(order):
    p = parent[u]
    if p >= 0:
        size[p] += size[u]
        inside[p] += inside[u] + size[u]
# the root's answer is already complete
ans = [0] * n
ans[0] = inside[0]
# moving the root from p to child u: the size[u] nodes
# below get 1 closer, the other n - size[u] get 1
# farther
for u in order[1:]:
    ans[u] = ans[parent[u]] + n - 2 * size[u]
return ans
```

GROUP 32 OF 43 | 5 QUESTIONS, ONE CODE TEMPLATE
Counting or reaching an amount: the knapsack loop

229. Maximize Shelf Space (Unbounded Knapsack) [M]

```
# BASE QUESTION (optimal): Reachable totals DP
# Idea: A total is reachable if removing one box leaves a
# reachable total.
def max_shelf_space(capacity, sizes):
    reachable = [True] + [False] * capacity
    for c in range(1, capacity + 1):
        reachable[c] = any(
            s <= c and reachable[c - s] for s in sizes
        )
    return max(
        c for c in range(capacity + 1) if reachable[c]
    )
```

230. Coin Change [M]

```
# HELPERS for the base question
INF = float("inf")


# BASE QUESTION (optimal): Bottom-up DP over amounts
# Idea: best[0] = 0, and for a from 1 to amount, best[a] =
# 1 + min(best[a - c]) over coins c <= a (infinity if none
# works). The answer is best[amount] or -1. Each amount is
# solved once.
def coin_change(coins, amount):
    # best[a] = fewest coins that make exactly a
    best = [0] + [INF] * amount
    for a in range(1, amount + 1):
        for c in coins:
            if c <= a and best[a - c] + 1 < best[a]:
                best[a] = best[a - c] + 1
    return -1 if best[amount] == INF else best[amount]
```

231. Partition an array into two sub-arrays with equal sum. [M]

```
# BASE QUESTION (optimal): Subset-sum as a bitset
# Idea: reach is an integer whose bit s is set when some
# subset sums to s; start with bit 0. For each number x do
# reach |= reach << x. The answer is whether bit total / 2
# is set.
def can_partition(nums):
    # reachable subset sums as the bits of one integer
    total = sum(nums)
    if total % 2:
        return False
    reach = 1
    for x in nums:
        # every old sum s also gives s + x
        reach |= reach << x
    return bool(reach >> (total // 2) & 1)
```

232. Explain how to find a target sum in an array. [M]

```
# BASE QUESTION (optimal): Subset-sum counting: choose P
# with sum (total + S) / 2
# Idea: Let need = (total + S) / 2 (return 0 if it is not
# a non-negative integer). ways[s] counts the subsets with
# sum s; for each number x update ways from high to low:
# ways[s] += ways[s - x]. The answer is ways[need].
def count_signs(nums, target):
    # +/- signs reaching `target` equal a subset with sum
    # (total + target) / 2 that gets the plus signs. So
    # the
    # question becomes: how many subsets have that sum?
    total = sum(nums)
    if abs(target) > total or (total + target) % 2:
        return -1
```

```
    goal = (total + target) // 2
    ways = [1] + [0] * goal
    for x in nums:
        # go downward so every number is used at most once
        for s in range(goal, x - 1, -1):
            ways[s] += ways[s - x]
    return ways[goal]
```

233. Reverse Coin Change [M]

```
# BASE QUESTION (optimal): Scan upward and add a coin
# whenever the table falls short
# Idea: Keep the ways table for the coins found so far.
# Answer: Return None (or raise) when dp[0] != 1, when an
# amount needs more than one extra way, or when the
# rebuilt table differs from the input at the end; the
# code does all three checks.
def coins_from_dp(dp):
    # backward: rebuild the table with the coins found so
    # far. The first amount where the real dp is exactly
    # one
    # bigger must be a new coin (it adds the single way
    # "just that coin"). Anything else is impossible.
    n = len(dp) - 1
    if not dp or dp[0] != 1:
        return None
    cur, coins = [1] + [0] * n, []
    for a in range(1, n + 1):
        if dp[a] == cur[a]:
            continue
        if dp[a] != cur[a] + 1:
            return None
        coins.append(a)
        for x in range(a, n + 1):
            cur[x] += cur[x - a]
    # the table rebuilt from the coins must equal the
    # input
    return coins if cur == list(dp) else None
```

GROUP 33 OF 43 | 5 QUESTIONS, ONE CODE TEMPLATE
Prefix sums, Tier 2: running total plus a dict of earlier totals

234. Maximum-Sum Subarray With Equal Endpoints [M]

```
# BASE QUESTION (optimal): Best earlier prefix per value
# Idea: For each end the best start is the same value with
# the smallest prefix sum so far.
def max_sum_equal_endpoints(nums):
    prefix = [0]
    for x in nums:
        prefix.append(prefix[-1] + x)
    best_start, best = {}, None
    for j, x in enumerate(nums):
        if (
            x not in best_start
            or prefix[j] < prefix[best_start[x]]
        ):
            best_start[x] = j
        i = best_start[x]
        key = (-(prefix[j + 1] - prefix[i]), i, j - i)
        if best is None or key < best:
            best = key
    return (
        nums[best[1] : best[1] + best[2] + 1]
        if best
        else []
    )
```

235. Maximum Subarray Sum With a Minimum Length Constraint [M]

```
# BASE QUESTION (optimal): Prefix sums and a running
# minimum
# Idea: Sweep the end of the subarray; the best start is
# the smallest prefix sum that leaves at least min_len
# elements.
def max_subarray_min_len(nums, min_len):
    prefix = [0]
    for x in nums:
        prefix.append(prefix[-1] + x)
    best, smallest = None, float("inf")
    for end in range(min_len, len(nums) + 1):
        # the prefix that is allowed to start a valid
        # subarray
        smallest = min(smallest, prefix[end - min_len])
        cand = prefix[end] - smallest
        best = cand if best is None else max(best, cand)
    return best
```

236. Solve array-sum and city-community problems [M]

```
# HELPERS for the base question
STEPS = ((1, 0), (-1, 0), (0, 1), (0, -1))


# BASE QUESTION (optimal): prefix sums in a dictionary
# Idea: Walk the array keeping the prefix sum.
def count_subarrays(nums, k):
    # prefix sums: a subarray ending here has sum k if
    # some
    # earlier prefix equals (prefix - k)
    seen, prefix, count = {0: 1}, 0, 0
    for x in nums:
        prefix += x
        count += seen.get(prefix - k, 0)
        seen[prefix] = seen.get(prefix, 0) + 1
    return count


# BASE QUESTION (optimal): prefix sums in a dictionary
# Idea: Walk the array keeping the prefix sum.
def communities(grid):
```

```
    # connected groups of buildings (1) in a grid
    seen, count = set(), 0
    for r in range(len(grid)):
        for c in range(len(grid[0])):
            if grid[r][c] != 1 or (r, c) in seen:
                continue
            count += 1
            seen.add((r, c))
            stack = [(r, c)]
            while stack:
                x, y = stack.pop()
                for dx, dy in STEPS:
                    a, b = x + dx, y + dy
                    if not (0 <= a < len(grid)):
                        continue
                    if not (0 <= b < len(grid[0])):
                        continue
                    land = grid[a][b] == 1
                    if land and (a, b) not in seen:
                        seen.add((a, b))
                        stack.append((a, b))
    return count
```

237. Find the maximum subarray sum. [M]

```
# BASE QUESTION (optimal): Kadane's algorithm
# Idea: cur is the best sum of a subarray ending at the
# current element: either the element alone or the element
# added to the previous cur. best is the maximum cur seen.
def max_subarray(nums):
    # Kadane: the best sum of a run ending here either
    # extends
    # the previous run or starts again at this element
    cur = best = nums[0]
    for x in nums[1:]:
        cur = max(x, cur + x)
        best = max(best, cur)
    return best
```

238. Given an integer array nums and an integer k, return true if nums has a subarray of at least two elements whose sum is a multiple of k. [H]

```
# BASE QUESTION (optimal): Prefix sums mod k with the
# first index of each remainder
# Idea: Walk the array keeping the running sum modulo k.
# If this remainder was seen before at index j and the
# current index i satisfies i - j >= 2, the elements
# between form a good subarray.
def has_good_subarray(nums, k):
    # prefix sums modulo k: two equal remainders at
    # indexes
    # at least 2 apart mean a block of 2 or more items
    # whose
    # sum is a multiple of k. Keep only the FIRST index of
    # each remainder: a later one only gives a shorter
    # block.
    first = {0: -1}
    total = 0
    for i, x in enumerate(nums):
        total += x
        # k == 0 means "equal to zero": compare the sums
        key = total % k if k else total
        if key not in first:
            first[key] = i
        elif i - first[key] >= 2:
            return True
    return False
```

GROUP 34 OF 43 | 5 QUESTIONS, ONE CODE TEMPLATE
Group by a canonical key

239. Given an array of strings, return the words that can be typed using letters of the alphabet on only one row of American keyboard. [E]

```
# HELPERS for the base question
QWERTY = ["qwertyuiop", "asdfghjkl", "zxcvbnm"]


# LAYOUT table (change it for other keyboards)
def row_map(rows):
    # letter -> index of its row; built once and reused
    return {ch: i for i, row in enumerate(rows)
            for ch in row}


# BASE QUESTION (optimal): Map each letter to its row and
# compare sets
# Idea: Build a dict from letter to row index (26
# entries). A word qualifies if all of its lowercased
# letters map to the same row, i.e. the set of rows has
# size one. Lowercase first so 'Alaska' works.
# Answer: Pass the rows as a parameter and build the
# letter-to-row map from it (a phone keypad or Dvorak
# works the same way). Letters that are on no row make the
# word invalid, which the code checks.
def one_row_words(words, rows=QWERTY):
    # FOLLOW-UP 3: pass rows= for another keyboard layout
    row_of = row_map(rows)
    out = []
    for w in words:
        letters = w.lower()
        # a letter that is on no row makes the word
        # invalid
        if any(c not in row_of for c in letters):
            continue
        # all letters must come from the same row
        if len({row_of[c] for c in letters}) <= 1:
            out.append(w)
    return out
```

240. Check if all substrings are anagrams of words [M]

```
# HELPERS for the base question
ALPHABET = "abcdefghijklmnopqrstuvwxyz"


# BASE QUESTION (optimal): Set of signatures, incremental
# count vectors
# Idea: Precompute the set of count-vector signatures of
# the dictionary words. For each start extend one letter
# at a time, update the count vector in O(1), and look up
# its tuple (once the length is at least 3).
def all_anagram_words(s, words, min_len=3):
    # every substring needs a dictionary word with the
    # same
    # letters. Count vectors are updated as the window
    # grows.
    index = {c: i for i, c in enumerate(ALPHABET)}
    sigs = set()
    for w in words:
        v = [0] * len(ALPHABET)
        for ch in w:
            v[index[ch]] += 1
        sigs.add(tuple(v))
    for i in range(len(s)):
        v = [0] * len(ALPHABET)
        for j in range(i, len(s)):
            v[index[s[j]]] += 1
            long_enough = j - i + 1 >= min_len
```

```
            if long_enough and tuple(v) not in sigs:
                return False
    return True
```

241. Group anagrams [M]

```
from collections import defaultdict


# BASE QUESTION (optimal): Dictionary keyed by the sorted
# letters
# Idea: Sort the letters of each word and use the
# resulting string as the key; words with the same key are
# anagrams. One pass, one dictionary.
def group_by_sorting(words):
    # anagrams share the same letters in sorted order
    groups = defaultdict(list)
    for w in words:
        groups["".join(sorted(w))].append(w)
    return list(groups.values())
```

242. Given an array of strings, return groups of "buddy strings," where the strings have the same length and the distance between corresponding characters is the same. [M]

```
from collections import defaultdict


# HELPERS for the base question
def shift_key(word):
    # the gaps between neighbouring letters, wrapping z to
    # a.
    # Words that are shifts of each other share this key.
    return tuple((ord(b) - ord(a)) % 26
                 for a, b in zip(word, word[1:]))


# BASE QUESTION (optimal): Group by the tuple of
# successive differences
# Idea: Two strings are buddies when every letter is
# shifted by the same amount, which means the gaps between
# neighbouring letters (mod 26) are identical. Use those
# gaps as the hash key.
def group_buddies(words, key=shift_key):
    # words of different length have keys of different
    # length, so they never land in the same group
    groups = defaultdict(list)
    for w in words:
        groups[(len(w), key(w))].append(w)
    return list(groups.values())
```

243. Given two strings and two allowed operations, swapping positions of letters and converting all occurrences of one character to another, can the first string be transformed into the second? [H]

```
from collections import Counter


# HELPERS for the base question
ALPHABET = 26


def can_group(have, want):
    # can the counts in `have` be grouped (each used once)
    # so that the group sums are exactly the counts in
    # `want`? Letters merge, they never split.
    if sum(have) != sum(want):
        return False
    have = sorted(have, reverse=True)
    left = list(want)

    def place(i):
        if i == len(have):
            return all(x == 0 for x in left)
        tried = set()
```

```
        for j in range(len(left)):
            # equal bins behave the same: try each size
            # once
            fits = left[j] >= have[i]
            if fits and left[j] not in tried:
                tried.add(left[j])
                left[j] -= have[i]
                if place(i + 1):
                    return True
                left[j] += have[i]
        return False

    return place(0)


# BASE QUESTION (optimal): (LeetCode 1657): same letter
# set and same multiset of frequencies
# Idea: Operation 1 reorders positions freely, so only the
# multiset of letters matters. Operation 2 exchanges the
# identities of two letters that exist, which permutes the
# frequencies among existing letters.
def both_operations(s, t, alphabet=ALPHABET):
    # swaps make positions irrelevant: only the letter
    # counts matter, and a conversion can only merge or
    # rename a letter
    if len(s) != len(t):
        return False
    cs, ct = Counter(s), Counter(t)
    if cs == ct:
        return True
    mine, goal = list(cs.values()), list(ct.values())
    if not can_group(mine, goal):
        return False
    # only identical sizes mean no merge happens. With
    # every
    # letter used there is no spare letter to rename
    # safely.
    if len(cs) == len(ct) == alphabet:
        return False
    return True
```

GROUP 35 OF 43 | 5 QUESTIONS, ONE CODE TEMPLATE
K-way merge: a heap of the next item from each source

244. Find k pairs with smallest sums [E]

```
import heapq


# BASE QUESTION (optimal): Min-heap over the sorted rows
# Idea: Push (a[i] + b[0], i, 0) for the first min(k, m)
# values of a. Pop the smallest, record the pair, and push
# the next pair in the same row, (a[i] + b[j + 1], i, j +
# 1). After k pops you have the k smallest sums.
def k_smallest_pairs(a, b, k):
    # a and b sorted: a heap holds the best unused pair of
    # each row i; popping (i, j) offers (i, j + 1)
    if not a or not b or k <= 0:
        return []
    rows = range(min(k, len(a)))
    heap = [(a[i] + b[0], i, 0) for i in rows]
    heapq.heapify(heap)
    out = []
    while heap and len(out) < k:
        _, i, j = heapq.heappop(heap)
        out.append((a[i], b[j]))
        if j + 1 < len(b):
            nxt = a[i] + b[j + 1]
            heapq.heappush(heap, (nxt, i, j + 1))
    return out
```

245. K Largest Elements Across Sorted Lists [M]

```
import heapq


# BASE QUESTION (optimal): A max-heap over the ends of the
# lists
# Idea: The next largest element is always at the end of
# one of the lists.
def k_largest_sorted(lists, k):
    heap = [
        (-lst[-1], i, len(lst) - 1)
        for i, lst in enumerate(lists)
        if lst
    ]
    heapq.heapify(heap)
    out = []
    while heap and len(out) < k:
        neg, i, j = heapq.heappop(heap)
        out.append(-neg)
        if j:
            heapq.heappush(
                heap, (-lists[i][j - 1], i, j - 1)
            )
    return out[::-1]
```

246. Sort a nearly sorted (or K sorted) array. [M]

```
import heapq


# BASE QUESTION (optimal): Min-heap of size K + 1
# Idea: Push the first K + 1 elements. Then for each
# remaining element, pop the smallest into the output and
# push the new element. At the end pop the heap empty.
def sort_k_sorted(arr, k):
    # every item is at most k places from its final spot,
    # so
    # the smallest of the next k + 1 items is the next
    # answer. Keep k items in a heap, and for each new one
    # push it and pop the smallest: O(n log k).
    heap = list(arr[:k])
```

```
    heapq.heapify(heap)
    out = []
    for x in arr[k:]:
        # push the new item and pop the smallest in one go
        out.append(heapq.heappushpop(heap, x))
    while heap:
        out.append(heapq.heappop(heap))
    return out
```

247. Merge k sorted linked lists. [H]

```
import heapq


# HELPERS used by the code below
class ListNode:
    def __init__(self, val=0, next=None):
        self.val, self.next = val, next


# HELPERS for the base question
def _to_list(head):
    out = []
    while head:
        out.append(head.val)
        head = head.next
    return out


# BASE QUESTION (optimal): Min-heap of the current heads
# Idea: Push the head of each non-empty list as (value,
# list index, node). Pop the smallest, append it to the
# output, and push its successor. The list index is a tie-
# breaker so nodes are never compared.
def merge_k_heap(lists):
    # a heap holds the current head of every list; the
    # node
    # index breaks ties so nodes are never compared
    heap = [(n.val, i, n)
            for i, n in enumerate(lists) if n]
    heapq.heapify(heap)
    dummy = tail = ListNode()
    while heap:
        _, i, node = heapq.heappop(heap)
        tail.next = node
        tail = node
        if node.next:
            nxt = node.next
            heapq.heappush(heap, (nxt.val, i, nxt))
    return dummy.next
```

248. Return top-k subset sums in descending order [H]

```
import heapq


# BASE QUESTION (optimal): Heap of the k smallest sums of
# absolute values
# Idea: Every subset sum equals (sum of positives) minus a
# subset sum of the absolute values, so the k largest sums
# are the sum of positives minus the k smallest subset
# sums of the sorted absolute values.
def top_k_sums(nums, k):
    # Idea: start from the best subset (all positives).
    # Every
    # other subset is that one with some elements
    # "flipped",
    # which costs |x|. So the k best subsets are the k
    # cheapest sets of flips: a heap over sorted |x|.
    base = sum(x for x in nums if x > 0)
    cost = sorted(abs(x) for x in nums)
    out = [0]
    heap = [(cost[0], 0)] if cost else []
    while heap and len(out) < k:
```

```
    s, i = heapq.heappop(heap)
    out.append(s)
    if i + 1 < len(cost):
        # two successors, neither is ever cheaper:
        # add the next element ...
        heapq.heappush(heap, (s + cost[i + 1], i + 1))
        # ... or move the last flipped one up by one
        moved = s - cost[i] + cost[i + 1]
        heapq.heappush(heap, (moved, i + 1))
return [base - c for c in out[:k]]
```

GROUP 36 OF 43 | 5 QUESTIONS, ONE CODE TEMPLATE
Tree recursion, Tier 2: pass a rule down, return a node or a yes/no up

249. Next Birthday (BST In-Order Successor with Wraparound) [E]

```
# BASE QUESTION (optimal): Descend the BST once
# Idea: A node above today is a candidate; its left
# subtree may hold a closer one.
def next_birthday(root, today):
    if root is None:
        return None
    best, node = None, root
    while node:
        if node.val > today:
            best, node = node.val, node.left
        else:
            node = node.right
    if best is not None:
        return best
    node = root
    while node.left:
        node = node.left
    return node.val
```

250. Lowest Common Ancestor [M]

```
# HELPERS for the base question
class BNode:
    def __init__(self, val, left=None, right=None):
        self.val, self.left, self.right = (
            val,
            left,
            right,
        )


# BASE QUESTION (optimal): One recursive pass
# Idea: dfs(node) returns node if node is p or q,
# otherwise combines its children: if both children return
# something, node is the LCA; if only one does, pass that
# up.
# Answer: The same recursion: collect the non-empty
# results from all children; if two or more children
# reported a hit this node is the answer, otherwise pass
# the single result (or None) up.
def lca(root, p, q):
    # each call reports where p or q was found below. A
    # node
    # with hits from two different children (or that is p
    # or
    # q itself) is the answer; a single hit is passed up.
    def find(node):
        if node is None:
            return None
        if node is p or node is q:
            return node
        # FOLLOW-UP 1: n-ary: collect hits from every
        # child
        hits = [r for r in map(find, node.children) if r]
        if len(hits) >= 2:
            return node
        return hits[0] if hits else None

    return find(root)
```

251. Determine if a given binary tree is a binary search tree (BST). [M]

```
# BASE QUESTION (optimal): Pass the allowed range (low,
# high) down
# Idea: Each node must lie strictly between low and high.
# The left child inherits (low, node.val) and the right
# child (node.val, high). The root starts with (-inf,
# +inf).
# Answer: Make one of the two bounds inclusive: for
# 'duplicates go left' every node must satisfy lo < val <=
# hi (the bound inherited from a left ancestor is
# inclusive, the one from a right ancestor is exclusive).
def is_bst(root, dup="none"):
    # FOLLOW-UP 1: dup="left" or "right" allows duplicates
    # every node must lie between the bounds from its
    # ancestors. dup says where an equal value may go:
    # "none" (no duplicates), "left" or "right".
    def fits(val, lo, hi):
        if lo is not None:
            if val < lo or (val == lo and dup != "right"):
                return False
        if hi is not None:
            if val > hi or (val == hi and dup != "left"):
                return False
        return True

    stack = [(root, None, None)]
    while stack:
        node, lo, hi = stack.pop()
        if node is None:
            continue
        if not fits(node.val, lo, hi):
            return False
        stack.append((node.left, lo, node.val))
        stack.append((node.right, node.val, hi))
    return True
```

252. Find mode in a trinary search tree [M]

```
# BASE QUESTION (optimal): In-order traversal counting
# runs (O(height) extra space)
# Idea: Walk left, middle, right with an explicit stack.
def find_modes(root):
    # in-order: equal values are next to each other, so a
    # running count needs no dictionary
    best, modes, cur, run = 0, [], None, 0

    def visit(v):
        nonlocal best, modes, cur, run
        run = run + 1 if v == cur else 1
        cur = v
        if run > best:
            best, modes = run, [v]
        elif run == best and v not in modes:
            modes.append(v)

    stack, node = [], root
    while stack or node:
        while node:
            stack.append(node)
            node = node.left
        node = stack.pop()
        visit(node.val)
        # the middle chain repeats node.val once per child
        mid = node.middle
        while mid:
            visit(mid.val)
            mid = mid.middle
        node = node.right
    return sorted(modes)
```

253. Next Upcoming Birthday in a Family Tree [M]

```
# BASE QUESTION (optimal): One traversal tracking the
# minimum distance
# Idea: Walk every distinct ancestor once and keep the
# smallest cyclic distance.
class Person:
    def __init__(
        self,
        name,
        month,
        day,
        parent1=None,
        parent2=None,
    ):
        self.name = name
        self.month = month
        self.day = day
        self.parent1 = parent1
        self.parent2 = parent2


# BASE QUESTION (optimal): One traversal tracking the
# minimum distance
# Idea: Walk every distinct ancestor once and keep the
# smallest cyclic distance.
def alarm_next_birthday(me, today_month, today_day):
    if me is None:
        return ""
    today = key(today_month, today_day)
    best, answer = None, ""
    stack, seen = [me], set()
    while stack:
        p = stack.pop()
        if id(p) in seen:
            continue
        seen.add(id(p))
        dist = (key(p.month, p.day) - today) % 372
        if best is None or dist < best:
            best, answer = dist, (p.month, p.day)
        stack += [
            q for q in (p.parent1, p.parent2) if q
        ]
    return answer
```

GROUP 37 OF 43 | 5 QUESTIONS, ONE CODE TEMPLATE
Random pick without repeats: swap the chosen item with the last one and pop

254. Random Generator Without Repeats [M]

```
import random


# BASE QUESTION (optimal): Partial Fisher-Yates with an
# active region
# Idea: Swap the drawn value to the end of the active
# region and shrink it.
class RandomGenerator:
    def __init__(self, n, seed=None):
        self.n = n
        self.values = list(range(n))
        self.size = n
        self.rng = random.Random(seed)

    def generate(self):
        if self.size == 0:
            raise IndexError(
                "all values have been generated"
            )
        i = self.rng.randrange(self.size)
        self.size -= 1
        self.values[i], self.values[self.size] = (
            self.values[self.size],
            self.values[i],
        )
        return self.values[self.size]
```

255. Random Song Shuffler With a No-Repeat Window [M]

```
from collections import deque


# BASE QUESTION (optimal): Eligible array, index map and
# swap-and-pop
# Idea: Remove the played song from the eligible array by
# swapping it with the last element; the song leaving the
# window is appended back.
class SongShuffler:
    def __init__(self, songs, k, seed=None):
        self.valid = list(songs)
        self.index = {
            s: i for i, s in enumerate(self.valid)
        }
        self.recent = deque()
        self.k = k
        self.rng = random.Random(seed)

    def play(self):
        i = self.rng.randrange(len(self.valid))
        song = self.valid[i]
        last = self.valid[-1]
        # swap-and-pop: the last song takes the hole
        self.valid[i] = last
        self.index[last] = i
        self.valid.pop()
        del self.index[song]
        self.recent.append(song)
        if len(self.recent) > self.k:
            back = self.recent.popleft()
            self.index[back] = len(self.valid)
            self.valid.append(back)
        return song
```

256. Design set with O(1) random access [M]

```
import random


# BASE QUESTION (optimal): Array of values plus a dict
# from value to index
# Idea: add appends to the array and records the index.
# delete finds the index from the dict, moves the last
# element into that slot, updates its index in the dict,
# pops the array and removes the key.
class FancySet:
    """add, delete and getRandom, each O(1) on average."""

    def __init__(self, rng=None):
        # the values in a list (for random access) and a
        # map
        # from value to its index in that list
        self.items, self.pos = [], {}
        self.rng = rng or random

    def add(self, x):
        if x in self.pos:
            return False
        self.pos[x] = len(self.items)
        self.items.append(x)
        return True

    def delete(self, x):
        if x not in self.pos:
            return False
        i, last = self.pos[x], self.items[-1]
        # move the last item into the hole, then drop the
        # tail
        self.items[i] = last
        self.pos[last] = i
        self.items.pop()
        del self.pos[x]
        return True

    def get_random(self):
        if not self.items:
            # say what happens on an empty set
            raise IndexError("empty set")
        i = self.rng.randrange(len(self.items))
        return self.items[i]
```

257. Unique Random Number Generator [M]

```
import random


# BASE QUESTION (optimal): Lazy Fisher-Yates with a
# dictionary
# Idea: Swap a random unused slot into position i,
# remembering only the moved slots.
class UniqueRandom:
    def __init__(self, n, seed=None):
        self.n = n
        self.drawn = 0
        self.moved = {}
        self.rng = random.Random(seed)

    def generate(self):
        if self.drawn == self.n:
            return None
        i = self.drawn
        j = self.rng.randrange(i, self.n)
        value = self.moved.get(j, j)
        # the value that sat at slot i takes the place of
        # the one just drawn
        self.moved[j] = self.moved.get(i, i)
        self.drawn += 1
        return value
```

258. Shuffle array using random integer API [H]

```
# HELPERS for the base question
def shuffle_by_keys(nums):
    keyed = sorted(
        (
            (rand_int(0, 10**9), i)
            for i in range(len(nums))
        )
    )
    return [nums[i] for _, i in keyed]


# BASE QUESTION (optimal): Fisher-Yates
# Idea: Walk i from n - 1 down to 1, pick j uniformly from
# 0..i (inclusive) and swap nums[i] with nums[j].
def shuffle(nums, rand_int):
    # Fisher-Yates: from the end, swap each position with
    # a
    # random one at or before it. All n! orders are
    # equally
    # likely because each step picks uniformly among i +
    # 1.
    for i in range(len(nums) - 1, 0, -1):
        j = rand_int(0, i)
        nums[i], nums[j] = nums[j], nums[i]
    return nums
```

GROUP 38 OF 43 | 5 QUESTIONS, ONE CODE TEMPLATE
Build and serialize: write a structure out, read it back with the same walk

259. Construct a binary tree given its preorder and inorder traversals. [M]

```
# HELPERS used by the code below
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right


# BASE QUESTION (optimal): Hash map of inorder positions
# and index bounds
# Idea: Map each value to its inorder index.
def build_pre_in(preorder, inorder):
    # preorder: root, left, right. The root's position in
    # the
    # inorder list splits it into the left and right
    # parts.
    where = {v: i for i, v in enumerate(inorder)}
    nxt = 0

    def build(lo, hi):
        nonlocal nxt
        if lo >= hi:
            return None
        root = TreeNode(preorder[nxt])
        nxt += 1
        mid = where[root.val]
        root.left = build(lo, mid)
        root.right = build(mid + 1, hi)
        return root

    return build(0, len(inorder))
```

260. Write functions to serialize and deserialize a list of strings. [M]

```
# BASE QUESTION (optimal): Length-prefix every string
# Idea: Write len(s), a separator, then s itself. To
# decode read digits up to the separator to get the
# length, then take exactly that many characters.
def serialize(items):
    # length prefix: "<length>#<text>" for every string.
    # The
    # reader knows exactly how much to read, so the text
    # may
    # hold any character, even "#" and digits.
    return "".join(f"{len(s)}#{s}" for s in items)


# BASE QUESTION (optimal): Length-prefix every string
# Idea: Write len(s), a separator, then s itself. To
# decode read digits up to the separator to get the
# length, then take exactly that many characters.
def deserialize(text):
    items, i = [], 0
    while i < len(text):
        # read the digits up to the "#"
        sep = text.index("#", i)
        size = int(text[i:sep])
        start = sep + 1
        items.append(text[start:start + size])
        i = start + size
    return items
```

261. Print a Directory Tree [M]

```
# BASE QUESTION (optimal): Nested dictionaries and a DFS
# Idea: Insert every path into a dict of dicts, then print
# depth first with two spaces per level.
def print_directory_tree(paths):
    root = {}
    for path in paths:
        node = root
        for part in path.split("/"):
            node = node.setdefault(part, {})
    lines = []

    def walk(node, depth):
        for name, kids in node.items():
            lines.append("  " * depth + name)
            walk(kids, depth + 1)

    walk(root, 0)
    return "\n".join(lines)
```

262. Implement a decode and encode function of a general N-ary tree. [H]

```
import json


# HELPERS for the base question
class Node:
    def __init__(self, val, children=None):
        self.val = val
        self.children = children or []


def preorder_pairs(root):
    # (value, number of children) for every node in pre-
    # order,
    # iterative so a very deep tree cannot overflow the
    # stack
    out = []
    stack = [root] if root else []
    while stack:
        node = stack.pop()
        out.append((node.val, len(node.children)))
        # reversed: the first child must be popped first
        stack.extend(reversed(node.children))
    return out


def build_preorder(pairs):
    # the reverse: rebuild the tree from the same pairs
    pairs = iter(pairs)
    first = next(pairs, None)
    if first is None:
        return None
    root = Node(first[0])
    # each stack entry is a parent and the children it
    # still has to receive
    stack = [[root, first[1]]]
    while stack:
        top = stack[-1]
        if top[1] == 0:
            stack.pop()
            continue
        top[1] -= 1
        val, count = next(pairs)
        child = Node(val)
        top[0].children.append(child)
        stack.append([child, count])
    return root


# BASE QUESTION (optimal): Pre-order with child counts,
# iterative decode
# Idea: Write (value, number of children) in pre-order.
```

```
# Answer: Do not invent delimiters. Put the values in a
# real container: a JSON list of alternating value and
# child count, which handles quoting for you, or length-
# prefix every value (5#hello). The structure part (the
# counts) stays the same.
def encode(root):
    # FOLLOW-UP 1: arbitrary string values
    # values may hold spaces, commas or brackets: JSON
    # does
    # the quoting, so no invented delimiter can clash
    flat = []
    for val, count in preorder_pairs(root):
        flat += [val, count]
    return json.dumps(flat)


# BASE QUESTION (optimal): Pre-order with child counts,
# iterative decode
# Idea: Write (value, number of children) in pre-order.
def decode(text):
    flat = json.loads(text)
    return build_preorder(zip(flat[::2], flat[1::2]))
```

263. Serialize and Deserialize a Graph [H]

```
from collections import deque


# BASE QUESTION (optimal): BFS to a text, two passes back
# Idea: One token per node with its neighbour values;
# rebuild by creating the nodes first and linking
# afterwards.
class Node:
    def __init__(self, val=0, neighbors=None):
        self.val = val
        self.neighbors = (
            neighbors if neighbors is not None else []
        )


# BASE QUESTION (optimal): BFS to a text, two passes back
# Idea: One token per node with its neighbour values;
# rebuild by creating the nodes first and linking
# afterwards.
class Codec:
    def serialize(self, node):
        if node is None:
            return ""
        seen, queue, parts = (
            {node.val},
            deque([node]),
            [],
        )
        while queue:
            cur = queue.popleft()
            parts.append(
                f"{cur.val}:"
                + ",".join(
                    str(n.val) for n in cur.neighbors
                )
            )
            for nb in cur.neighbors:
                if nb.val not in seen:
                    seen.add(nb.val)
                    queue.append(nb)
        return ";".join(parts)

    def deserialize(self, data):
        if not data:
            return None
        rows = [
            part.split(":") for part in data.split(";")
        ]
```

```
nodes = {int(v): Node(int(v)) for v, _ in rows}
for v, nbs in rows:
    nodes[int(v)].neighbors = [
        nodes[int(x)]
        for x in nbs.split(",")
        if x
    ]
return nodes[int(rows[0][0])]
```

GROUP 39 OF 43 | 5 QUESTIONS, ONE CODE TEMPLATE
Games: win or lose with a cache, and guessing with feedback

264. Choose optimal guesses for green-only Wordle [M]

```
from collections import defaultdict


from fractions import Fraction


# HELPERS used by the code below
def green(guess, answer):
    # green-only feedback: which positions match exactly
    return tuple(g == a for g, a in zip(guess, answer))


def partition(guess, words):
    # group the possible answers by the feedback they
    # would give
    groups = defaultdict(list)
    for w in words:
        groups[green(guess, w)].append(w)
    return list(groups.values())


def score(guess, words, weight=None):
    # expected number of answers left after this guess:
    # sum(group^2) / total, with optional per-word weights
    weight = weight or (lambda w: 1)
    total = sum(weight(w) for w in words)
    spread = sum(sum(weight(w) for w in g) ** 2
                 for g in partition(guess, words))
    return Fraction(spread, total)


# BASE QUESTION (optimal): Bitsets per (position, letter),
# splitting groups with AND
# Idea: Index the words: bits[pos][letter] is an integer
# whose bit i is set when word i has that letter at pos.
def best_guess(words, weight=None):
    # the guess that leaves the fewest answers on average
    # (ties: alphabetical, so the result is deterministic)
    def rank(g):
        return (score(g, words, weight), g)
    return min(words, key=rank)
```

265. Wordle-Style 5-Letter Minimum-Guess Strategy [H]

```
# HELPERS for the base question
# letters from most to least common in English words
BY_FREQUENCY = "etaoinshrdlcumwfgypbvkjxqz"


# Answer: Every call returns at most one bit, so at least
# ceil(log2(candidates)) calls are needed. The entropy
# split reaches it only when a question can cut the
# candidates exactly in half; real dictionaries usually
# need a few more.
def split_score(words, i, c):
    # how evenly does asking (position i, letter c) split?
    yes = sum(w[i] == c for w in words)
    return min(yes, len(words) - yes)


# BASE QUESTION (optimal): Frequency order, or entropy
# split when a dictionary is known
# Idea: Without a dictionary: for each position ask
# letters from most to least common and stop at the first
# yes; if 25 letters said no the 26th is implied and needs
# no call.
def by_frequency(query):
```

```
    # no dictionary: common letters first, stop at the
    # first
    # yes, and skip the 26th letter (it must be the one
    # left)
    calls, word = 0, []
    for i in range(5):
        for k, c in enumerate(BY_FREQUENCY):
            if k == 25:
                word.append(c)
                break
            calls += 1
            if query(i, c):
                word.append(c)
                break
    return "".join(word), calls


# BASE QUESTION (optimal): Frequency order, or entropy
# split when a dictionary is known
# Idea: Without a dictionary: for each position ask
# letters from most to least common and stop at the first
# yes; if 25 letters said no the 26th is implied and needs
# no call.
def with_dictionary(query, dictionary):
    # keep the words that still fit every answer so far
    words, calls = list(dictionary), 0
    while len(words) > 1:
        options = {(i, w[i]) for w in words
                   for i in range(5)}
        # ask the question that cuts the list closest to
        # half
        i, c = max(options, key=lambda o: split_score(
            words, o[0], o[1]))
        calls += 1
        hit = query(i, c)
        words = [w for w in words if (w[i] == c) == hit]
    return words[0], calls
```

266. Determine winner in stack-merging game [H]

```
from functools import lru_cache


# HELPERS for the base question
def moves(state, cap):
    # merge two stacks if the new stack fits under the cap
    seen = set()
    for i in range(len(state)):
        for j in range(i + 1, len(state)):
            if state[i] + state[j] <= cap:
                rest = [x for k, x in enumerate(state)
                        if k not in (i, j)]
                merged = state[i] + state[j]
                new = tuple(sorted(rest + [merged]))
                # equal results are the same move
                if new not in seen:
                    seen.add(new)
                    yield new


# BASE QUESTION (optimal): Memoize on the sorted state
# Idea: Merging does not depend on order, so a sorted
# tuple identifies the position. Each state is solved once
# and then looked up. This is the standard fix for 'many
# move orders reach the same position'.
def first_player_wins(stacks, cap, misere=False):
    # the player who cannot move loses (misere: wins)
    @lru_cache(None)
    def wins(state):
        options = list(moves(state, cap))
        if not options:
            return misere
        # a winning move leaves the opponent in a losing
```

```
        # state
        return any(not wins(nxt) for nxt in options)

    return wins(tuple(sorted(stacks)))
```

267. Find secret word with match-count feedback [H]

```
from collections import Counter


# HELPERS for the base question
def match(a, b):
    # number of positions holding the same letter
    return sum(x == y for x, y in zip(a, b))


# BASE QUESTION (optimal): Guess the word that minimises
# the size of its largest match-count group
# Idea: For each candidate guess count how many other
# candidates would fall into each match-count group, and
# pick the guess whose biggest group is smallest (a
# minimax choice).
def best_guess(candidates):
    # minimax: the guess whose worst answer leaves the
    # fewest
    # candidates (ties: alphabetical)
    def worst(g):
        sizes = Counter(match(g, w) for w in candidates)
        return max(sizes.values()), g
    return min(candidates, key=worst)


# BASE QUESTION (optimal): Guess the word that minimises
# the size of its largest match-count group
# Idea: For each candidate guess count how many other
# candidates would fall into each match-count group, and
# pick the guess whose biggest group is smallest (a
# minimax choice).
def find_secret(words, match_count, limit=10):
    # match_count(guess) is the black-box API
    candidates, guesses = list(words), 0
    while candidates and guesses < limit:
        guess = best_guess(candidates)
        guesses += 1
        score = match_count(guess)
        if score == len(guess):
            return guess, guesses
        # only words with the same score against this
        # guess
        # can still be the secret
        candidates = [w for w in candidates
                      if match(guess, w) == score]
    return None, guesses
```

268. Babylon Tile-Stack Merge Game [H]

```
import itertools


# HELPERS for the base question
class Babylon:
    # a stack is (height, colour of its top tile)
    def __init__(self, stacks=None):
        if stacks is None:
            # 12 tiles: Y, W, G, B three times each
            stacks = [(1, c) for c in "YWGB" * 3]
        self.stacks = list(stacks)

    def legal_moves(self):
        # put stack i on top of stack j
        moves = []
        for i, j in itertools.permutations(
                range(len(self.stacks)), 2):
            h1, c1 = self.stacks[i]
```

```
            h2, c2 = self.stacks[j]
            # same height, or the same colour on top
            if h1 == h2 or c1 == c2:
                moves.append((i, j))
        return moves

    def after(self, i, j):
        # the stacks after the move (the original is kept)
        (h1, c1), (h2, _) = self.stacks[i], self.stacks[j]
        rest = [s for k, s in enumerate(self.stacks)
                if k not in (i, j)]
        # heights add; the top colour is the moved stack's
        return rest + [(h1 + h2, c1)]

    def play(self, i, j):
        if (i, j) not in self.legal_moves():
            raise ValueError("illegal move")
        self.stacks = self.after(i, j)

    def is_over(self):
        # the player with no move loses
        return not self.legal_moves()


# BASE QUESTION (optimal): Negamax with memoisation on the
# sorted stack tuple
# Idea: legal_moves lists the ordered pairs that may
# merge, apply builds the next state, and wins(state) is
# true when some move reaches a state where the opponent
# loses.
def wins(stacks):
    # does the player to move win with best play?
    cache = {}

    def solve(state):
        if state in cache:
            return cache[state]
        game = Babylon(state)
        result = False
        for i, j in game.legal_moves():
            nxt = tuple(sorted(game.after(i, j)))
            # a move to a position the opponent loses wins
            if not solve(nxt):
                result = True
                break
        cache[state] = result
        return result

    # sorted stacks: equal positions share one cache entry
    return solve(tuple(sorted(stacks)))
```

GROUP 40 OF 43 | 5 QUESTIONS, ONE CODE TEMPLATE
Sliding window: grow on the right, shrink on the left while it is too bad

269. Temperature Monitoring Data Structure [M]

```
from collections import deque


# HELPERS for the base question
class MonotoneQueue:
    """Best window value is kept at the front."""

    def __init__(self, better):
        self.items = deque()
        # better(a, b): a is at least as good as b
        self.better = better

    def push(self, key, value):
        # values the new one beats can never be best
        while self.items and self.better(
                value, self.items[-1][1]):
            self.items.pop()
        self.items.append((key, value))

    def drop_while(self, too_old):
        # forget entries that left the window
        while self.items and too_old(self.items[0][0]):
            self.items.popleft()

    def best(self):
        return self.items[0][1]


# Answer: Minimum uses a second monotonic deque with the
# opposite comparison. A median needs two heaps with lazy
# deletion or a sorted container, O(log N) per update,
# because it is not decomposable like sum and max.
class SlidingStats:
    def __init__(self):
        self.items = deque()
        self.total = 0
        self.maxq = MonotoneQueue(lambda a, b: a >= b)
        self.minq = MonotoneQueue(lambda a, b: a <= b)

    def add(self, key, value):
        self.items.append((key, value))
        self.total += value
        self.maxq.push(key, value)
        self.minq.push(key, value)

    def expire(self, too_old):
        # drop readings that left the window everywhere
        while self.items and too_old(self.items[0][0]):
            self.total -= self.items.popleft()[1]
        self.maxq.drop_while(too_old)
        self.minq.drop_while(too_old)

    def average(self):
        return self.total / len(self.items)

    def maximum(self):
        return self.maxq.best()

    def minimum(self):
        return self.minq.best()


# BASE QUESTION (optimal): Running sum plus a monotonic
# deque
# Idea: Keep the window in a deque and a running sum; on
# insert add the value and drop the oldest when the window
```

```
# is full.
class Monitor(SlidingStats):
    """Window = the last n readings."""

    def __init__(self, n):
        super().__init__()
        self.n, self.seen = n, 0

    def insert(self, x):
        # the reading number is its key
        self.add(self.seen, x)
        self.seen += 1
        # keep only the newest n readings
        limit = self.seen - self.n
        self.expire(lambda key: key < limit)
```

270. Shortest Subarray with K Distinct Integers [M]

```
from collections import Counter


# BASE QUESTION (optimal): Sliding window with a frequency
# map
# Idea: Expand the right end, counting each value. While
# the window has at least k distinct values, record its
# length and shrink from the left (deleting a value from
# the map when its count reaches 0).
def shortest_k_distinct(nums, k):
    # a window grows on the right; whenever it holds at
    # least k distinct values, record it and shrink from
    # the
    # left as far as it stays valid
    seen, left, best = Counter(), 0, None
    for right, x in enumerate(nums):
        seen[x] += 1
        while len(seen) >= k:
            size = right - left + 1
            if best is None or size < best:
                best = size
            seen[nums[left]] -= 1
            if seen[nums[left]] == 0:
                del seen[nums[left]]
            left += 1
    return -1 if best is None else best
```

271. Find the minimum window substring. [M]

```
from collections import Counter


# HELPERS for the base question
def valid_windows(s, t):
    # sliding window: grow `right` until the window holds
    # all
    # of t, then shrink `left` while it still does. Yield
    # every (left, right) that is valid when it is seen.
    need = Counter(t)
    missing = len(t)
    left = 0
    for right, ch in enumerate(s):
        if need[ch] > 0:
            missing -= 1
        need[ch] -= 1
        while missing == 0:
            yield left, right
            # drop the leftmost character and see if it
            # hurts
            need[s[left]] += 1
            if need[s[left]] > 0:
                missing += 1
            left += 1


# BASE QUESTION (optimal): Sliding window with a 'still
```

```
# needed' counter
# Idea: need[c] is how many more of c the window requires,
# and missing counts the distinct requirements still
# unmet.
def min_window(s, t):
    # the shortest window; the first one if there is a tie
    best, width = None, 0
    for left, right in valid_windows(s, t):
        if best is None or right - left < width:
            best, width = (left, right), right - left
    return s[best[0]:best[1] + 1] if best else ""
```

272. Footpath Beauty After Repainting [M]

```
# BASE QUESTION (optimal): Sliding window with at most m
# repaints
# Idea: The longest window holding at most m other-colour
# blocks is the answer.
def max_beauty_after_repaint(block_colors, c, m):
    left, other, best = 0, 0, 0
    for right, color in enumerate(block_colors):
        other += color != c
        while other > m:
            other -= block_colors[left] != c
            left += 1
        best = max(best, right - left + 1)
    return best
```

273. Necklace Cut into Two Halves with Equal D/R Counts [H]

```
# BASE QUESTION (optimal): Fixed-length circular window
# with the discrete intermediate value argument
# Idea: If each arc has half of the D's and half of the
# R's, each arc has length n/2, and the total D and R
# counts must both be even. Slide a window of length n/2
# around the circle (use s + s), tracking its D count.
def necklace_two_cuts(s):
    n = len(s)
    d_total, r_total = s.count("D"), s.count("R")
    # each piece needs half of both letters
    if n % 2 or d_total % 2 or r_total % 2:
        return None
    half, target = n // 2, d_total // 2
    doubled = s + s
    # slide a window of length n/2 around the circle
    window = doubled[:half].count("D")
    for i in range(n):
        if window == target:
            # cuts at i and i + n/2 give two good halves
            first = doubled[i:i + half]
            second = doubled[i + half:i + n]
            return i, (i + half) % n, first, second
        # move the window one step: add the new, drop the
        # old
        window += (doubled[i + half] == "D")
        window -= (doubled[i] == "D")
    return None
```

GROUP 41 OF 43 | 5 QUESTIONS, ONE CODE TEMPLATE
Parent pointers: store who is above each node and climb

274. Tree Node. [E]

```
# BASE QUESTION (optimal): One CASE expression
# Idea: CASE tests in order: no parent means Root, an id
# that appears as someone's parent means Inner, anything
# else is a Leaf.
# Root: no parent. Inner: has a parent and a child.
# Leaf: has a parent and no child. The Root test comes
# first, so a one-node tree is a Root, not a Leaf.
TYPES_SQL = """
SELECT t.id,
  CASE
    WHEN t.p_id IS NULL THEN 'Root'
    WHEN EXISTS (SELECT 1 FROM tree_node_table c
                 WHERE c.p_id = t.id) THEN 'Inner'
    ELSE 'Leaf'
  END AS node_types
FROM tree_node_table t
ORDER BY t.id
"""
```

275. Employee Hierarchy Add / Delete [M]

```
# BASE QUESTION (optimal): Parent map plus children sets
# Idea: Keep manager[emp] and reports[emp] (a set). Add
# inserts into both. Delete reattaches each report to the
# deleted node's manager and removes the node from its
# manager's set.
# Answer: Walk up from the proposed new manager through
# manager pointers; if you meet the employee being moved,
# the move would put a node under its own descendant, so
# reject it. O(depth).
class Org:
    def __init__(self, ceo):
        self.root = ceo
        self.manager = {ceo: None}
        self.reports = {ceo: set()}
        # hire order, used for "most senior" choices
        self.hired = {ceo: 0}
        self.clock = 1

    def add(self, emp, mgr):
        if emp in self.manager or mgr not in self.manager:
            return False
        self.manager[emp] = mgr
        self.reports[emp] = set()
        self.reports[mgr].add(emp)
        self.hired[emp] = self.clock
        self.clock += 1
        return True

    def senior(self, people):
        # the person hired earliest
        return min(people, key=lambda p: self.hired[p])

    def delete(self, emp, policy="grandparent"):
        if emp not in self.manager:
            return False
        mgr = self.manager[emp]
        kids = set(self.reports[emp])
        # FOLLOW-UP 3: the CEO leaves
        if mgr is None and not kids:
            # the CEO alone: nothing could take over
            return False
        # FOLLOW-UP 2: promote the most senior report
        if policy == "senior" or mgr is None:
            # the most senior report takes the place of
            # emp
            heir = self.senior(kids) if kids else None
```

```
        else:
            heir = mgr
        for r in kids:
            if r != heir:
                self.reassign(r, heir)
        if mgr is None:
            # the promoted report becomes the new CEO
            self.root = heir
            self.manager[heir] = None
        elif policy == "senior" and heir != mgr:
            self.reassign(heir, mgr)
        if mgr is not None:
            self.reports[mgr].discard(emp)
        del self.manager[emp], self.reports[emp]
        return True

    def reassign(self, emp, new_mgr):
        old = self.manager[emp]
        if old is not None and old in self.reports:
            self.reports[old].discard(emp)
        self.manager[emp] = new_mgr
        self.reports[new_mgr].add(emp)

    # FOLLOW-UP 1: move an employee, reject cycles
    def move(self, emp, new_mgr):
        # walk up from the new manager: meeting emp is a
        # cycle
        node = new_mgr
        while node is not None:
            if node == emp:
                return False
            node = self.manager[node]
        self.reassign(emp, new_mgr)
        return True
```

276. Validate parent array forms a tree [M]

```
# HELPERS for the base question
# Answer: Return which rule failed: number of roots, an
# out-of-range parent, or the node where the cycle was
# detected (the node that was met twice on one walk).
def reason(parent):
    # None if valid, else the first rule that fails
    n = len(parent)
    roots = parent.count(-1)
    if roots != 1:
        return f"{roots} roots (need exactly 1)"
    for i, p in enumerate(parent):
        if p != -1 and not 0 <= p < n:
            return f"node {i}: out-of-range parent {p}"
        if p == i:
            return f"node {i} is its own parent"
    # 0 = unseen, 1 = on the current walk, 2 = checked
    state = [0] * n
    for start in range(n):
        path, x = [], start
        while x != -1 and state[x] == 0:
            state[x] = 1
            path.append(x)
            x = parent[x]
        # meeting a node twice on one walk is a cycle
        if x != -1 and state[x] == 1:
            return f"cycle through node {x}"
        for v in path:
            state[v] = 2
    return None


# BASE QUESTION (optimal): One root, then colour each node
# once while walking up
# Idea: Check there is exactly one -1 and all other values
# are in range. Then walk up from each unvisited node
# marking nodes as 'on this walk'.
```

```
def validate_parent_array(parent):
    return reason(parent) is None
```

277. Lowest Common Ancestor of a Binary Tree III [M]

```
# HELPERS for the base question
def lca_set(p, q):
    seen = set()
    while p:
        seen.add(id(p))
        p = p.parent
    while q:
        if id(q) in seen:
            return q
        q = q.parent
    return None


# BASE QUESTION (optimal): Two pointers that switch to the
# other node at the end
# Idea: Move a from p and b from q one step up per round.
# When a pointer falls off the root, restart it at the
# other node.
def lca_swap(p, q):
    # two pointers climb to the root; at the top each one
    # jumps to the other node's start. Both walk the same
    # total distance, so they meet at the LCA (or both
    # reach
    # None together when the nodes are in different
    # trees).
    a, b = p, q
    while a is not b:
        a = a.parent if a else q
        b = b.parent if b else p
    return a
```

278. Root a Tree So It Becomes a Binary Tree [M]

```
# BASE QUESTION (optimal): First node of degree at most 2
# Idea: Only the root has as many children as its degree;
# every other node has one fewer.
def find_binary_tree_root(n, edges):
    degree = [0] * n
    for a, b in edges:
        degree[a] += 1
        degree[b] += 1
    for node in range(n):
        if degree[node] <= 2:
            return node
    return -1
```

GROUP 42 OF 43 | 5 QUESTIONS, ONE CODE TEMPLATE
Build or transform a grid with index arithmetic

279. Filled ASCII Diamond [E]

```
# BASE QUESTION (optimal): Build each row from its width
# Idea: Row i has 2 * (R - |i - R|) + 1 x's with |i - R|
# underscores on each side.
def render_diamond(R):
    rows = []
    for i in range(2 * R + 1):
        pad = abs(i - R)
        rows.append(
            "_" * pad
            + "x" * (2 * (R - pad) + 1)
            + "_" * pad
        )
    return rows
```

280. Connected Crop Arrangement on a Grid [M]

```
# HELPERS for the base question
def snake_cells(n, m):
    # row 0 left to right, row 1 right to left, ...
    return [(i, j if i % 2 == 0 else m - 1 - j)
            for i in range(n) for j in range(m)]


def fill_along(grid, path, counts):
    # hand out the path to crop 1, crop 2, ... in turn
    k = 0
    for crop, cnt in enumerate(counts, 1):
        for _ in range(cnt):
            r, c = path[k]
            grid[r][c] = crop
            k += 1
    return grid


# BASE QUESTION (optimal): Fill crops along a snake path
# Idea: Generate the snake order of the cells and hand out
# count[0] cells to crop 1, the next count[1] to crop 2,
# and so on. Consecutive cells of the snake share an edge,
# so each crop is connected.
def arrange_crops(n, m, counts):
    # every cell is used exactly once
    assert sum(counts) == n * m
    grid = [[0] * m for _ in range(n)]
    # touching path cells keep each crop connected
    return fill_along(grid, snake_cells(n, m), counts)
```

281. Rotate Any m×n 2D Matrix [M]

```
# BASE QUESTION (optimal): Rotate 90 degrees clockwise by
# reverse + transpose
# Idea: Rotating clockwise puts the last row first, so
# reverse the rows and transpose (zip with *). A non-
# square matrix changes shape from m x n to n x m, which
# is why it cannot be done in place the way LeetCode 48
# is.
def rotate_cw(matrix):
    # reverse the rows, then swap rows and columns
    return [list(row) for row in zip(*matrix[::-1])]
```

282. Solve matrix groups and recipe inventory [M]

```
# BASE QUESTION (optimal): Blocks: one pass updating a
# minimum per block
# Idea: Each cell updates the minimum of block (r // k, c
# // k). No block list is built; the last blocks may be
# smaller when k does not divide the size.
def block_mins(matrix, k):
    # the minimum of every k x k block; edge blocks may be
    # smaller, which r // k and c // k handle by
    # themselves
    m, n = len(matrix), len(matrix[0])
    rows, cols = (m + k - 1) // k, (n + k - 1) // k
    out = [[float("inf")] * cols for _ in range(rows)]
    for r in range(m):
        for c in range(n):
            cell = out[r // k][c // k]
            out[r // k][c // k] = min(cell, matrix[r][c])
    return out
```

283. Range Sum Query 2D (Immutable Matrix) [M]

```
# BASE QUESTION (optimal): 2D prefix sums and inclusion-
# exclusion
# Idea: P holds the sum of the rectangle from the top-left
# corner; a query combines four entries.
class NumMatrix:
    def __init__(self, matrix):
        rows, cols = len(matrix), (
            len(matrix[0]) if matrix else 0
        )
        self.p = [
            [0] * (cols + 1) for _ in range(rows + 1)
        ]
        for r in range(rows):
            for c in range(cols):
                # the cell plus the block above and the
                # block to the left,
                # minus the corner block that was counted
                # twice
                self.p[r + 1][c + 1] = (
                    matrix[r][c]
                    + self.p[r][c + 1]
                    + self.p[r + 1][c]
                    - self.p[r][c]
                )

    def sum_region(self, r1, c1, r2, c2):
        p = self.p
        return (
            p[r2 + 1][c2 + 1]
            - p[r1][c2 + 1]
            - p[r2 + 1][c1]
            + p[r1][c1]
        )
```

GROUP 43 OF 43 | 5 QUESTIONS, ONE CODE TEMPLATE
Split into pieces, fix each piece, join (and pack words into lines)

284. Implement string formatting and request tracking tasks [E]

```
from collections import deque


# HELPERS for the base question
ROTATE = {"0": "0", "1": "1", "6": "9",
          "8": "8", "9": "6"}


# BASE QUESTION (optimal): snake_case to camelCase
# Idea: Split on '_' and drop empty parts, which handles
# leading, trailing and repeated underscores in one move.
# The first word stays lowercase and every later word is
# capitalized.
def snake_to_camel(s):
    # foo_bar -> fooBar (empty parts are ignored)
    words = [w for w in s.split("_") if w]
    if not words:
        return ""
    rest = [w.capitalize() for w in words[1:]]
    return words[0] + "".join(rest)


# BASE QUESTION (optimal): snake_case to camelCase
# Idea: Split on '_' and drop empty parts, which handles
# leading, trailing and repeated underscores in one move.
# The first word stays lowercase and every later word is
# capitalized.
# Answer: Parameterise the separator and the group size:
# the lowest group may differ from the others (3 digits,
# then groups of 2 in the Indian system). The loop peels a
# group from the right each time.
def format_number(n, sep=",", sizes=(3,)):
    # FOLLOW-UP 5: sep= and sizes=(3, 2) for other locales
    # group the digits from the right: 1234567 ->
    # 1,234,567.
    # sizes = (3, 2) is the Indian style 12,34,567: the
    # first
    # group has 3 digits, every later one has 2.
    sign = "-" if n < 0 else ""
    digits = str(abs(n))
    parts, i = [], 0
    while len(digits) > sizes[min(i, len(sizes) - 1)]:
        size = sizes[min(i, len(sizes) - 1)]
        # peel one group off the right end
        parts.append(digits[-size:])
        digits = digits[:-size]
        i += 1
    parts.append(digits)
    return sign + sep.join(reversed(parts))


# BASE QUESTION (optimal): snake_case to camelCase
# Idea: Split on '_' and drop empty parts, which handles
# leading, trailing and repeated underscores in one move.
# The first word stays lowercase and every later word is
# capitalized.
def is_rotation_valid(s, mapping=ROTATE):
    # does the number read the same upside down (180
    # degrees)?
    # a character not in the map makes it invalid
    if any(ch not in mapping for ch in s):
        return False
    # rotate every digit and read the result backwards
    turned = "".join(mapping[ch] for ch in reversed(s))
    return turned == s
# BASE QUESTION (optimal): snake_case to camelCase
```

```
# Idea: Split on '_' and drop empty parts, which handles
# leading, trailing and repeated underscores in one move.
# The first word stays lowercase and every later word is
# capitalized.
class RequestTracker:
    """begin(id, t) and end(id, t). Finished requests are
    printed in the order they BEGAN: one is printed only
    when everything that began before it is done."""

    def __init__(self):
        self.order = deque()
        self.begin_at, self.end_at = {}, {}

    def begin(self, rid, t):
        if rid in self.begin_at:
            raise ValueError("duplicate begin")
        self.begin_at[rid] = t
        self.order.append(rid)

    def end(self, rid, t):
        if rid not in self.begin_at:
            raise ValueError("end without begin")
        if t < self.begin_at[rid]:
            raise ValueError("ends before it begins")
        self.end_at[rid] = t
        # print from the front while the oldest are
        # finished
        out = []
        while self.order and self.order[0] in self.end_at:
            first = self.order.popleft()
            out.append((first, self.begin_at[first],
                        self.end_at[first]))
        return out
```

285. Snake Case to Camel Case with Repository-Wide Refactor [M]

```
# BASE QUESTION (optimal): Convert names, rename only
# defined identifiers via tokens
# Idea: snake_to_camel capitalises each part after the
# first. owned_names collects the functions, arguments and
# variables the code defines (using the syntax tree),
# minus an allow-list of names that must stay.
def snake_to_camel(name):
    # foo_bar -> fooBar (input is a valid snake_case name)
    head, *rest = name.split("_")
    caps = [p[:1].upper() + p[1:] for p in rest]
    return head + "".join(caps)
```

286. Google Doc Line Wrap (Word Wrap with Whitespace and Newlines) [M]

```
# HELPERS for the base question
# Answer: Keep the same loop but append the finished line
# to a list. Build
# each line from a list of words and join once, so each
# word is copied once.
def wrap_lines(text, width, break_long=False):
    lines = []
    # a newline always starts a new paragraph (and a new
    # line)
    for paragraph in text.split("\n"):
        cur = ""
        words = paragraph.split()
        if break_long:
            pieces = [split_long(w, width) for w in words]
            words = [p for chunk in pieces for p in chunk]
        for w in words:
            if not cur:
                cur = w
            elif len(cur) + 1 + len(w) <= width:
                # the word and one space still fit
                cur += " " + w
            else:
                lines.append(cur)
                cur = w
```

```
        # the last (or only, or empty) line of the
        # paragraph
        lines.append(cur)
    return lines


# Answer: Before placing a word, while it is longer than
# the width flush the current line and emit width-sized
# chunks of the word, then continue with the remainder as
# a normal word.
def split_long(word, width):
    # a word longer than the line is cut into pieces
    starts = range(0, len(word), width)
    return [word[i:i + width] for i in starts]


# BASE QUESTION (optimal): Greedy with a running line
# length
# Idea: Split on newlines first (each paragraph ends its
# line), then for each word compare cur_len + 1 +
# len(word) with the width. Only the length is tracked,
# never the joined line.
def wrap_count(text, width):
    return len(wrap_lines(text, width))
```

287. Implement a function to justify a list of words within a given width. [H]

```
# HELPERS for the base question
def left_align(line, width, measure=len):
    text = " ".join(line)
    return text + " " * (width - measure(text))


def center(line, width, measure=len):
    # leftover spaces split on both sides, the odd one
    # right
    text = " ".join(line)
    spare = width - measure(text)
    left = spare // 2
    return " " * left + text + " " * (spare - left)


# Answer: Replace len(word) by a measure(word) function
# (for example font metrics or East Asian width) and
# compare against the pixel width.
# Answer: Replace len(word) with a measure function and
# compare against the pixel width; the extra space is
# distributed in whole space units, so the remainder rule
# stays the same.
def pack_greedy(words, width, measure=len):
    # put as many words as fit on every line
    lines, line, used = [], [], 0
    for w in words:
        size = measure(w)
        # `used` counts the words and one space between
        # each
        if line and used + 1 + size > width:
            lines.append(line)
            line, used = [], 0
        used += size + (1 if line else 0)
        line.append(w)
    if line:
        lines.append(line)
    return lines


# Answer: Replace len(word) by a measure(word) function
# (for example font metrics or East Asian width) and
# compare against the pixel width.
# Answer: Replace len(word) with a measure function and
# compare against the pixel width; the extra space is
# distributed in whole space units, so the remainder rule
# stays the same.
```

```
def spread(line, width, measure=len):
    # fully justified: spaces between the words, extra
    # ones
    # go to the LEFT gaps first. One word stays left-
    # aligned.
    if len(line) == 1:
        return line[0] + " " * (width - measure(line[0]))
    spaces = width - sum(measure(w) for w in line)
    gaps = len(line) - 1
    each, extra = divmod(spaces, gaps)
    out = ""
    for i, w in enumerate(line[:-1]):
        out += w + " " * (each + (1 if i < extra else 0))
    return out + line[-1]


# BASE QUESTION (optimal): Greedy line packing
# Idea: Pack as many words as fit on a line (counting one
# space between words). Spread the spare spaces over the
# gaps, giving the extra ones to the leftmost gaps.
# Answer: Only the final-line branch changes. For full
# justification drop the special case for the last line
# (it would be spread like every other line).
# Answer: Drop the special case for the last line (a line
# with more than one word is spread like the others).
# Single-word lines stay left-aligned.
def justify(words, width, last="left", measure=len,
            lines=None):
    # last: "left" (the classic rule), "full" (spread it
    # too)
    # or "center". `lines` lets the caller bring its own
    # line breaks (see pack_optimal).
    lines = lines or pack_greedy(words, width, measure)
    out = []
    for i, line in enumerate(lines):
        if i < len(lines) - 1:
            out.append(spread(line, width, measure))
        # FOLLOW-UP 1, 3: justify the last line too
        elif last == "full":
            out.append(spread(line, width, measure))
        # FOLLOW-UP 1: centre the last line
        elif last == "center":
            out.append(center(line, width, measure))
        else:
            out.append(left_align(line, width, measure))
    return out
```

288. Convert Valid Snake Case to Lower Camel Case [H]

```
# BASE QUESTION (optimal): Split on underscores and
# capitalize
# Idea: Split on '_', keep the first word, and capitalize
# the first letter of each other word with a join. For
# valid input there are no empty parts.
def snake_to_camel(s):
    # foo_bar_baz -> fooBarBaz
    first, *rest = s.split("_")
    caps = [w[:1].upper() + w[1:] for w in rest]
    return first + "".join(caps)
```